

A Systematic Review of Automated Query Reformulations in Source Code Search

MOHAMMAD MASUDUR RAHMAN, Dalhousie University, Canada
 CHANCHAL K. ROY, University of Saskatchewan, Canada

Fixing software bugs and adding new features are two of the major maintenance tasks. Software bugs and features are reported as change requests. Developers consult these requests and often choose a few keywords from them as an ad hoc query. Then they execute the query with a search engine to find the exact locations within software code that need to be changed. Unfortunately, even experienced developers often fail to choose appropriate queries, which leads to costly trials and errors during a code search. Over the years, many studies have attempted to reformulate the ad hoc queries from developers to support them. In this systematic literature review, we carefully select 70 primary studies on query reformulations from 2,970 candidate studies, perform an in-depth qualitative analysis (e.g., Grounded Theory), and then answer seven research questions with major findings. First, to date, eight major methodologies (e.g., term weighting, term co-occurrence analysis, thesaurus lookup) have been adopted to reformulate queries. Second, the existing studies suffer from several major limitations (e.g., lack of generalizability, the vocabulary mismatch problem, subjective bias) that might prevent their wide adoption. Finally, we discuss the best practices and future opportunities to advance the state of research in search query reformulations.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**; **Traceability**; **Maintaining software**; *Search-based software engineering*; *Software reverse engineering*;

Additional Key Words and Phrases: Concept location, bug localization, Internet-scale code search, automated query reformulation, term weighting, query quality analysis, machine learning, systematic literature review

ACM Reference format:

Mohammad Masudur Rahman and Chanchal K. Roy. 2023. A Systematic Review of Automated Query Reformulations in Source Code Search. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 159 (September 2023), 79 pages.

<https://doi.org/10.1145/3607179>

1 INTRODUCTION

Software maintenance costs up to 80% of the total budget in modern software development [50, 59]. One of the major aspects of maintenance is to deal with software bugs and failures, which cost the global economy billions of dollars every year [3]. A few of these bugs could even lead

The work of M. M. Rahman was partly completed as a Ph.D. candidate at the University of Saskatchewan, Canada.

This research was supported by a Tenure-track startup grant, Dalhousie University, an International Dean's Scholarship from University of Saskatchewan (2014–2017), a Saskatchewan Innovation & Opportunity Scholarship (2017–2018), and the Natural Sciences and Engineering Research Council of Canada (NSERC).

Authors' addresses: M. M. Rahman, Dalhousie University, Halifax, NS, B3H 1W5, Canada; email: masud.rahman@dal.ca; C. K. Roy, University of Saskatchewan, Saskatoon, SK, S7N 5C9, Canada; email: chanchal.roy@usask.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1049-331X/2023/09-ART159 \$15.00

<https://doi.org/10.1145/3607179>

to massive fatalities (e.g., Boeing 737 MAX crashes [4, 5, 7, 10], Therac-25 accidents).¹ Software developers thus deal with hundreds of bugs and failures to ensure the high quality of their software products [13]. They might also need to frequently enhance the features of their software to remain competitive in the market. We commonly refer to these maintenance tasks as *change tasks* for the sake of brevity. Each of these change tasks is triggered by a *software change request* either in the form of a bug report [195] or simply a feature request containing plain texts [71, 87, 153]. Developers consult these change requests to implement the required changes in the software code.

As a part of change implementation, the location of code that needs a change must be identified. Traditionally, various static and dynamic analyses have been used for this task. Since the past decade, various code search techniques, especially powered by Information Retrieval (IR) methods, have drawn the attention of the software engineering community. To find the location of code requiring a change, developers often choose a few important keywords from a change request as an ad hoc query. Then they execute the query with a code search engine (e.g., Lucene [6]) and attempt to find the exact locations within the software code that need to be changed as a part of fixing bugs or enhancement of features. Unfortunately, this has been challenging, and even experienced developers often fail to choose the right search queries from a change request [87]. As a result, they experience difficulties in detecting the appropriate locations within the code and spend the majority of their time in numerous trials and errors [87, 95, 111]. One might think of using all keywords of a change request as a query. However, the change requests are not originally written to be used as search queries. Thus, they often make verbose queries, which are noisy and ineffective [38]. The developers might also attempt to improve their ad hoc queries by adding suitable keywords from the software code. However, Furnas et al. [53] suggest that there is little chance (e.g., 10%–15%) that they might guess the right keywords from the software code. In addition, the software code has an unrestricted vocabulary that evolves over time due to frequent code-level changes (e.g., addition of new features) [69]. In short, selecting the right keywords either from the change requests or from the software code is a major challenge during code search. Thus, developers are badly in need of automated supports for constructing appropriate search queries and detecting relevant software code during maintenance.

Developers extensively search for code within a local codebase when dealing with bugs and features. However, they also frequently search for relevant code snippets on the Internet (e.g., GitHub [9]) to accomplish various programming tasks including code reuse, fixing bugs, and feature enhancement [103]. According to an earlier study [26], developers spend about 19% of their programming time searching for relevant code on the Internet. During this code search, they also choose ad hoc search queries using a few keywords. However, 76% of their queries need one or more reformulations to succeed [15, 17]. Thus, regardless of the problem context, choosing the right queries and locating the desired code are challenging, which warrants appropriate automated supports for the developers.

Automated support for constructing queries and then for searching the code of interest (e.g., software bugs, features) has been an active area of research for decades. Several studies [35, 38, 54, 63, 65, 71, 75, 87, 153–156, 158, 176, 183, 206] attempt to support developers either by constructing search queries for them from a change request or by reformulating their chosen ad hoc queries. These studies adopt various methodologies and algorithms as follows. First, there have been several studies that construct queries from a change request by employing term weighting algorithms [47, 87, 156, 158, 212], natural language discourse analysis [35], and meaningful heuristics [87]. Second, there have been other studies that reformulate (or reconstruct) a given search query by employing the relevance feedback mechanism [54, 63, 65, 115, 155, 183], spatial code proximity

¹<https://bit.ly/2KU9IR2>.

analysis [125, 181, 183], query difficulty estimation [61, 63, 64, 155], term-query co-occurrence analysis [71, 154, 208], thesaurus lookup [71, 176], and software repository mining [75, 154, 206, 208]. Third, there has been another group of studies that reformulate queries for Internet-scale code search by capturing complementary keywords from crowdsourced contents [135, 213], well-established thesauri (e.g., WordNet) [103, 104, 114], and software repositories [34, 121, 123, 135, 159, 166]. Thus, there has been a significant number of studies that *reformulate* queries to support source code search. Unfortunately, to the best of our knowledge, there exists no systematic literature review that analyzes, categorizes, or critically examines these studies, which is essential to advance the state of research on this topic—*automated query reformulations for source code search*.

Contributions. In this systematic literature review, we investigate the existing research on automated query reformulations supporting code search that were conducted during the past 15+ years. We carefully select 70 primary studies out of 2,970 candidates that were collected from 11 widely used publication databases (e.g., ACM Digital Library, IEEE Xplore; see Figure 4). Then we critically examine each of these primary studies by going through their full texts and by identifying their methodologies for reformulating queries, evaluations, and limitations. We apply the *Grounded Theory approach* in our analysis using three levels of coding (open, axial, and selective) [58]. The method has been widely used for literature reviews since it can derive important theories that are firmly grounded within the qualitative data [35, 93, 185]. Using this method, we categorize our selected studies into several categories in terms of their adopted methodologies and reported limitations. We also provide statistical evidence of conducted research over the past two decades. We also compare and contrast between query reformulations in local code searches and that of Internet-scale code search. Finally, we discuss several recommended practices and open questions in automated query reformulations supporting code search, and suggest the future research directions in this domain. We thus make the following contributions in this article:

- (a) (RQ_1) *Taxonomy based on methodology and algorithm*: We identify the methodologies or algorithms used by primary studies to reformulate their queries and classify the studies based on these dimensions. We found that, to date, eight major methodologies and algorithms have been used to reformulate queries for code search (see Table 2). About 40% of these studies use term weighting and the relevance feedback mechanism, and 46% make use of semantic relations, word co-occurrences, and thesaurus lookup, whereas 44% of the studies rely on data mining and API recommendations to reformulate their search queries.
- (b) (RQ_2) *Evaluation and validation*: We analyze the evaluation methods, performance metrics, and validation targets used by the primary studies and report our findings. About 64% of our primary studies use re-enactment-based evaluation, 26% perform developer studies, and 9% of the studies use a combination of both methods. They use a total of 30 performance metrics where a few metrics (e.g., Hit@K, Mean Average Precision (MAP), recall) (see Figure 12) were frequently used. However, human participation is generally low (see Figure 19), and the number of queries used to evaluate queries in the Internet-scale code search is often small (see Table 6). About 30% of our primary studies were also selected for comparison by the later studies.
- (c) (RQ_3) *Limitations and challenges of primary studies*: We analyze the limitations and challenges of the primary studies that were indicated either explicitly or implicitly in their papers. We find that their approaches suffer from eight major limitations or challenges (see Table 9), including noisy keywords in their queries, the vocabulary mismatch problem, lack of generalizability, human-induced biases, weak evaluation, and other prevailing issues that might prevent them from adoption by software practitioners.
- (d) (RQ_5) *Similarities and differences between local and Internet-scale code search*: About 58% of our 70 primary studies perform query reformulations to support local code searches

(e.g., concept location, bug localization, feature location), whereas the remaining 42% focus on Internet-scale code search. The primary studies from these two searches differ in their adopted methodologies (see Figure 25), reformulation types (see Figure 26), and evaluation methods (see Figure 28). However, surprisingly, both groups of studies suffer from a common set of issues such as lack of generalizability, weak evaluation, and noisy queries (see Figure 30). Future work on software code search can benefit from these empirical insights.

- (e) (RQ_6) *Recommended practices in query reformulations for code search*: We identify several recommended practices in query reformulations based on experimental evidence and the general understanding of the software engineering community. For example, collecting queries from professional developers [63] and selecting both open and closed source systems [88, 139, 140] are recommended for designing an experiment. Reformulated queries should be executed against multiple search engines [132] to ensure a sound evaluation. Similarly, each proposed technique should be compared against state-of-the-art techniques through replication [66, 83]. Furthermore, the replication package should be shared publicly to advance the current state of knowledge [84].
- (f) (RQ_7) *Open challenges and research opportunities*: We identify several open challenges in automated query reformulations supporting code search and determine the scope of future work (Section 5). We found that the state-of-the-art keyword selection algorithms might not be enough to identify appropriate keywords either from change requests or from software code. Future work can focus on adding more contexts (e.g., time-awareness) to the keyword selection algorithms, designing an appropriate fitness function for GA-based solutions [131], leveraging the structures from source code [108, 155], or using neural language modeling (e.g., word embeddings [23, 127, 208]) to deliver better quality queries for the code search.
- (g) *Replication package*: We provide a replication package² containing experimental metadata to reproduce our findings. It contains a comprehensive collection of 70 primary studies on automated query reformulations supporting code search that were conducted over the past 15+ years (see Table 11). They were carefully selected from a total of 2,970 candidate studies (and 11 publication databases) through six levels of noise filtration [93, 147] (see Figure 4, Section 3). We also present the statistical evidence of conducted research using various analyses (e.g., see Tables 11 and 12, and Figure 24) and discuss the recent trends of this topic.

Structure of the Article. The rest of this article is organized as follows. Section 2 presents the background concepts and terminologies of automated query reformulations. Section 3 discusses the methodology of our systematic literature review. Section 4 presents our review results. Section 4.1 focuses on the methods, algorithms, and data sources used in the primary studies. Section 4.2 discusses the evaluation methods of the primary studies, whereas Section 4.3 points out their limitations and challenges. Section 4.5 compares and contrasts between local code searches and Internet-scale code search in terms of their query reformulations. Section 4.6 focuses on the recommended practices. Section 5 discusses the open challenges and scopes for future research in this domain. Section 6 identifies the threats to validity of our findings, Section 7 discusses the related parallel surveys, and Section 8 concludes our article with the major findings.

2 BACKGROUND

2.1 Automated Query Reformulation

Searching for relevant code is a frequent activity in modern software development [26]. One major part of this search operation is to choose an appropriate query that reflects the information need. Unfortunately, during code search, software developers fail even 88% of the time to choose the

²<https://bit.ly/3QpZkSx>.

right search queries [87, 95]. When their search query fails, they attempt to reformulate the query by adding new keywords, by removing noisy keywords, or by replacing the existing keywords with more appropriate ones. When these modifications are performed using automated means, they are called *automated query reformulations* [63, 115, 176]. Thus, the query reformulation tasks are three types, as follows:

- (a) *Query expansion*: A given search query is expanded with synonyms, semantically similar keywords, or complementary keywords. Developers often need to expand their initial queries since they are generally short (e.g., two to three keywords on average) [48, 166]. According to existing studies [17, 166], up to 76% of developer queries need to be expanded during code search.
- (b) *Query reduction*: A given search query is reduced by removing the noisy, ambiguous, or less discriminating keywords. The keywords that occur in more than 25% of the documents in a corpus are less discriminating [63, 154]. They are often removed from a query during code search.
- (c) *Query replacement*: The keywords of a given query are replaced with more appropriate keywords. Examples of query replacement include spelling corrections [55], query generalization, and query specialization [114, 129]. During code search, developers often learn new information from their unsuccessful searches and then calibrate their search by replacing the old query keywords with more appropriate keywords.

2.2 Context of Query Reformulation

Context refers to the environment where a task is performed. It has been found to be useful for many software engineering tasks (e.g., code completion, exception handling) [14, 41, 56, 74, 152]. Similarly, the context plays a key role in query reformulation for the code search. In particular, the queries and their reformulation practices often depend on the type of code search for which they are intended. Based on the locality and type of a corpus, code searches can be divided into the following two major categories:

- (a) *Local code search*: The code is searched within a local codebase that contains only one software project. Figure 1 shows how queries are constructed from a change request in the context of local code search. The developer selects either the whole texts or a few important keywords from the change request as an ad hoc query (see steps 1–4 in Figure 1). Then the query is executed by a code search engine that returns the source code that needs to be modified (see steps 5 and 6 in Figure 1). If the query fails, it is reformulated with automated techniques by carefully analyzing the change request texts and the retrieved source code (see step 7 in Figure 1). There also have been several attempts [35, 87, 156] to support the developer with an initial query from the change request to avoid frequent trials and errors during the code search (see step 3 in Figure 1). Software bugs and features are located within source code using local code search.
- (b) *Internet-scale code search*: The code is searched within an Internet-scale codebase that contains thousands, if not millions, of software projects (e.g., GitHub,³ SourceForge⁴). Figure 2 shows how search queries are reformulated in the context of Internet-scale code search. The developer selects a few keywords as an ad hoc query and then attempts to find out the relevant code from the corpus using a search engine (see steps 1–4 in Figure 2). If the query fails, it is reformulated with automated techniques using thesaurus lookup [114, 159, 213]

³<https://github.com>.

⁴<https://sourceforge.net/>.

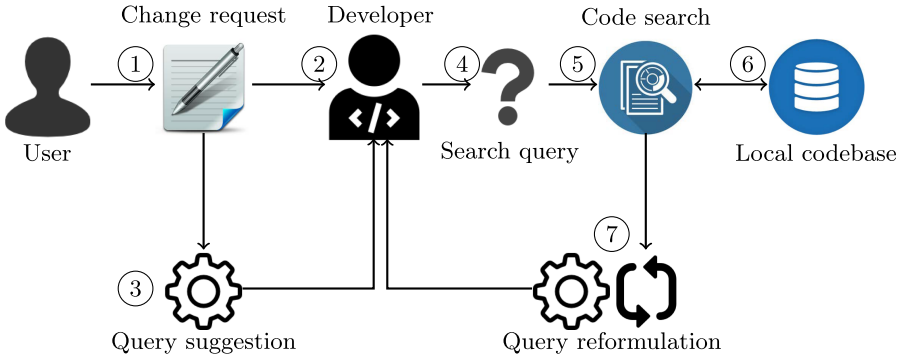


Fig. 1. Automatic query reformulations in local code search.

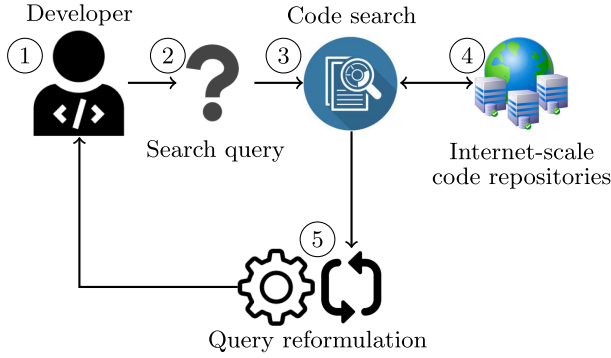


Fig. 2. Automatic query reformulations in Internet-scale code search.

or intent refinement [78]. Software developers spend about 19% of their programming time searching for relevant, reusable code snippets from the Internet [26].

2.3 Steps of Automated Query Reformulation

During code search, queries are reformulated in multiple steps. These steps might vary based on either type of reformulation or type of code search. However, most existing approaches [54, 63, 64, 155, 156, 183] share a common set of steps (Figure 3) as follows:

- (a) *Query feedback collection*: The first step of query reformulation is to collect feedback on a query (see steps 2–4 in Figure 3). One way to collect feedback is the *relevance feedback method* [169], which has been widely used in the IR domain. In this method, each of the documents retrieved by a given query is examined and annotated as either *relevant* or *irrelevant* to the query [54, 115]. The relevance feedback can be of three types, as follows:
 - *Explicit relevance feedback*: Developers provide *explicit* feedback and annotate each of the retrieved documents (by a search query) as either relevant or irrelevant [54]. Although the explicit feedback could be accurate and meaningful, capturing it regularly from the developers is time consuming and sometimes even impossible.
 - *Implicit relevance feedback*: Since explicit feedback is costly, several studies capture low-cost feedback that is *implicitly* provided by the developers [85]. This feedback is derived from a developer's reactions toward the retrieved documents, such as eye movements, document examination patterns, and keyword deletion or retention patterns.

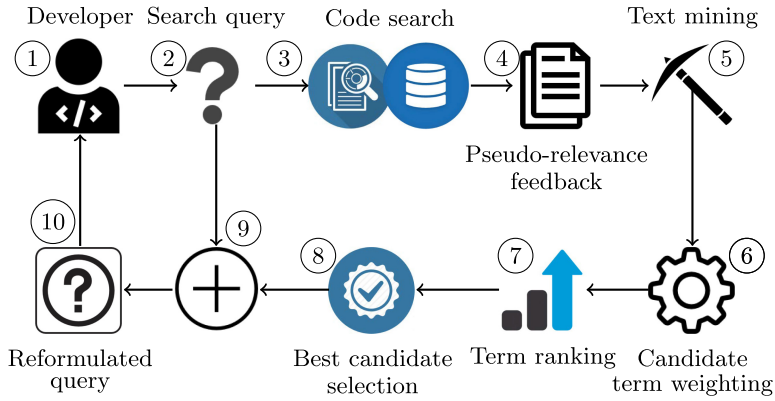


Fig. 3. Steps of automated query reformulation using PRF.

- *Pseudo-relevance feedback*: Unlike the preceding two types, Pseudo-Relevance Feedback (PRF) does not warrant any developer intervention. In other words, the top-K (e.g., $K = 5$) source documents retrieved by a given query are naively considered as *relevant* to the query [63, 71, 169]. It is also known as *blind relevance feedback*. There has been significant evidence [63, 154, 155, 158] that suggests the effectiveness of this relevance feedback mechanism in query reformulation.
- (b) *Candidate keyword selection*: Once relevance feedback is collected, the next step is to select appropriate keywords to reformulate the given query using the feedback. Many studies [63, 156, 183] analyze the annotated documents collected as a part of feedback and attempt to find important keywords from them using various term weighting algorithms (see steps 5 and 6 in Figure 3). Term weight is a numerical proxy to the relative importance of a keyword within a document [82]. Many of the term weighting algorithms (e.g., Section 2.4) are borrowed from the IR domain.
- (c) *Reformulation of the search query*: Once the candidate keywords are collected, they are ranked and the top few keywords are used to expand a given query (see step 7 in Figure 3). Less important keywords and the keywords non-existent in the corpus are discarded from the given query [63, 163]. A few studies [63, 130, 155] argue that the same term weighting algorithm might not always work. Thus, they construct multiple reformulated versions of a given query using multiple term weighting algorithms and then deliver the best-reformulated query using supervised machine learning (see steps 8–10 in Figure 3).

2.4 Term Weighting

Term weighting is a popular method to determine the relative importance of individual terms within any body of texts (e.g., change request, source code) [82]. It is often used to select candidate keywords during automated query reformulations (see step 6 in Figure 3). Several term weighting algorithms had been proposed and widely used over the past few decades. They can be classified into the following three broad categories:

- (a) *Frequency-based term weighting*: The importance of a keyword is determined based on its occurrence frequencies. The arguably most popular frequency-based method is TF-IDF [82], which stands for Term Frequency (TF) times Inverse Document Frequency (IDF). TF counts the occurrences of a term (or keyword) within a document, whereas DF counts the number of documents (within a corpus) containing the term. However, IDF, the inverse of document frequency, determines the informativeness of a term by penalizing the highly frequent terms

(e.g., stop words). Thus, TF-IDF determines a term's importance without considering the dependencies that it might have on other terms.

- (b) *Graph-based term weighting*: Unlike the frequency-based methods, graph-based methods [21, 156, 158] capture the dependencies among keywords during their weight calculation. Existing studies capture co-occurrences [126], syntactic dependencies [21], and hierarchical dependencies [158] among the keywords, and transform a text document into a graph structure. In this graph, the keywords are represented as nodes and their dependencies as edges. Then the important nodes (or keywords) are identified using a graph-based, recursive algorithm, namely PageRank [27]. Source code has an unrestricted, evolving vocabulary that could be larger than that of regular texts [69]. Furthermore, the code is rich in terms of structures and dependencies (e.g., static relations). Thus, graph-based algorithms leveraging dependencies could be a suitable choice to select important keywords from the source code during query reformulations [155, 158].
- (c) *Probabilistic term weighting*: Unlike the preceding two types, probabilistic term weighting methods make use of Information Theory to calculate the term weights [28, 30, 32]. One of the popular probabilistic methods is Kullback–Leibler Divergence, which determines the divergence of a keyword (within a document) from the random occurrence probability. In other words, if a keyword is frequent across numerous documents within a corpus (a.k.a. a generic keyword), it does not contain unique information, which makes it less important. On the contrary, a keyword that is specific to only a few documents often contains meaningful information, which makes it important.

2.5 Implications of Automated Query Reformulation

Choosing the right queries during code search is a challenging problem. Hence, automated supports for constructing queries are highly warranted for developers. However, automatic reformulations of queries have both positive and negative implications, as follows:

- (a) *Benefits of query reformulation*: Developers often use short queries to search for relevant code, which might not always reflect their information needs [48]. Thus, expanding these short queries with similar or complementary keywords often improves their performance. According to existing literature [160], $\approx 20\%$ performance gain could be achieved in code search using automatically reformulated queries. Carefully reformulated queries might also help developers locate their desired code with reduced cognitive or manual efforts.
- (b) *Costs of query reformulation*: Automatic reformulations might hurt the search queries that are already good (or appropriate) [31, 63, 155]. Adding extra keywords makes them noisy [179]. Thus, no reformulation is better for them than inappropriate reformulations [140, 179, 184]. There also exist a few difficult queries that cannot be improved using traditional reformulation approaches [31, 57].

Given these twofold implications of query reformulations, contemporary approaches attempt to maximize the benefits and minimize the costs of reformulation. For example, several studies [63, 64, 155] adopt supervised machine learning and query quality analysis to improve the poor queries and to preserve the good queries during the code search.

3 METHODOLOGY

We conduct a systematic literature review [93, 94, 147] on the topic of query reformulations for code search. We start our investigation by asking seven research questions, then provide a comprehensive analysis of the topic—*automated query reformulations* in code search. We provide empirical evidence of the relevant research and establish the state-of-the-art practices. Our goals

Table 1. Research Questions

Ref#	Question
General questions	
RQ ₁	Which methodologies (e.g., methods, algorithms) have been used for automated query reformulations targeting code search in the literature?
RQ ₂	Which methods, metrics, or subject systems have been used to evaluate and validate the research on automated query reformulations?
RQ ₃	What are the major challenges of automated query reformulations intended for code search? How many of them have been solved to date by the literature?
Statistical questions	
RQ ₄	How many research articles have been published to date on automated query reformulations? What are the venues in which the research was published?
Focused questions	
RQ ₅	What are the differences and similarities between query reformulations for local code search and query reformulations for Internet-scale code search?
RQ ₆	What are the recommended practices in query reformulation for source code search?
RQ ₇	What are the scopes for future work in the area of automated query reformulation targeting the code search?

are to summarize the current state of research in this domain and to outline future research directions. We refer to the guidelines of Kitchenham and Charters [94] for our systematic review as follows.

3.1 Research Questions

Specifying the research questions is one of the most important steps of the systematic literature review [93, 94]. We ask seven research questions in our survey. Our questions are classified into three categories: *general questions*, *statistical questions*, and *focused questions*. Table 1 shows our seven research questions.

Our general questions target the *generic* aspects of query reformulation approaches such as their algorithms, methodologies, or used corpora (RQ₁), their evaluation and validation methods (RQ₂), and the challenges or limitations they experience (RQ₃). Our statistical questions attempt to gather the *statistical* evidence of the conducted research over the years (RQ₄). Finally, our focused questions concentrate on the *specific* aspects of automated query reformulations targeting code search. They compare between local code search and Internet-scale code search in terms of their query reformulation practices (RQ₅), identify the best practices in query reformulations for code search (RQ₆), and also identify future research directions (RQ₇).

3.2 Search Strategy

The next step of the systematic literature review is to collect a complete set of primary studies that can help us answer the research questions. This step involves the construction of appropriate search keywords on a research topic and then the accumulation of a non-biased set of studies from appropriate publication databases (e.g., IEEE Xplore, ACM Digital Library).

Construction of Search Keywords. Choosing appropriate keywords and optimizing them for search are crucial to the discovery of available studies on a research topic. Research questions determine the scopes and motivations of a systematic review. Kitchenham and Charters [94] suggest that these questions should be broken down into individual facets such as search keywords.

Then they should be expanded with their synonyms, abbreviations, and alternative spelling using Boolean operators. Earlier studies often use a PIO (Population + Intervention + Outcome) criterion [93, 142] to identify their search keywords. The PIO approach determines three types of keywords, presented next.

Population terms encompass all aspects of a research topic including its application scopes, technologies, and standards. In the context of automated query reformulation targeting code search, we thus consider the following terms as the population of search keywords.

Information retrieval, IR, text retrieval, TR, bug localization, concept location, feature location, FLT, concern location, Internet-scale code search, code search engine, search engine, local code search, code search, source code search, and code search query.

Intervention terms focus on the specific aspect of a topic under study. In the context of automated query reformulation, we thus consider the following terms as our intervention keywords.

Query reformulation, query expansion, query reduction, query formulation, query refinement, automated query expansion, AQE, query suggestion, query recommendation, term selection, query replacement, query difficulty, query quality, keyword selection, keyword extraction, search term identification, search query, search term, and search keyword.

Outcome terms are related to the factors that are of importance to the stakeholders (e.g., developers). In the context of query reformulation targeting code search, these outcomes could be the improvement in result ranks, reduced effort in code search, and reduced effort in localizing the bugs. However, we did not want to restrict our search too much. Thus, we do not include the *outcome* terms in our search keywords like the earlier study [93]. Finally, we construct our keyword set for the systematic literature review as Population AND Intervention.

Source of Information. Our goal was to collect as many as studies possible on the topic of interest from the literature. In this work, we choose 11 electronic publication databases for our study. These databases store most of the research conducted in the fields of Computer Science and Software Engineering [93]. In particular, we collect all journal articles and conference papers published between 1998 and 2021. To the best of our knowledge, 2004 was the year when the very first primary study (that of Marcus et al. [120]) was published. However, 1998 was the inception year of Google's PageRank algorithm [27], which had been frequently used since then both by IR [21, 126] and code search communities [123, 124]. We thus choose 1998 as the starting year of our investigation to avoid missing any relevant studies accidentally. Our investigation finishes with 2021 as the ending year. First, 2,871 candidate studies, conducted between 1998 and 2018, were collected as a part of the Ph.D. comprehensive examination of the first author. Recently, we included 99 more studies conducted between 2018 and 2021. We thus collect a total of 2,970 potentially relevant articles and papers on query reformulations targeting code search for our systematic review.

3.3 Study Selection

We exclude irrelevant studies from the 2,970 initial results and select only the relevant ones for our systematic review. We apply three different filters and choose as our primary studies the ones that meet certain conditions and quality standards. In the context of our systematic literature review, we carefully exclude such studies that

- C_1 : do not address *code search*,
- C_2 : do not address *query reformulation*,
- C_3 : do not address *query reformulation for code search*.

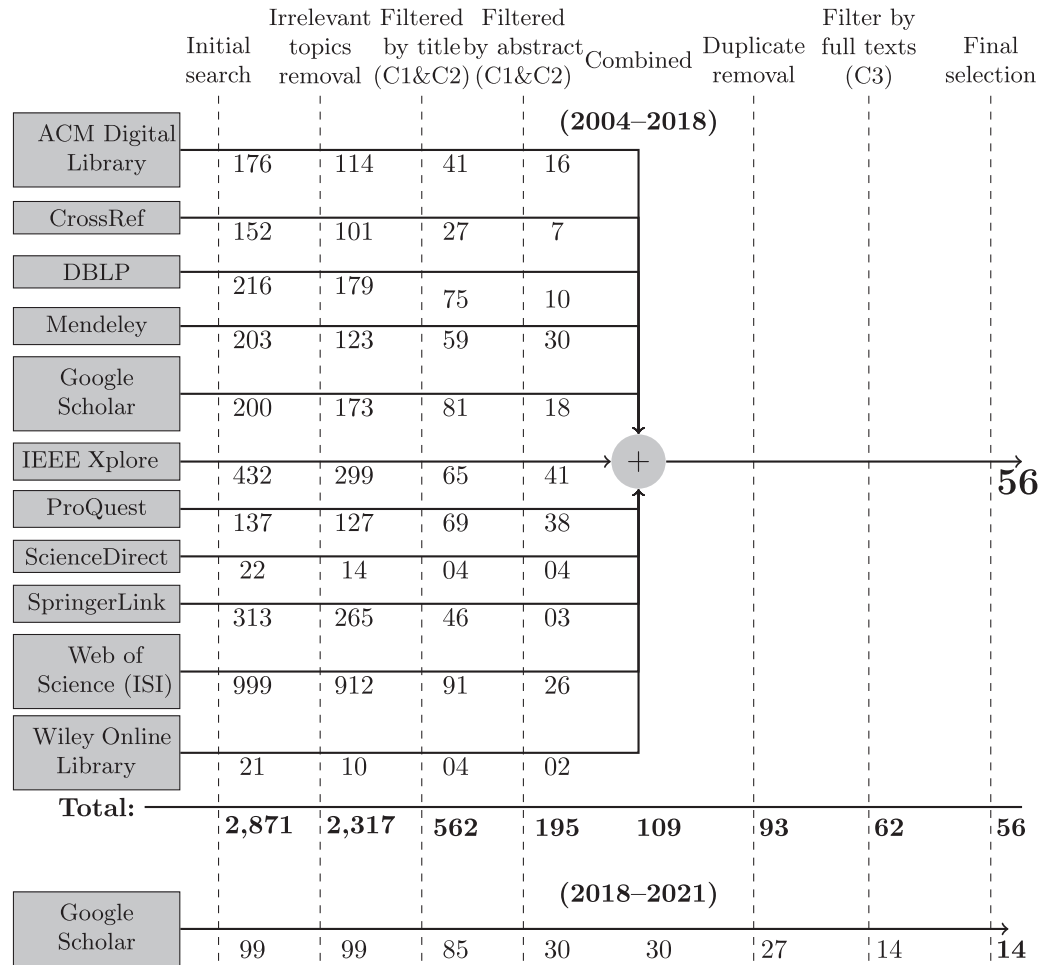


Fig. 4. Selection of primary studies.

Figure 4 shows different steps of our study selection process. We apply multiple filters to the initial search results across several steps and choose the final set as follows.

(a) Removal of Irrelevant Topics. Initial search results often contain entries that are not related to software engineering. This might happen due to an accidental keyword matching between the query and the full texts from the papers. We use a semi-automatic approach to remove these irrelevant studies and to ensure the quality of our results. First, we analyze the keywords from the *title* of each result and carefully identify the words that might be *irrelevant* to source code search and query reformulation. For example, we found that the results containing “library” or “libraries” in their title mostly deal with document search from digital libraries. However, the results containing “mobile” in their title mostly deal with mobile-based location search or recommendation, which is not related to our topic of interest (query reformulation in code search). Thus, we discard the studies that contain any of these keywords in their *title*.

Image, multimedia, multilingual, bilingual, video, library, libraries, digital, database, databases, db, chinese, japanese, trec, mobile, medical, geo, music, sql, audio, and speech

To ensure that we are not losing any relevant studies, we also analyze the discarded results by these keywords: {"library," "libraries," "mobile"}. We found that only 2 of their 62 results were related to code search but did not deal with any query reformulations, which shows the viability of our filtration step. Thus, we removed 554 (18.65%) irrelevant results, and our initial studies were reduced to 2,416 (81.35%) after this step.

(b) Noise Filtration Using the Title. After the removal of irrelevant topics, we are still left with thousands of results. Manually analyzing them for relevance is still impractical. We thus adopt another heuristic to reduce the noise from our results. Since we are interested about the studies related to query reformulation and code search, we discard any studies that *do not* contain at least one of the following important keywords in their *title*.

query, code, developer, change, bug, concept, feature, concern, and software

We believe that the studies that miss these keywords in their titles might not be relevant to our systematic review. It should be noted that we do not include "search" in the preceding list. We found that the keyword "search" might be quite generic and could lead to various irrelevant studies related to document search or location search rather than code search. This semi-automated filtration step discarded 1,769 (59.56%) irrelevant studies, and our collection was to 647 (21.78%) potentially relevant studies. We also repeat the filtration step including the "search" keyword in the preceding list, analyze the results from three publication databases (ACM Digital Library, IEEE Xplore, and Google Scholar) and found no change in the total number of primary studies.

(c) Noise Filtration Using the Abstract. Noise filtration using the title might not be sufficient enough, and the collection still could contain irrelevant studies. We thus manually analyze the titles from 647 studies and their corresponding abstracts to understand the underlying research. We noticed that despite having the desired keywords, many of these studies do not deal with query reformulations and code search. We thus manually discarded 422 (14.21%) studies, and our collection was reduced to 225 (7.58%) relevant studies.

(d) Result Merging and Duplicate Removal. In this step, we merge the relevant studies from each of the 11 publication databases and discard the duplicate studies. Since the same study could be indexed by multiple databases, the duplication might occur during retrieval. Even multiple versions of the same study (e.g., peer reviewed, preprint) could be retrieved from multiple databases. We thus manually analyzed 225 studies and discarded a total of 105 (3.54%) duplicate studies. This step left us with a total of 120 (93 + 27) relevant studies.

(e) Noise Filtration Using Full Texts. We were unable to determine the subject matter of several studies even after reading their titles and abstracts. We thus read the full texts of 120 research papers, and discarded several irrelevant studies. Many of these studies discuss either code search algorithms or query reformulations in general but not the techniques for query reformulations. Since our topic was *query reformulation for code search*, we considered them irrelevant to our survey and discarded them from the collection. This filtration left us with 76 studies. We also discarded short papers (e.g., posters, doctoral symposiums) and non-peer-reviewed items (e.g., technical reports, dissertations). However, a few short papers (e.g., new ideas, tools) were retained due to their strong relevance to our topic of interest. Since our focus was only on the *techniques* for query reformulations, we also discarded such studies that conducted developer surveys, interviews, or case studies on concept location, feature location, and Internet-scale code search. This filtration step led us to a total of 64 primary studies. It should be noted that despite the preceding careful, time-consuming steps, relevant studies might be still omitted due to sub-optimal queries or the limitations of search engines provided by the publication databases. We thus added a few studies that were relevant (according to our prior work experience) but missed by the search process accidentally. Finally, we chose a collection of 70 (2.36%) papers as the primary studies for our systematic review.

3.4 Quality Assessment

Table 15 (presented later in Appendix A) provides an overview of our selected primary studies, their adopted methodologies, and offered features. As a standard practice, we investigate whether these studies meet certain quality criteria or not [93, 94]. In particular, we carefully go through each of these 70 studies, capture their subject matters, and then answer 10 standard quality questions, as follows:

- Q_1 : Is there a clear statement about the aim of the research?
- Q_2 : Is there an adequate description about the context of the research?
- Q_3 : Is there a review about the related work from the literature?
- Q_4 : Is there an adequate description of a query reformulation technique targeting code search (e.g., bug localization, concept location, Internet-scale code search)?
- Q_5 : Has the approach been validated?
- Q_6 : Does the conclusion reflect the aim or purpose of the conducted study?
- Q_7 : Is there a clear statement of findings?
- Q_8 : Does the study discuss its limitations or threats to validity?
- Q_9 : Does the study recommend further research?
- Q_{10} : Does the study provide any supports for the replication (e.g., replication package)?

Table 16 (presented later in Appendix B) shows responses to each of the preceding questions. We record the response as either “Yes” (●), “No” (⊖), or “Somewhat Yes” (◐, ◑). We record “Yes” when the supporting evidence is clearly visible, “No” when the supporting materials are not found at all, and “Somewhat Yes” when the relevant evidence is weak or ambiguous. From Table 16, we see that majority of our selected studies meet the quality criteria. Although a few studies do not provide an exhaustive description of their query reformulation approach, they are still retained in this survey due to their overall relevance. Many of these studies also do not provide a publicly available replication package. However, it does not hurt the overall goal of our literature review. Thus, they were also retained.

3.5 Grounded Theory Based Analysis

We use a Grounded Theory approach [58] to analyze our primary studies. It is widely used in social science research to derive meaningful theories that are firmly grounded on the data (e.g., interview scripts). Recently, this approach has also found applications in Software Engineering research [35, 150, 175]. We systematically analyze the full texts from each of the primary studies, derive important theories using this approach, and then answer several of our research questions. The Grounded Theory approach involves the following three stages of coding.

(a) Open Coding. Open coding involves breaking down the gathered data into identifiable, interesting chunks and annotating them with appropriate key phrases (a.k.a. open codes) [150]. We analyze the full text of each primary study, attempt to understand a concept of interest (e.g., methodology, validation, shortcomings), and then record our observations using suitable key phrases. The key idea was to keep an open mind and choose as many codes as needed to represent each study. For example, we used 209 open codes [11] to represent the *methodologies* adopted by our 70 primary studies to reformulate their queries. We spent ≈ 100 man-hours in this open coding task.

(b) Axial Coding. Axial coding establishes relationships among the open codes. In this stage, we place our open codes into a spreadsheet and establish connections among them using various colors. We annotate the key phrases with the same color if they are somehow related. We consider not only lexical overlap but also the semantic relatedness to connect the key phrases [150]. Our goal was to convert the open codes into low-level categories. This stage provided a set of 21 tentative categories [11] that explain the methodologies adopted by our 70 primary studies.

Table 2. Methodologies Used for Query Reformulation

ID	Methodology
M1	Use of term weights and relevance feedback
M2	Mining of dependency graphs
M3	Extraction of semantic relations from co-occurrences and thesauri
M4	Use of conceptual and syntactic relations
M5	Machine learning
M6	API recommendation using data mining
M7	Genetic algorithms
M8	Miscellaneous

(c) **Selective Coding.** Selective coding identifies the core variables behind a given phenomenon [58, 175]. In our case, automated reformulation of a given query is the target phenomenon. Although axial coding provides a set of low-level categories, we carefully analyze them and merge them into high-level categories based on their themes and semantic relatedness. This stage provided eight key categories of methodologies (Table 2) that were used to reformulate search queries by our primary studies. Each of these categories is represented using a set of semantically related key phrases [11].

4 RESULTS

We present our detailed analyses and findings in this section. We carefully go through each of the 70 primary studies, consult their major sections (e.g., problem descriptions, methodologies, algorithms), and then summarize our non-trivial observations using the Grounded Theory approach [58]. We divide our findings and discussions into multiple logical sections and attempt to answer each of our research questions (see Table 1) as follows.

4.1 Answering RQ₁: Methodologies for Automated Query Reformulations

We identify eight key methodologies used by our primary studies for query reformulation. We use Grounded Theory, a popular qualitative analysis approach, to come up with them. Tables 2 and 3 show the methodologies and the primary studies that adopt them. We discuss each of these methodologies and their reported strengths and weaknesses next.

(M1) **Use of Term Weights and Relevance Feedback.** Term weighting determines the *relative importance* of a term (or word) within a body of texts (e.g., document). It has been an integral part of the IR domain [82, 171]. Jones [82] first introduced a term weighting method, namely TF-IDF for Vector Space Model (VSM)-based document retrieval. According to the VSM, a text document can be modeled as a vector of distinct words that are found in the document (see Section 2.4 for the details on TF-IDF). In other words, the terms that are frequently used within a target document but rarely used across the whole corpus are considered to be *important* within the target document. Existing studies extract these terms as important keywords from a document [119]. Besides TF-IDF, its variants (e.g., TF, IDF) are also used for term weighting on an ad hoc basis.

Several primary studies [42, 54, 64, 87, 108, 130, 140, 157, 180, 196, 212] use term weighting methods (e.g., TF-IDF) to reformulate their search queries. In particular, they use these methods in association with the *relevance feedback mechanism* [169]. Relevance feedback is a way to capture complementary information on a query that can be used to reformulate the query (see Section 2.3 for details). First, a set of tentatively relevant documents are collected using the feedback mechanism. Second, candidate terms are extracted from these documents using one or more term weighting methods. Third, the candidate terms are then ranked based on their relative importance (a.k.a. term weights) and only top-K terms are used to reformulate a query. Hayes et al.

Table 3. Primary Studies and Their Adopted Methodologies

ID	Primary Studies	Total
M1	S1, S2, S4, S7, S9–S12, S14, S19, S21, S22, S24–S26, S30, S34, S38, S40, S43, S45, S50, S51, S54, S59, S61, S64, S70	28 (40%)
M2	S4, S19, S22, S24, S25, S31, S33, S40, S46, S56, S57, S61	12 (17%)
M3	S2–S4, S6, S13, S15–S21, S23, S24, S27, S28, S35, S37, S38–S43, S45, S49, S50, S57, S58, S60, S65, S70	32 (46%)
M4	S1, S5, S12, S16–S18, S36, S37, S42, S43, S47–S49, S51, S53, S62	16 (23%)
M5	S8, S10, S11, S22, S29, S32, S35, S58, S62, S63, S66, S67	12 (17%)
M6	S2, S7, S12, S15–S20, S26, S28, S34, S35, S37, S38, S39, S41, S43, S46, S49, S51, S52, S54, S55, S57, S58, S61, S63, S65, S66, S70	31 (44%)
M7	S9, S62, S67–S69	5 (7%)
M8	S1, S3–S7, S11–S15, S17, S21, S24, S26, S29, S37, S39, S41, S44, S46, S53, S57, S60, S62, S67	26 (37%)

[68] first introduce relevance feedback and term weighting (e.g., TF-IDF) in the context of software engineering problems. They reformulate search queries to recover traceability links and to reduce false-positive trace links. Lucia et al. [115] critically examine and extend their work by conducting multiple case studies and using various software artifacts (e.g., use cases, requirements, test cases). However, Gay et al. [54] were the first to use term weighting and relevance feedback to reformulate search queries in the context of code search (e.g., concept location). Since then, many studies have adopted term weighting and relevance feedback in their query reformulation approaches [63, 155, 180, 183, 196]. Haiduc et al. [63] employ Rocchio's expansion [163], Dice similarity [33], and Robertson Selection Value (RSV) [162] to reformulate a query where they use TF-IDF as a proxy of term importance. Their query difficulty models also make use of several variants of TF-IDF (e.g., avgIDF, maxIDF) [64, 130]. Pérez et al. [140] adopt similar query reformulation techniques and term weighting methods to find relevant features from a software model corpus. They later combine expert feedback with Rocchio's method to expand their search queries [141, 145]. Eddy et al. [47] assign varying weights to the terms from various locations of a source document to identify the target software features. Wang et al. [196] use explicit feedback from developers to expand their queries (i.e., Rocchio's expansion) in the context of Internet-scale code search. However, they use their expanded query to re-rank the results retrieved by a given query rather than the code search.

To reformulate a search query, several studies capture relevant API classes, tags, and software-specific terminologies from a programming Q&A site, Stack Overflow, where they employ various keyword selection methods including term weighting [105, 121, 135, 154, 157]. Rahman and Roy [157] collect relevant Q&A threads from Stack Overflow for a given query using PRF, identify the important API classes from their embedded code segments using TF-IDF, and then use the top-ranked API classes to expand the query. Nie et al. [135] employ Rocchio's expansion method and identify software-specific keywords from the Q&A threads of Stack Overflow using TF-IDF. Martie et al. [121] select frequent API classes from the relevant source documents retrieved by a query to expand their query. Kevic and Fritz [87] use TF-IDF and three other lightweight heuristics (e.g., part of speech, position) to identify search keywords from a change request to support the concept location task. Zamani et al. [212] extend the classical term frequency metric (i.e., TF) with time awareness and improve the feature location task. They suggest that important terms within a source document are not only more frequent but also more recent than the noisy terms. They make use of version control history of a software system to gather the term frequency and timing related meta data. Gibiec et al. [57] make use of relevant web pages from three search engines (e.g., Google, Bing, and Yahoo!) to expand the *stubborn* trace queries. In particular, they extract

complementary key phrases from these web pages based on their occurrences (e.g., domain term frequency) and expand the trace queries.

Although TF-IDF has been a popular method for term weighting, it suffers from a major limitation. TF-IDF and other frequency-based methods fail to capture the dependencies among terms, which can be important to determine their relative importance [21, 126]. These dependencies could be statistical, syntactic, semantic, or hierarchical (see Section 2.4). Many existing studies from the IR domain [21, 126] make use of these dependencies to determine term weights. However, Rahman and Roy [156] were the first to leverage both statistical and syntactic relationships among terms to formulate queries for code search. They first transform a change request into a text graph using these dependencies and then apply the PageRank algorithm to the graph to identify the top search keywords for concept location. They later extend and adapt their approach to source code [155] and stack traces [158], and reformulate search queries for IR-based bug localization. They also report the significant benefit of using graph-based term weighting over traditional alternatives (e.g., TF-IDF) in query reformulation. Several later studies [90, 91, 146] also adopt graph-based approaches to reformulate their code search queries.

About 40% (28/70) of the primary studies use relevance feedback and term weighting to reformulate their queries during various code search operations (e.g., concept location, bug localization, Internet-scale code search). Relevance feedback captures complementary items on a query (e.g., relevant documents), whereas term weighting methods identify important keywords from them to reformulate the query. Existing studies leverage several proxies (e.g., frequency, dependencies) to determine the relative importance of search keywords. Thus, the effectiveness of their reformulated queries depends on the appropriateness of these adopted proxies for term importance.

(M2) Mining of Dependency Graphs. Several primary studies [18, 60, 158, 196, 197] leverage dependencies among items (e.g., words, API classes) to construct queries for code search. Wang et al. [197] mine an appropriate sub-graph from the program dependency graph of a given code segment and construct a dependency query to find structurally similar code segments from a code-base. Their goal was to support code-level changes by automatically detecting the similar code segments. They later incorporate explicit feedback from developers in their code search [196]. Panchenko et al. [137] extract an Abstract Syntax Tree (AST) from a given code example and reformulate it into an XPath query to detect structurally similar code examples. Balachandran [18] also address the same problem by extracting appropriate sub-trees from the AST as a reformulated query. Designing a search query from a given code segment is often called *query by example* in the literature [18, 137]. Zhang et al. [215] expand the object-oriented methods (i.e., corpus documents) with neighboring methods where the neighbors are detected based on their call dependencies, temporal proximity, and semantic similarity. Wang et al. [192] detect syntactic and semantic facets from the results retrieved by a given query and use them to reformulate the query for concept location task. Rahman and Roy also mine three types of graphs—trace graphs [158], text graphs [156], and API co-occurrence graphs [155, 157]—to reformulate queries for various code search operations (e.g., bug localization, concept location, Internet-scale code search). Several later studies [90, 91, 146] extend their work, in which they use graph-based mining to reformulate their queries.

About 16% (11/70) of the primary studies mine dependency graphs to reformulate queries for their code search. They extract dependencies from both structured (e.g., source code) and unstructured items (e.g., texts) to develop these graphs. Unlike texts, the graphs have the potential to capture more contextual information (e.g., call dependencies, AST, syntactic dependencies), which could significantly benefit the reformulated queries.

(M3) Extraction of Semantic Relations from Co-Occurrences and Thesauri. Unlike the preceding term weighting and graph-mining studies, several primary studies [55, 75, 106, 120, 154, 172, 183, 207] leverage semantic relations between query keywords and candidate terms to reformulate their search queries. Sisman and Kak [183] first use *spatial code proximity* and suggest that terms co-occurring with query keywords in close proximity within source code are suitable candidates for query expansion. They were the first to expand search queries for IR-based bug localization. They extend their idea later with term ordering and language modeling [181]. Yang and Tan [207] mine the mapping between code and corresponding comment, and construct a software-specific thesaurus—*SWordNet*. Then they reformulate a given query with semantically similar, alternative words collected from their thesaurus to support the concept location task. Similarly, Howard et al. [75] analyze method signatures and their corresponding leading comments from source code, and extract semantically similar word pairs to support query reformulations. Both of the preceding approaches are subject to the availability of sufficient comments or documentations in the source code.

Shepherd et al. [176] extract Verb-Direct Object (V-DO) pairs from method signatures and code comments, and recommend equivalent verbs and DOs to support the developers in their query reformulation. They suggest that if two V-DO pairs share the same direct object, then their verbs could be considered as equivalent. However, all free-form queries cannot be modeled as a V-DO pair [71]. Hill et al. [71] extend this work by relaxing the constraints of V-DO pairs and by suggesting phrasal representations of the method signatures. Developers thus could locate query keywords within these phrases, determine the relevance of corresponding methods, and then manually choose the candidate terms for query expansion. Marcus et al. [120] construct a latent semantic space using Latent Semantic Indexing (LSI) and suggest semantically similar words to reformulate a given query. They also consider the co-occurrences between query keywords and candidate terms within source code to estimate the semantic relatedness.

Several other studies leverage the co-occurrences among items (e.g., terms, tags) within programming Q&A threads [105, 106, 154] and search logs [29, 172] for query reformulation. Stack Overflow, a popular programming Q&A site, discusses software-specific concepts and terminologies in millions of questions and answers. These questions and answers have been frequently used to support several software engineering tasks [105, 154, 157, 211]. Li et al. [106] leverage explicit and implicit co-occurrences of tags from the same question and the duplicate questions, respectively, and reformulate a given query with semantically similar tags. They calculate mutual information between any two tags with the query keyword being one of them, then improve a given query using alternative tags. Rahman and Roy [154] analyze the contextual words of both query keywords and candidate terms in the *titles* of Stack Overflow questions, and suggest appropriate terms for query reformulation based on their contextual similarity. Satter and Sakib [172] mine co-occurrences between query keywords and candidate terms in the past queries from code search logs, and suggest the frequently co-occurred terms for query expansion.

Although co-occurrence between query keywords and candidate terms has been a popular idea for query reformulation, two similar words might not always co-occur in the same context [80, 101]. According to Hussain and Bisson [80], high-order co-occurrences could be a possible solution to this issue. However, most of the preceding primary studies consider only the first-order co-occurrences among terms to estimate their semantic relatedness, which might not be enough.

Several primary studies [55, 103, 104, 106, 114, 176, 191] adopt well-established thesauri such as WordNet [129, 198] to reformulate their search queries. Shepherd et al. [176] accept a V-DO pair as a search query and suggest synonyms from WordNet to expand their query. Developers are then responsible for choosing the right keywords for their queries during concept location. Ge et al. [55] later extend this work with multi-level keyword suggestion and typo correction leveraging

WordNet. Lu et al. [114] replace each keyword of a given query with synonyms extracted from WordNet and outperform the technique of Hill et al. [71] in the concept location task. Lemos et al. [103] also capture synonyms and antonyms from WordNet and predefined types from a Java-type thesaurus to reformulate queries during Internet-scale code search.

Most of the preceding studies are inspired by parallel research on automated query expansion from the IR domain [33]. However, existing findings [184] also suggest that the same word can hold two different semantics for regular texts and source code. For example, the term *new* represents an adjective in regular texts, whereas it means a verb (e.g., object creation) in object-oriented source code [206, 207]. In other words, query expansion approaches solely based on an English language thesaurus (e.g., WordNet) might not be effective for code search. Given such a limitation, several primary studies [75, 106, 206, 207] focus on developing software-specific thesauri to support query reformulations where they leverage the co-occurrences among keywords within the same context (e.g., method signatures, question tags).

Yang and Tan [207] first construct a software-specific thesaurus, namely SWordNet, containing 8.5 million word pairs extracted from nine open source projects. This thesaurus was later used to collect semantically similar words and to reformulate queries for code search [104, 207]. Kevic and Fritz [86] mine change requests, corresponding changed code, and the developer's interactions with the IDE during code-level changes, and construct a dictionary that maps the problem domain vocabulary to solution domain vocabulary. Then they translate a search query (a.k.a. change request) into corresponding source code elements (e.g., methods, classes) using the dictionary to improve the concept location task. Vinayakarao et al. [191] mine programming concepts (e.g., *integer array*) and corresponding syntactic representations (e.g., `int arr[]`) from the Q&A threads of Stack Overflow, and construct a mapping database. Then they augment the source code statements with appropriate programming concepts from the database to improve code comprehension tasks. Although thesaurus-based approaches are popular and widely used, they might suffer from a lack of generalizability. In other words, the thesauri should be constructed from appropriate corpora (e.g., codebase, Stack Overflow) and must be updated frequently to remain relevant.

About 46% (32/70) of our primary studies make use of term co-occurrences and well-established thesauri (e.g., WordNet, SWordNet) to reformulate their queries during code search. However, semantically similar terms might not always co-occur in the same context, which warrants higher order co-occurrences. Thesauri-based approaches might suffer from the lack of generalizability, and the thesauri might also require frequent updates to remain relevant.

(M4) Use of Conceptual and Syntactic Relations. Several studies [46, 71, 81, 91, 114, 156, 191, 205, 210, 212] leverage conceptual relations (e.g., domain ontology) and syntactic relations among words (e.g., Part-of-Speech (POS) tagging) to reformulate search queries. Ontology defines inter-relations among various concepts of an application domain (e.g., Software Engineering) [200]. Modern software systems are inherently complex and deal with numerous concepts ranging from high-level problem descriptions to low-level programming solutions. One way to preserve and manipulate this conceptual knowledge is to construct an ontology from these systems. A few of the primary studies [46, 81, 205] leverage the ontology to reformulate their queries during code search.

Dourdas et al. [46] reformulate a problem domain query into a solution service query using their ontology constructed from the source code. They capture the mapping between proven solution classes and requirement classes in their ontology. Wursch et al. [205] construct another ontology using semantic web technology where they capture structural and data dependency relationships among classes, methods, and attributes from a software system. Their approach accepts a quasi-natural language query from the developer, reformulates it into a structured SPARQL query, and then retrieves a list of relevant program elements. Unlike Wursch et al. [205], Jiang et al. [81]

construct ontological models from multiple software repositories such as source code, bug-fixing history, and bug reports. Then they reformulate a free-form query into multiple candidate questions by capturing the query conditions and then mapping them to ontological concepts. Developers then could choose the most appropriate question from them and locate the relevant program elements. Several other studies [57, 108] also make use of domain-specific concepts to reformulate their queries and to improve their search. Although ontology-based studies are reportedly useful to reformulate natural language queries into complex structured queries [205], they are still limited by their underlying technologies (e.g., semantic web, SPARQL). In addition, maintaining an ontology is costly since the changes in source code are frequent and they need to be reflected in the ontology.

Besides conceptual relations (e.g., ontology), several primary studies [71, 91, 114, 156, 191, 210, 212] leverage the syntactic or linguistic properties of texts to reformulate their search queries. As a standard practice, they often perform POS tagging and then select or emphasize the nouns and verbs in their query reformulation. The underlying idea is that these words convey more important semantics than the others. However, POS tagging is often noisy with technical texts (e.g., bug reports, change requests), which could hurt the query reformulation.

About 23% (16/70) of our primary studies make use of conceptual (e.g., ontology) and syntactic relations (e.g., POS tagging) to reformulate their queries during code search. Although they have been used by several studies, maintaining a software-specific ontology could be challenging and the POS tagging with technical texts could be noisy.

(M5) Machine Learning. Several primary studies [29, 63, 64, 77, 87, 112, 130, 155, 204] adopt machine learning to reformulate their queries during code search. Kevic and Fritz [87] capture *POS*, *term weight*, *position*, and *notation* of each term from a change request, and train a logistic regression model. Their model then accepts an incoming change request and returns top-three terms as a search query for concept location. Liu et al. [112] train an encoder-decoder model using deep neural networks and translate a natural language query into relevant API methods to support Internet-scale code search. Wu and Yang [204] suggest that the changed version of code might hold the true intent of a search query that retrieves the original code. Being motivated by this idea, Huang et al. [77] train a deep belief network with original code and their changed versions from thousands of GitHub projects, then expand a given query with appropriate terms predicted from the changed code. Lawrie and Binkley [99] develop an automated summarizer using a sequence-to-sequence model and reformulate a bug report into a summary to support the bug localization task. Cao et al. [29] train a sequence-to-sequence model with an attention mechanism where they use original and reformulated queries from the search logs of a Stack Overflow Q&A site. Then they use their model to reformulate a given query during code example search at Stack Overflow. Although sequence-to-sequence models have been popular for reformulating queries, they might require a large amount of training data.

Determining the quality of a search query before its actual reformulation has been a popular idea in the IR domain [31]. Haiduc et al. [64] first analyze and predict the difficulty of queries in the context of code search. They train their model using 21 pre-retrieval metrics from the IR domain, then predict the quality of a given query during concept location. Mills et al. [130] later extend this model with seven post-retrieval metrics (i.e., 28 in total) and evaluate the model performance extensively. Given a search query, Haiduc et al. [63] generate a list of four reformulation candidates using Rocchio's expansion [163], RSV [162], Dice similarity [33], and query reduction [33]. Then they train a query difficulty model to suggest the best reformulated query. Rahman and Roy [155] also extract four reformulation candidates by analyzing method signatures and field signatures from source code, then suggest the best candidate using the query difficulty analysis [64]. Although the query difficulty analysis has been popular and cheap, its metrics might not be as

accurate as the query performance metrics (e.g., MAP, Query Effectiveness (QE)). It should be noted that query difficulty metrics (e.g., keyword frequency, keyword entropy) are often used to prevent poor queries from execution when the ground truth is not known. On the contrary, query performance metrics are computed using the ground truth information, which makes them more reliable.

Over the past few years, word embedding technology has been leveraged to tackle the vocabulary mismatch problem in code search. Zhang et al. [213] learn the embeddings of query keywords and API classes from a corpus of $\approx 25,000$ open source projects using a three-layer neural network and Continuous Bag-of-Words algorithm. Then they translate a natural language query into relevant API classes to support code search. Similarly, Rahman and Roy [157] learn the word embeddings from 1.40 million Q&A threads of Stack Overflow using the Skip-gram algorithm [23] and expand a natural language query with relevant API classes. Both preceding studies report significant performance improvement in their code search.

About 17% (12/70) of our primary studies use machine learning, query difficulty analysis, and word embeddings to reformulate their queries during code search. These technologies have high potential to improve query reformulations and code search; however, they are also restricted by their training datasets, learning algorithms, and model parameters.

(M6) API Recommendation Using Data Mining. Many primary studies [29, 108, 116, 148, 157, 159, 172, 180, 213] mine code search logs and software repositories (e.g., API documentation, programming Q&A websites) to reformulate search queries through the recommendation of relevant API classes or methods. Bajracharya and Lopes [17] first capture the search logs from an Internet-scale code search engine called *Koders*, and analyze the patterns, topics, and user behaviors during code search from the logs. They report that the majority of natural language queries from these logs required query reformulations, which indicates the need for appropriate query reformulation techniques. Satter and Sakib [172] also analyze the logs from *Koders* [15], construct a term-term co-occurrence matrix from the past queries, and then suggest frequently co-occurred terms from the matrix to expand a given query. Raghothaman et al. [148] analyze the *click-through* data from the Bing search engine that contains millions of (query, URL) pairs. They mine the logical mapping between each query and API classes from the corresponding URL, construct a conditional probabilistic model, and then suggest relevant API classes to reformulate a given query. Cao et al. [29] analyze the search logs from a Stack Overflow Q&A website, train a sequence-to-sequence model using original and reformulated queries, and then generate a reformulated query for a given query. Dietrich et al. [44] also capture the query pairs from developers containing original and reformulated queries, mine query transformation rules, and use them to reformulate a query for traceability link recovery.

Several primary studies mine relevant API classes from API documentation [39, 116], programming Q&A websites [42, 157, 159, 180, 191], codebase, and version control history [60, 77, 79, 108, 213] to reformulate a given query. Lv et al. [116] identify relevant API classes to a query considering textual similarity between the query and candidate API documentation from MSDN, and use them to expand the query. Chatterjee et al. [39] enrich the API invocations in source code with corresponding API documentation to tackle the *vocabulary mismatch problem* [53] in code search.

Rahman et al. [159] first capture the co-occurrences between query keywords and API classes in the Q&A threads of Stack Overflow, and suggest relevant API classes to expand a given query. Several later studies [42, 150] extend this work. Other primary studies reformulate a query with relevant keywords [135, 154, 180] and tags from Stack Overflow [105, 106]. Nie et al. [135] capture software-specific keywords from the relevant Q&A threads using PRF, question popularity, and term weighting method, then expand a given query for code search. Li et al. [105] construct a

software-specific tag database and reformulate a given query with synonymous and relevant tags from Stack Overflow questions.

Zhang et al. [213] mine relevant API classes from $\approx 25,000$ open source projects to expand a given query. Lin et al. [108] use *Recodoc* [43], a TF-IDF based traceability recovery technique, to reformulate a given query into relevant API entities. Huang et al. [77] mine thousands of code commits from the version control history and suggest API elements from the changed code using a deep belief network to reformulate a query. Similarly, Huang et al. [79] attempt to capture the intent of a query from the changes made to its initially retrieved code, then expand the query with API elements extracted from these changes. Although expanding a given query with relevant API classes, keywords, and tags was found to be useful, determining the true intent of a query still remains a challenge and unlocking that could lead to more accurate query reformulation.

About 44% (31/70) of our primary studies mine relevant API classes, keywords, and tags from various sources such as search logs, API documentation, programming Q&A sites, open source projects, and version control history. Many of these approaches are inspired by the parallel studies from IR. Although mining relevant items from these sources was found to be useful, determining the true intent of a search query remains a challenge. In addition, the logs from widely used commercial search engines (e.g., Google, Bing) might not be available for public use.

(M7) Genetic Algorithms. Several primary studies [91, 99, 131, 140, 146] use Genetic Algorithms (GAs) to reformulate their search queries. Mills et al. [131] first use a GA to select the near-optimal query from a bug report to support bug localization. They demonstrate that bug reports often contain near-optimal queries, although there might not be any explicit hints for bug localization (e.g., program elements, stack traces), which is interesting. However, their approach uses QE to determine the *fitness* of query candidates. In other words, the approach will need ground truth to find the near-optimal query, which makes it unsuitable for practical use. A few later studies [99, 151] revisit their findings with different datasets and reach similar conclusions. According to Rahman et al. [151], although the majority of bug reports contain near-optimal keywords, the GA-based approach might not be able to identify them from a few bug reports (e.g., $\approx 12\%$) despite the availability of ground truth information. Kim and Lee [91] perform multi-objective optimization using 15 fitness functions and reformulate a query using a GA to support bug localization. However, Pérez et al. [146] use GAs to reformulate corpus documents (e.g., model fragments) in the context of a feature location task. They employ four query reformulation types—expansion [163], replacement [57], reduction [63], and selection [156]—as the mutation operations and textual similarity between the query and corpus document as the fitness function in their GA-based feature location. Although they report benefit in reformulating the corpus documents (e.g., model fragments), the reformulation of queries was not found to be effective by one of their earlier works [140].

About 7% (5/70) of our primary studies use GAs to reformulate their search queries and corpus documents. Although GA-based approaches have high potential for effective query reformulations, designing an appropriate, practical fitness function remains a challenge. In addition, the GA-based approaches could also be costly due to their large search space constructed by all of the keywords from either a bug report or a change request.

(M8) Miscellaneous. Many primary studies [18, 39, 57, 71, 87, 92, 102, 116, 121, 159, 176, 180, 210, 212] use lightweight heuristics and ad hoc metrics to reformulate their queries during code search. Zamani et al. [212] select only nouns as query keywords from a feature request to improve the feature location task. Yu et al. [210] make use of nouns, verbs, and adjectives from the texts. Shepherd et al. [176] model search queries for concern location as V-DO pairs. Hill et al. [71] expand

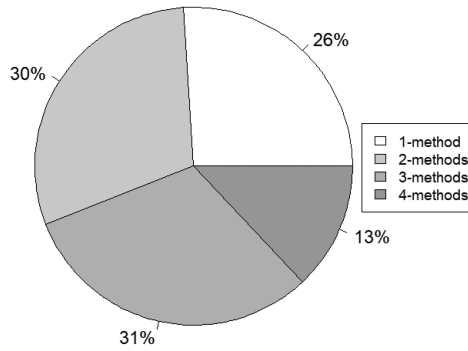


Fig. 5. Primary studies adopting multiple methodologies in the query reformulation.

this technique with other POS. Kevic and Fritz [87] consider four lightweight heuristics such as *POS*, *term weight*, *notation*, and *position* of each keyword within a change request to separate the query keywords from the rest using machine learning.

Rahman et al. [159] capture co-occurrences between query keywords and API classes in the Q&A threads of Stack Overflow, apply three lightweight heuristics to detect the relevant API classes, and then use them to expand a query for Internet-scale code search. Unlike Rahman et al. [159], several studies [39, 108, 116] rely on textual similarity between API documentations and a query to find the relevant API classes. Other studies simply extract the structural entities such as API classes or API methods from the queries [210], test cases [102], accepted answers of Stack Overflow [180], and relevance feedback documents [121].

Lapeña et al. [97] replace a query with the most relevant requirement specification using textual similarity to improve feature location. Wang et al. [192] extract multiple facets (e.g., structural, intent, dependency) from the initial results of a query to help developers further refine the query interactively. Chaparro and Marcus [38] adopt stepwise query reduction and suggest that removal of even only one noisy keyword can significantly improve the query. Chaparro et al. [35] perform natural language discourse analysis, identify Observed Behavior (OB), Expected Behavior (EB), and Steps to Reproduce (S2R) from a bug report (a.k.a. query), and use OB as a reduced version of the query for localizing bugs. Zhang et al. [215] identify the neighbors of a given method in a codebase considering semantic similarity, temporal proximity, and call dependency, and expand the method's body with the neighboring methods to improve feature location. Sisman and Kak [183] also leverage spatial code proximity to reformulate their query for bug localization. Lu et al. [113] expand a search query leveraging several object-oriented relations (e.g., inheritance, implementation) from the source code. Kim and Lee [91] also apply several heuristics as fitness functions in their GA-based query reformulations to locate buggy code.

About 37% (26/70) of our primary studies use lightweight heuristics and ad hoc metrics to reformulate their queries during code search. These heuristics and metrics often offer low-cost alternatives to the established but costly approaches (e.g., thesaurus-based reformulation). However, they might not always be effective and thus need to be complemented by more sophisticated query reformulation techniques (e.g., term weighting).

We also provide the statistical summary of our study methodologies, their overlap, and several other items such as reformulation types, relevance feedback, and working contexts. Figure 5 shows how one or more methodologies are used by the primary studies. We see that 26% of the studies use only one of the eight methodologies (see Table 2). The rest 74% (52/70) of them use two or more

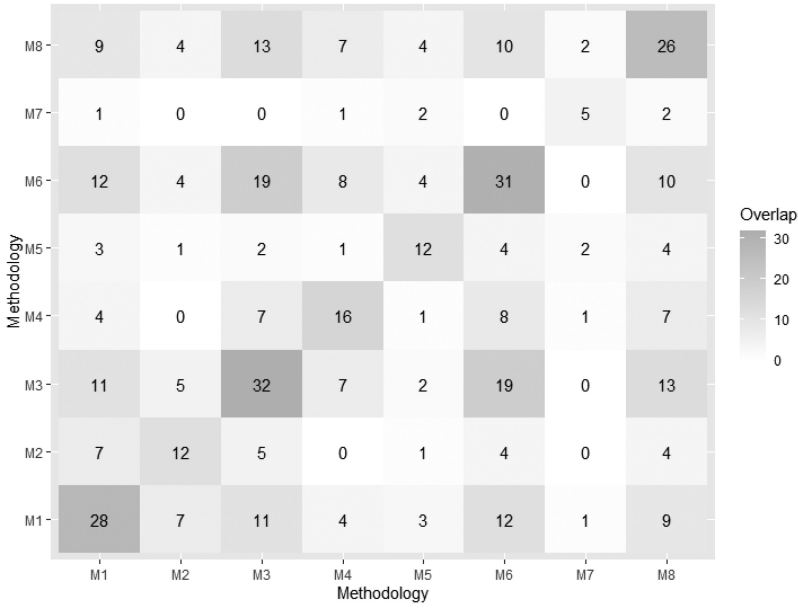


Fig. 6. Overlap of the methodologies used by the primary studies.

methodologies to reformulate their queries during code search. In other words, one methodology might not be sufficient for the majority of primary studies. We also see that 13% of the studies combine four different methodologies to reformulate their queries during code search. Figure 6 further demonstrates how the eight methodologies are combined across the primary studies. We see that three methodologies—use of term weights and relevance feedback (M1), extraction of semantic relations from co-occurrences and a thesaurus (M3), and API recommendation using data mining (M6)—are shared by 60% (42/70) of the primary studies. Such a finding can be explained as follows. First, many term weighting methods consider such words as *important* that frequently co-occur with query keywords in the relevant contexts, such as search logs [172], change request [156, 158], source code [71, 155, 176, 181, 183], stack traces [158], and programming Q&A websites [105, 154]. Second, they might also capture such API classes to reformulate a query that frequently occurs in the relevant contexts [121] and are semantically similar to the query [157, 213]. Third, the thesaurus-based approaches are connected to mining since the construction of any software-specific thesaurus involves extensive mining of software repositories [75, 157, 159, 206, 207]. Figure 7 further demonstrates the trends on adoption of various methodologies in query reformulations. We see that the use of thesauri (M3) and data mining (M6) was prevalent in search query reformulation, which might be inspired by the availability of well-established thesauri (e.g., WordNet) and open source projects (e.g., GitHub). The use of term frequency and relevance feedback was also remarkable. The other methodologies were less frequent. It also should be noted that the research activities on search query reformulations has increased significantly since 2013.

Figure 8 shows how different types of query reformulations are used by our primary studies. We see that 56% of studies expand their queries by adding appropriate keywords, whereas 13% reduce their queries by discarding the noisy keywords. However, 31% of our primary studies adopt a combination of multiple techniques to reformulate their search queries.

Figure 9 shows how different mechanisms to collect relevance feedback are used by the primary studies. We see that 19% of studies capture explicit developer feedback on the queries for their

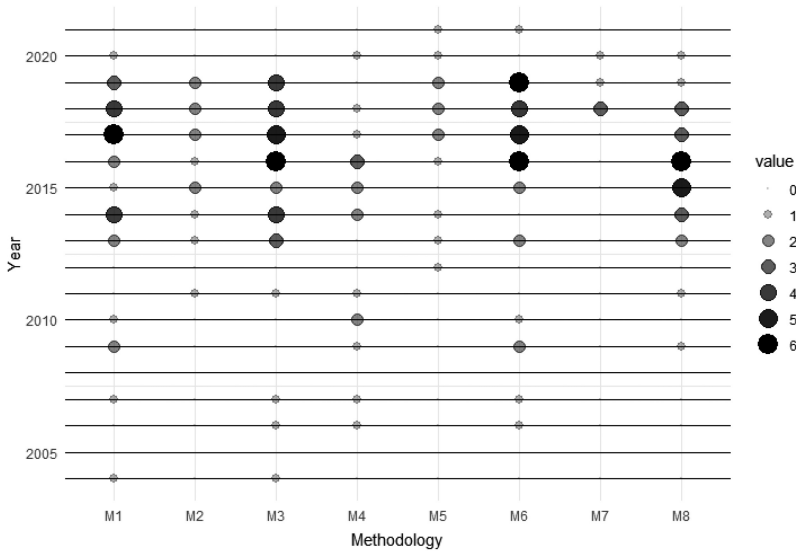


Fig. 7. Adoption of methodologies by the primary studies over the years.

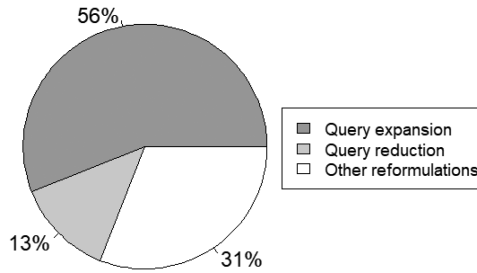


Fig. 8. Primary studies performing different types of query reformulations.

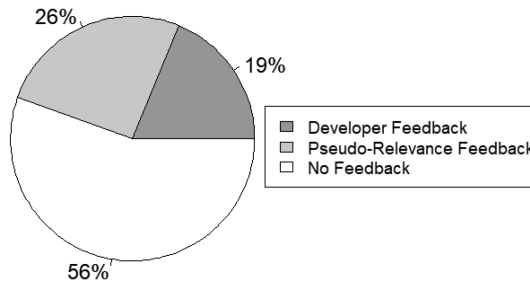


Fig. 9. Primary studies using different relevance feedback mechanisms in their query reformulations.

reformulations. Unfortunately, capturing feedback from the developers could be costly and sometimes infeasible. Thus, about 26% of studies reformulate their queries using PRF without directly involving the developers. However, the remaining 56% of studies do not make use of any relevance feedback mechanism to reformulate their queries.

Figure 10 shows how the primary studies reformulate their queries to improve code search in various problem contexts. For example, 25% of studies reformulate their queries to search various

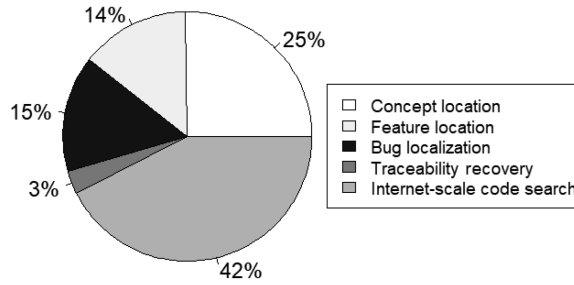


Fig. 10. Query reformulations used to search code in different working contexts.

software concepts implemented in the source code. Software features are a type of concept that can be experienced and exercised by software users. About 14% of studies reformulate their queries to locate features in the source code. Software bugs are the erroneous features that need to be removed from a software system. We see that 15% of our primary studies reformulate their queries to find bugs in the source code. However, 42% of the primary studies reformulate their queries to find relevant, reusable code segments from Internet-scale code repositories such as GitHub and SourceForge, and programming Q&A sites such as Stack Overflow.

We also further investigate the preceding dimensions and report interesting patterns. Table 4 shows frequently used methodologies for query reformulations in various code search contexts. We see that the local code searches such as concept location and feature location are more likely to use term weighting algorithms to expand their search queries. However, Internet-scale code search techniques are more likely to use various data mining approaches, semantic relations, and popular thesauri (e.g., WordNet) to expand their queries. We also see that the studies focus more on query expansion rather than any other form of reformulation.

Summary for RQ₁: The primary studies on search query reformulations use eight major methodologies including term weighting and relevance feedback (40%); semantic relations, term co-occurrences, and thesaurus (46%); machine learning and query difficulty analysis (17%); and data mining and API recommendation (44%). The majority of these studies (56%) expand their queries, and almost half of them capture relevance feedback on their queries. Furthermore, 57% of studies help find relevant software entities (e.g., software bugs, features), whereas the remaining studies help find relevant, reusable code examples from the Internet-scale code repositories (e.g., GitHub).

4.2 Answering RQ₂: Evaluation and Validation

Our primary studies use various methods to evaluate and validate their performance and to place their techniques in the literature. In particular, they use several evaluation methods, performance metrics, subject systems, query collections, ground truth, and validation targets. In the following, we discuss each of these evaluation dimensions with relevant statistics.

(a) Evaluation Methods. Search queries can be evaluated either through execution or without execution. Based on the presence of query execution, evaluation methods can be divided into two categories: *pre-retrieval* and *post-retrieval* [63]. Pre-retrieval methods evaluate a search query based on its linguistic and lexical aspects (e.g., coherence, specificity) without executing the query [33]. Since these aspects of a query can be captured during the indexing of a corpus, the pre-retrieval evaluation methods are lightweight and cost-effective. Unfortunately, they are less reliable because they do not analyze the results retrieved by a query [65]. On the contrary, post-retrieval evaluation

Table 4. Frequently Used Methodologies for Query Reformulation in Various Code Searches

Code Search	Frequently Used Methodologies	#Studies	Reformulation
Concept location	M1: Use of term weights and relevance feedback	10	Query expansion
	M4: Use of conceptual and syntactic relations	7	
	M3: Extraction of semantic relations from co-occurrences and thesauri	6	
Feature location	M1: Use of term weights and relevance feedback	6	Expansion + reduction
	M3: Extraction of semantic relations from co-occurrences and thesauri	4	
	M7: Genetic algorithms	2	
Bug localization	M8: Miscellaneous	7	Query expansion
	M2: Mining of dependency graphs	4	
	M5: Machine learning	3	
Internet-scale code search	M6: API recommendation using data mining	21	Query expansion
	M3: Extraction of semantic relations from co-occurrences and thesauri	19	
	M8: Miscellaneous	13	

methods are more reliable but costly. They execute a query and compare its results with the ground truth to evaluate the performance [33, 63]. All of our selected primary studies adopt post-retrieval methods for their evaluation and validation. A few studies [64, 130] also use linguistic and textual metrics to separate high-quality search queries from poorly designed queries.

Construction of ground truth can be another dimension to classify the evaluation methods. Based on this dimension, query evaluation methods can be divided into two categories: *re-enactment-based evaluation* and *developer study*. In case of re-enactment based evaluation, ground truth information is captured from historical data or existing benchmarks [63]. For example, to assess a search query from a bug report, the changed files from corresponding bug-fix commit are considered as the ground truth [35, 158, 183]. However, in the case of developer study, the participants are responsible for constructing the ground truth [213] and for determining relevance of the retrieved results by a query [103, 116, 172, 196, 197]. From Figure 11, we see that 64% of our primary studies evaluate their queries through re-enactment, whereas 26% of them involve human participants for query evaluation. Interestingly, 9% of the primary studies use both types of evaluation and 1% of them employ case studies for their evaluation. Although the re-enactment-based method uses solid ground truth, it might not always reflect a realistic evaluation scenario since the stakeholders (e.g., developers) are not directly involved. On the contrary, involving human developers could also introduce subjective bias if appropriate mitigation strategy is not in place (e.g., inter-rater agreement analysis, polling [213]). In addition, involving a large number of human participants could be costly. Thus, a combination of re-enactment and developer study is an ideal choice for effective evaluation. However, only 9% of the primary studies adopt such a combination in their query evaluation.

(b) Performance Metrics. Each of our primary studies uses one or more performance metrics to evaluate their reformulated queries. Since the majority of these studies borrow algorithms or techniques from several other research domains such as IR, Machine Learning, and Recommendation Systems, the performance metrics are also taken from these domains. In the following, we discuss these metrics and their adoption by the primary studies.

Figure 12 shows the top-10 performance metrics that are frequently adopted by our primary studies on automated query reformulation. An overview of these 10 metrics can be found in Appendix C. We see that Hit@K, recall, MAP, and Mean Reciprocal Rank (MRR) are used by more than 30% of the studies. Hit@K determines whether there exists any ground truth within the top-K results, whereas recall determines the percentage of ground truth present in the top-K

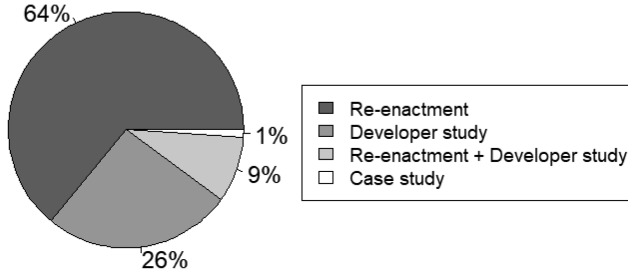


Fig. 11. Types of evaluation conducted by the primary studies.

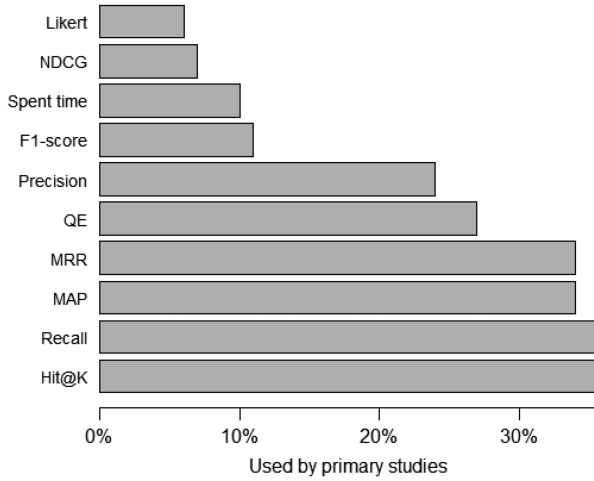


Fig. 12. Top-10 most frequently used performance metrics by the primary studies.

results. However, both MAP and MRR emphasize on the position of ground truth within the retrieved results. Another performance metric that estimates a developer's effort to find the first ground truth within the retrieved results (a.k.a. QE) is used by 27% of the primary studies. There also exist other metrics such as Normalized Discounted Cumulative Gain (NDCG) and Likert that are used by less than 10% of the studies.

We also investigate how the top-10 performance metrics have been used during the past 15+ years. From Figure 13, we see that most of these metrics, such as Hit@K, MAP, MRR, and QE, enjoyed increased adoption since 2013. Our investigation in Figure 14 suggests that such an adoption might be connected to re-enactment-based evaluation, which has been adopted by 64% of the primary studies (see Figure 11). We also further investigate how the primary studies adopt these metrics in different working contexts. From Figure 15, we note that Hit@K, recall, MAP, and MRR were frequently used by the primary studies across different working contexts. Although each of them was popular in Internet-scale code search, Hit@K, MAP, and MRR were found to be especially popular in bug localization and concept location. It also should be noted that QE was found to be more popular in bug localization and concept location than in Internet-scale code search. In other words, detection of the first relevant element is quite important in local code searches (e.g., bug localization, concept location), which can help identify other relevant elements from a corpus through their static dependencies [63].

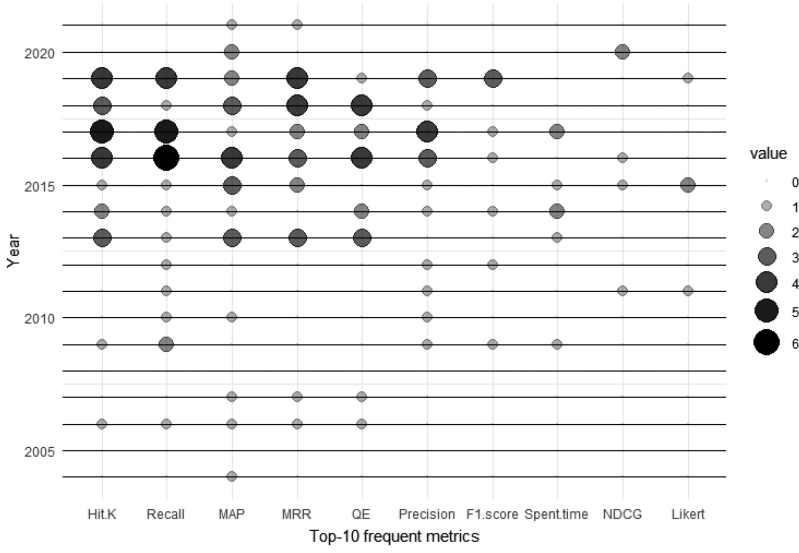


Fig. 13. Use of the top-10 most frequently used performance metrics by primary studies over the years.

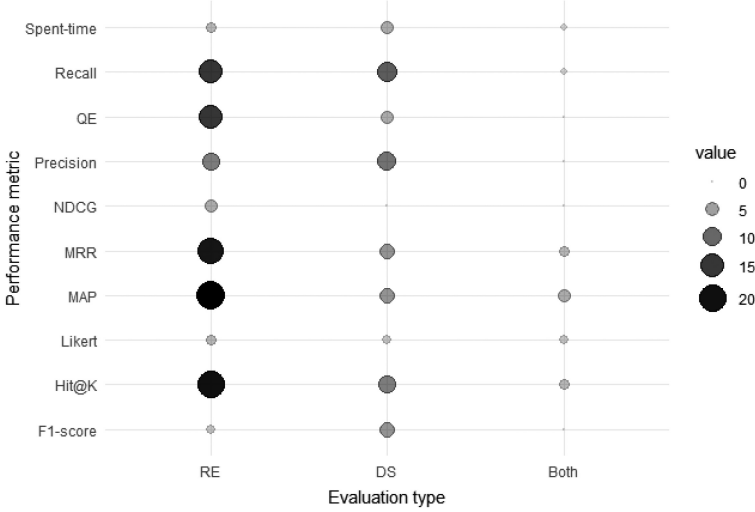


Fig. 14. Use of the top-10 most frequently used performance metrics in different evaluation methods. RE, re-enactment; DS, developer study.

We also further analyze how each study uses these top-10 metrics. Figure 16 summarizes our analysis. We not only see the metric adoption ratio for each of the studies but also their distribution. According to Figure 16(b), at least half of the studies select 2 metrics, whereas 25% of them use 3 or more metrics. However, at least 75% of the studies use at least 1 of the 10 metrics discussed previously. We also found that at least 5% of the primary studies adopt 5 metrics to evaluate and validate their reformulated queries. All of these findings and statistics can serve as an important guideline for choosing appropriate performance metrics in future work.

(c) Subject Systems. Each of our primary studies uses 1 or more subject systems to evaluate their reformulated queries. Table 5 shows the popular subject systems used in four different

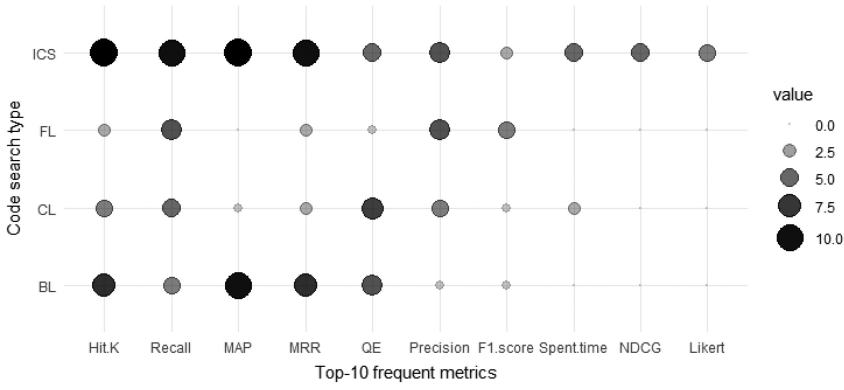


Fig. 15. Use of the top-10 most frequently used performance metrics in different code searches over the years. CL, concept location; FL, feature location; BL, bug localization; ICS, Internet-scale code search.

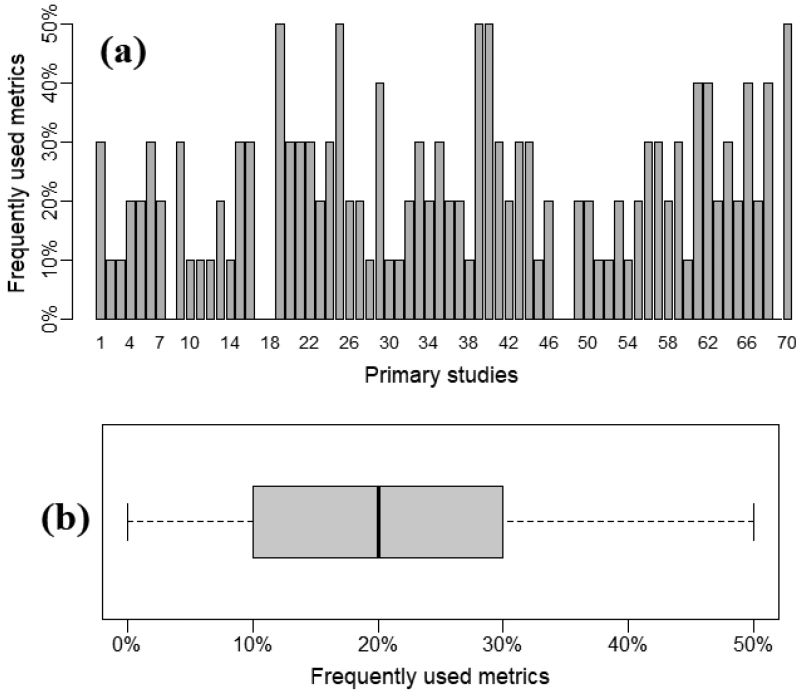


Fig. 16. (a) Percentage of the top-10 most frequently used metrics by each primary study. (b) Box plot of the percentage measures.

working contexts. In case of concept location, we see that 8 systems including Adempiere, JEdit, ECF, and FileZilla are selected by at least three primary studies. However, a total of 20 subject systems were used by at least two primary studies to evaluate their reformulated queries and to locate their desired concepts in the software code. In case of feature location, only 2 systems (CAF and JEdit) are selected by two or more primary studies where JEdit is common. However, in the case of bug localization, we see that 20+ systems are selected by at least two or more primary studies on average. It should be noted that the primary studies from concept location, feature

Table 5. Frequently Used Subject Systems for Experiment in Various Code Searches

Code Search	Frequently Used Systems
Concept location	Adempiere (4), JEdit (4), ECF (3), FileZilla (3), eclipse.jdt.core (3), WinMerge (3), aTunes (3), eclipse.pde.ui (3), Rhino (2), eclipse.jdt.ui (2), Log4j (2), Tomcat70 (2), iReport (2), eclipse.jdt.debug (2), Sando (2), Jajuk (2), Sling (2), JavaHMO (2), Eclipse (2), JBidwatcher (2)
Feature location	CAF (4), JEdit (2)
Bug localization	AspectJ (4), Lucene (3), ECF (3), Pig (3), eclipse.jdt.ui (3), Solr (3), Derby (3), eclipse.jdt.core (3), Tomcat70 (3), OpenJPA (3), ZooKeeper (3), BookKeeper (3), eclipse.jdt.debug (3), Mahout (3), eclipse.pde.ui (3), Tika (3), ZXing (2), JodaTime (2), SWT (2), Chrome (2), Eclipse (2)
Internet-scale code search	GitHub (9), Stack Overflow (8), SourceForge (4), OSChina (2), Java2s (2), Javadb (2), KodeJava (2), OpenHub (2), Google (2), JEdit (2)

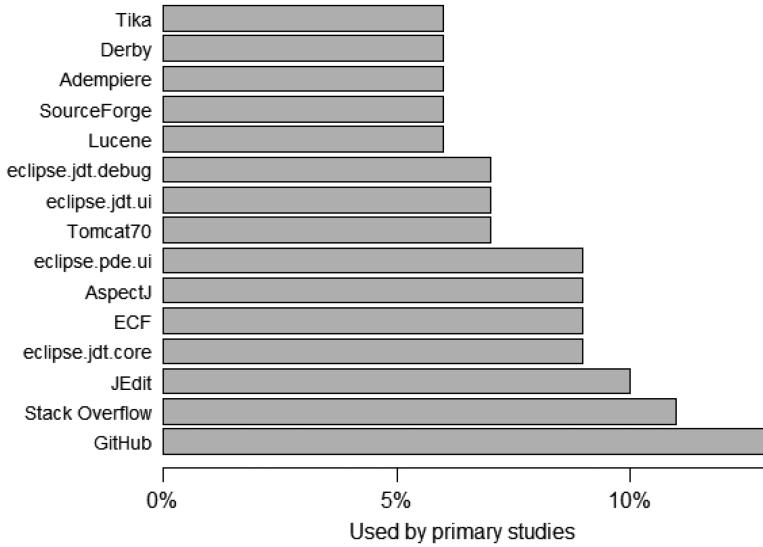


Fig. 17. Top-15 frequently used subject systems by the primary studies.

location, and bug localization share several subject systems (e.g., ECF, JEdit, Eclipse). This might be partially explained by their methodological overlap, as demonstrated in Table 4. On the contrary, the primary studies from Internet-scale code search mostly use large-scale open source software platforms (e.g., GitHub, SourceForge, OpenHub, OSChina) and popular programming Q&A websites (e.g., Stack Overflow, KodeJava, Java2s) for their evaluation and validation. In other words, they use hundreds, if not thousands, of subject systems from these platforms in their experiment.

Figure 17 further demonstrates the top-15 popular subject systems (or software platforms) used by our primary studies across different working contexts. We see that GitHub and Stack Overflow are the most popular choices due to their large collection of open source projects and millions of question-answering threads containing code examples. JEdit, a Java-based IDE, has also been a popular subject system among the primary studies from multiple code search contexts. In addition, several other systems such as eclipse.jdt.core, ECF, and AspectJ have been used by $\approx 10\%$ of the primary studies. The use of diverse subject systems is essential to mitigate the threats to external validity of the experimental findings [211].

Table 6. Search Queries Used for Experiment in Various Code Searches

Code Search	Query Count	#Studies	Frequent Reformulation
Concept location	1–25	5	Query expansion
	1,000–5,000	5	
	25–50	2	
Feature location	1–25	3	Expansion + reduction
	25–50	2	
	150–250	2	
Bug localization	1,000–5,000	4	Query expansion
	250–500	2	
	500–1,000	2	
Internet-scale code search	1–25	8	Query expansion
	25–50	5	
	50–75	5	

(d) Search Queries and Human Participants. Each of our primary studies selects a set of search queries to evaluate their query reformulation techniques. Ideally, all queries (to be reformulated) should be collected from human developers. However, involving humans is costly, and thus only a few studies [44, 54, 63, 141] collect actual queries from the developers. The majority of primary studies use historical artifacts (e.g., past bug reports) [63, 156, 183], reuse queries from earlier datasets [172, 213], or simply construct their queries from secondary sources (e.g., programming Q&A forums, tutorial sites) [157, 159]. These sources often deliver a good enough representation of actual search queries from human developers. Furthermore, they can provide a relatively large dataset at low cost, which is essential to conduct a large-scale empirical evaluation.

The number of used queries is an important aspect of evaluation. We collect the number of queries used by each of our primary studies and summarize our analysis in Figure 18 and Table 6. From Figure 18, we see that 23% (16/70) of the studies use 25 or fewer queries, whereas 14% (10/70) of them select 26 to 50 queries for their evaluation. However, 16 studies select 150 to 500 search queries, whereas 14 other studies evaluate their reformulation techniques using 500+ search queries. We further breakdown our analysis across different working contexts. From Table 6, we see that the primary studies from concept location and bug localization use the maximum number of queries, which might be partially explained by the availability of thousands of bug reports or change requests in the open source projects. However, the number of used queries in Internet-scale code search is generally small. Unlike that of preceding working contexts, these queries cannot be easily extracted from the historical artifacts and are often collected from human developers, which might partially explain their small numbers.

Human participation is an important aspect of search query evaluation. We found that 34% (24/70) of our primary studies involve human participants in their evaluation. Figure 19 shows the statistics on human participation in query evaluation. We see that the majority of these studies (80%) involve 25 or fewer participants, whereas 16% involve up to 50 developers. However, only one primary study involves 250+ human participants in a large-scale developer survey. Furthermore, we found that 58% (14/24) of human participation is associated to the primary studies from Internet-scale code search. However, the number of developers involved in each study is small on average (i.e., 1–25). Involving human participants is always a great idea to complement an empirical evaluation. However, this involvement is often costly, which possibly explains the small number of participants recruited by the majority of our primary studies.

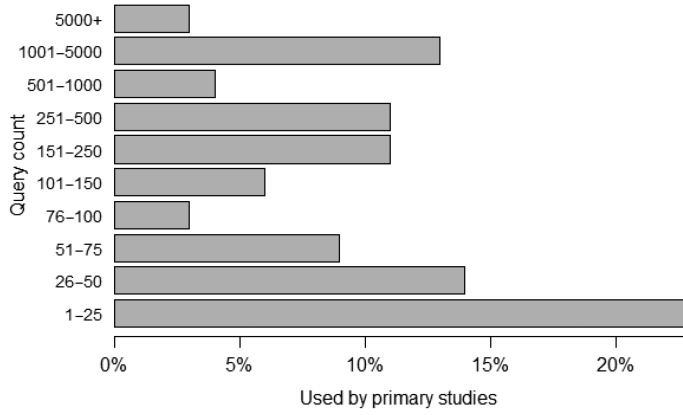


Fig. 18. Number of search queries used by primary studies.

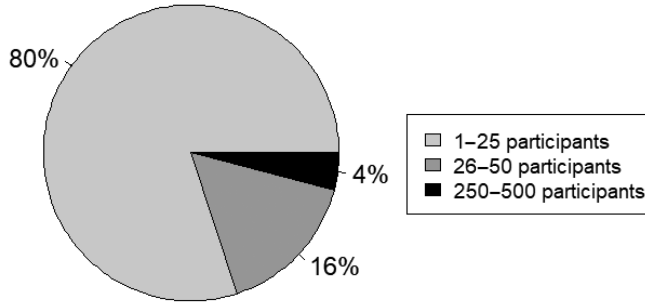


Fig. 19. Number of participants involved in primary studies in their evaluation.

(e) Validation Targets. Comparison with closely related existing studies (a.k.a. validation targets) is an essential step to place any work in the literature. About 99% of our primary studies validate their experimental findings by comparing with one or more existing alternatives. We collect the validation targets from each study and summarize our analysis in Table 7 and Figure 20.

From Table 7, we see that *Baseline* and *Rocchio's method* [163] have been frequently selected for comparison by the primary studies across three different search contexts: *concept location*, *feature location*, and *bug localization*. Baseline search query refers to the initial version of a query that gets reformulated by the primary studies. However, Rocchio's method is a popular technique for query expansion borrowed from the IR domain [33, 163]. About 33% of our primary studies compare their reformulated queries with that from Baseline and Rocchio's methods. Besides them, other techniques, such as RSV, Dice expansion, and query reduction, have also been frequently selected for comparison by the primary studies across concept location and feature location. However, the primary studies that reformulate queries to localize software bugs compare with several bug localization techniques, such as BugLocator [217], AmaLgam+ [195], BLIZZARD [158], BLUiR [168], and that of Sisman and Kak [183]. However, the primary studies from Internet-scale code search frequently compare their approaches with several well-established general-purpose (e.g., Google search) or specialized search engines (e.g., GitHub search, Stack Overflow search) and a few established search algorithms, such as BM25 [178] and TF-IDF [82]. Furthermore, several primary studies, such as QECK (S34), CodeHow (S15), and RACK (S39), were also chosen by the later studies to validate their findings.

Table 7. Frequently Used Validation Targets for Comparison in Various Code Searches

Code Search	Frequently Used Validation Targets
Concept location	Rocchio's method [163] (4), Baseline (4), Pessimistic constant [64] (2), RSV [162] (2), Optimistic constant [64] (2), Random classification [64] (2), Shepherd et al. [176] (2), Query reduction [63] (1), Eclipse IDE-based search (1), Dice expansion [63] (1), Conquer [164] (1), Sisman and Kak [183] (1), Manual search (1), Text-based search engine (1), GES [176] (1), FLAT3 [173] (1), Logistic regression [122] (1), Refoqus [63] (1), Kevic and Fritz [87] (1), GREP (1), ELex (1), Hill et al. [71] (1)
Feature location	Baseline (6), Rocchio's method [163] (2), Dice expansion [63] (2), RSV [162] (2), Query reduction [63] (2), LDA [22] (2), Eclipse search (1), LSI [170] (1), SWUM [72] (1), Shepherd et al. [176] (1), Manual search (1), VSM [82] (1), Sridhara et al. [184] (1)
Bug localization	Baseline (6), BugLocator [217] (4), BLIZZARD [158] (2), Rocchio's method [163] (1), BRTracer [202] (1), BLUIR [168] (1), Lucene [6] (1), Lobster [133] (1), Sisman and Kak [183] (1), rVSM [217] (1), Manual search (1), BLIA [209] (1), AmaLgam+ [195] (2), Relevance model [98] (1), MULAB [216] (1), OB [35] (1), Wang and Lo [194] (1), GA (1), STRICT [156] (1)
Internet-scale code search	Google search (6), GitHub search (5), BM25 [178] (5), QECK [135] (4), TF-IDF [82] (4), Stack Overflow search (3), OpenHub search (3), CodeHow [116] (3), Portfolio [123] (3), Baidu search (2), Baseline (2), OSChina search (2), Manual search (2), RACK [159] (2), VF (VSM + Item set mining) (2), SourceForge search (2), Lu et al. [114] (2), Rocchio's method [163] (1), Ohloh search (1), NQE [112] (1), Dice expansion [63] (1), Swordnet [207] (1), WordNet [129] (1), FIM [29] (1), HRED-qs [29] (1), GooglePS [29] (1), Word2Vec [127] (1), K-NN Linear (1), Thung et al. [188] (1), Codota search [8] (1), Bing search (1), Cocabu [180] (1), BIKER [76] (1), Zhang et al. [213] (1), RSV [162] (1), NLP2API [157] (1), KBCS [103] (1), Jungoloid [118] (1), NCS [112] (1), LDA [22] (1)

Figure 20 further demonstrates the top-15 validation targets selected by the primary studies across different search contexts. We see that reformulated queries are often compared with their initial, non-reformulated versions (a.k.a. Baseline) to make sure that the reformulation is useful. About 18% (13/70) of our primary studies perform such a comparison. Several traditional alternatives, such as Google search, GitHub search, and Stack Overflow search, have also been popular. Furthermore, six existing approaches including RSV, Dice, and TF-IDF have also been used for comparison by 5% or more of the primary studies on average. Comparison with appropriate existing alternatives is essential to claim the technical superiority of a proposed technique. Unfortunately, due to the unavailability of replication packages, such a comparison might not always be possible. For example, only $\approx 30\%$ of the primary studies were selected for comparison by the later studies, which might be partially explained by our findings on the replication packages presented later in Table 16 (Appendix B).

(f) Statistical Tests. The use of statistical tests is an essential part of testing any hypothesis and drawing any meaningful conclusions [52]. About half of the primary studies (51%, 36/70) conduct one or more statistical tests to draw their conclusions (e.g., superiority over the existing

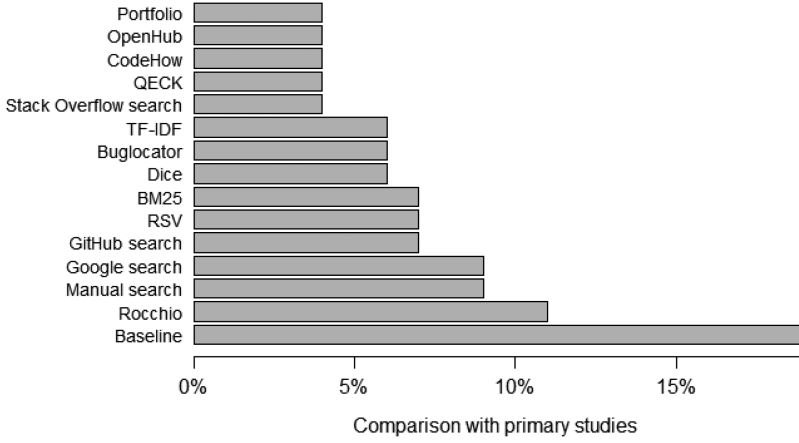


Fig. 20. Top-15 frequently used validation targets by primary studies.

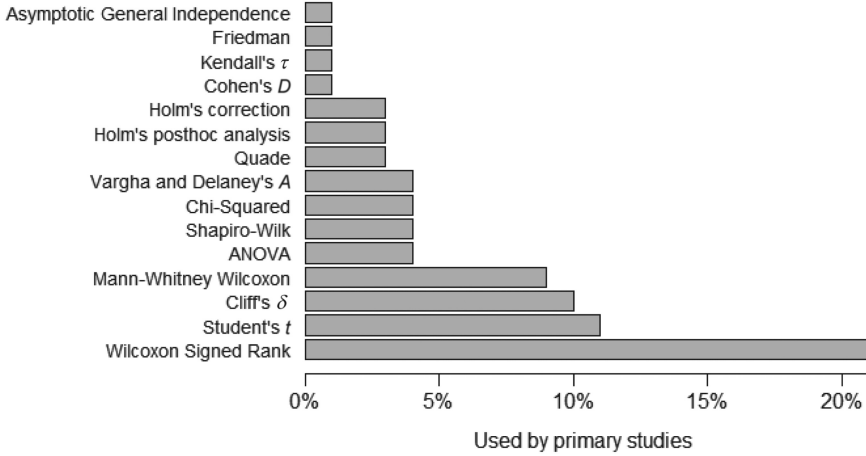


Fig. 21. Statistical tests used by primary studies.

alternatives). Table 8 provides an overview of the conducted tests, and Figures 21 and 22 show their usage statistics.

From Figure 21, we see that Wilcoxon Signed Rank is the most frequently used statistical test. It has been adopted by 20% or more (15/70) of our primary studies. It is a non-parametric paired test for two related, dependent samples, which makes it suitable for comparison between a proposed technique and a competing alternative. Student's t is another paired test that assumes normality for its samples. It has been used by 11% of the primary studies. However, Cliff's δ is a non-parametric test to determine the effect size between any two samples, which is often accompanied by another test—Mann-Whitney Wilcoxon. Both of these tests are suitable for the samples with unknown distribution and are used by 8% to 10% of our primary studies. Besides these four, there have been 11 other statistical tests (e.g., ANOVA, Shapiro-Wilk, Chi-squared), each of which has been used by $\approx 5\%$ of our 70 primary studies.

We also investigate how the top-five frequent tests (according to Figure 21) were used by the primary studies in different working contexts (e.g., bug localization, Internet-scale code search). From

Table 8. Statistical Tests Conducted by Primary Studies

Category	Statistical Test	Overview
Non-parametric	Mann-Whitney Wilcoxon	The test determines whether any two samples are drawn from the same distribution or not. It does not assume any specific distribution for its samples.
	Wilcoxon Signed Rank	The test determines if two related, dependent samples are significantly different or not from each other. Both samples should have an equal number of elements.
	Friedman	The test determines if three or more samples have identical population distributions or not. The data can be either quantitative or ordinal.
	Kendall's τ	The test determines the strength and direction of association between any two measured quantities.
	Quade	The test is an extension of the Wilcoxon Signed Rank test for three or more related samples.
	Shapiro-Wilk	The test determines whether a sample is drawn from a normal distribution or not.
	Cliff's δ	The test quantifies the difference between any two samples beyond their p -value interpretation.
	Vargha and Delaney's A	The test quantifies the difference between any two samples using a probabilistic measure. It has a linear relationship with Cliff's δ .
	Holm's correction	The test is used to counteract the problem of multiple comparisons and to control the family-wise error rate.
Parametric	Paired t	The test determines whether two related, dependent samples are significantly different from each other or not. It also assumes t distribution for its samples.
	ANOVA	The acronym stands for Analysis of Variance. The test determines whether three or more independent populations are significantly different from each other, with the assumption that their distributions are normal.
	Chi-squared	The test determines the difference between any two categorical variables in the same population. It assumes χ^2 distribution for the population and shows how the two variables might be connected to each other.
	Cohen's D	The test quantifies the difference between any two samples with an assumption of normality and homogeneity in their variance. It often accompanies the t -test and ANOVA.
	Asymptotic General Independence	The test determines whether any two random variables are asymptotically independent of each other. Here, asymptotic aspect refers to a very large sample size for each variable.

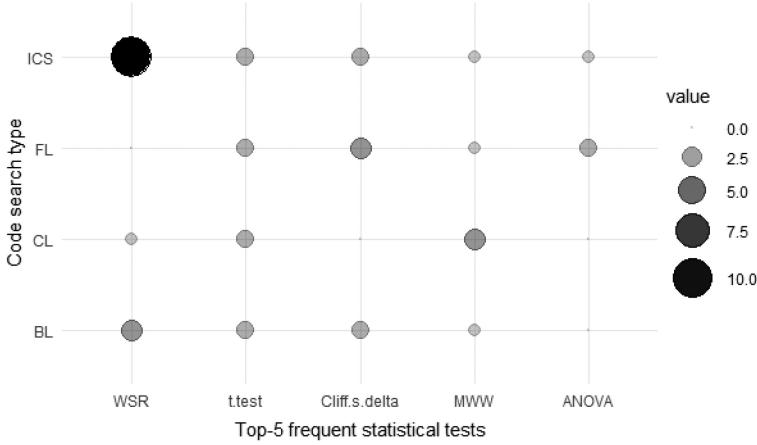


Fig. 22. Adoption of five frequently used tests by different search types. WSR, Wilcoxon Signed Rank; MWW, Mann-Whitney Wilcoxon; ANOVA, Analysis of Variance.

Figure 22, we see that Wilcoxon Signed Rank test has been more popular in Internet-scale code search than in local code searches. However, the t -test has been used in all four working contexts: bug localization, concept location, feature location, and Internet-scale code search. Both of the preceding tests involve paired comparison, which is often used to validate a proposed technique against existing baselines. It also should be noted that the primary studies from bug localization and feature location made use of at least four statistical tests, whereas the studies from Internet-scale code search adopted all top-five statistical tests in their experiments.

Summary of RQ₂: About 62% of our primary studies use re-enactment-based evaluation, 29% of them involve human participants, and only 7% of the studies combine both types of evaluation. About 80% of our studies that involve human participants involve fewer than 25 developers each. We found that the use of performance metrics (e.g., Hit@K, MAP, MRR) might vary across evaluation types or working contexts. We also found that 70 primary studies selected more than 100 subject systems for their experiments. The studies from Internet-scale code search use a smaller number of queries than the others for their evaluation. About 30% of our primary studies were also selected for comparison by the later studies. Our primary studies use a total of 15 statistical tests, where the Wilcoxon Signed Rank test has been adopted by more than 20% of the studies.

4.3 Answering RQ₃: Challenges and Limitations in Automated Query Reformulations for Source Code Search

Search queries often consist of one or more keywords that are unstructured. On the contrary, software code is full of structures (e.g., syntactic relations) and has an unrestricted vocabulary [69]. In addition, search queries and source code might be written by different persons using their own vocabularies that have little chance of being the same [53, 184]. Due to these inherent differences between a query and the code, constructing the right search query is challenging. Although each of our primary studies attempts to mitigate these challenges, they might not be entirely successful. We critically examine the methodology, evaluation, validation, and threats to validity sections from each study, and summarize their reported or implied limitations. Similar to RQ₁, we use the Grounded Theory approach (Section 3.5) to analyze the primary studies and detect their challenges and issues. Tables 9 and 10 show the key challenges and limitations of our primary studies. We

Table 9. Challenges and Limitations of the Primary Studies

ID	Challenge and Limitation
CH1	Injection of noisy keywords in the original query
CH2	Extra cognitive burden on the developers
CH3	Lack of generalizability
CH4	Issues toward practical adoption
CH5	Human bias and lack of rigor in evaluation
CH6	Inappropriate use of external tools and dependencies
CH7	Lack of a sound theory
CH8	Miscellaneous

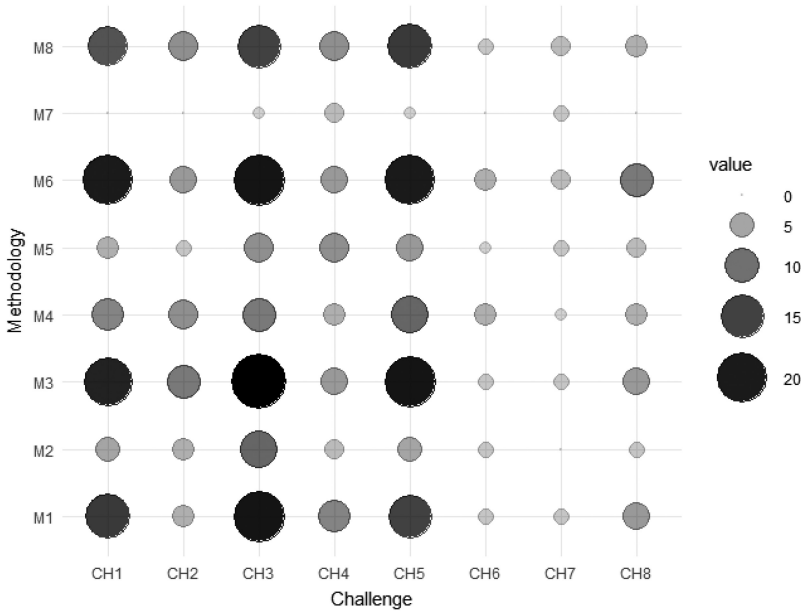


Fig. 23. Methodologies associated to different challenges encountered by the primary studies.

also determine the association between methodologies and challenges by combining the findings from RQ₁ and RQ₃, and Figure 23 summarizes our investigation. In this section, we discuss each of the key challenges and their mitigation strategies in detail.

CH1: Injection of Noisy Keywords in the Original Query. One of the major challenges of code search is the vocabulary mismatch between a query and the source code. In essence, each of our primary studies attempts to overcome this challenge. Unfortunately, almost half of them (51%) might fail due to their adoption of inappropriate methodologies and selection of noisy keywords. For example, several studies [60, 97, 108, 116, 210] rely on lexical similarity between a query and the API documentation to detect the relevant APIs and to expand the query. However, textual similarity alone might not be sufficient to overcome the vocabulary mismatch problems [53, 136], and thus noisy keywords (e.g., false-positive API classes) could be added to the original query [64, 65]. Many studies [55, 103, 104, 114, 176] adopt English language thesauri such as WordNet [129] to replace or expand a query with synonyms or semantically similar words. However, WordNet is based on natural language texts rather than source code. As a result, the keywords suggested by WordNet

Table 10. Primary Studies, Their Challenges, and Limitations

ID	Primary Studies	Total
CH1	S1, S5, S7, S14–S16, S19, S20, S23, S25–S29, S35, S36, S38–S46, S50, S51, S53, S54, S55, S57–S59, S63, S64, S70	36 (51%)
CH2	S3, S12, S13, S17, S24, S27–S31, S33, S35, S43, S47–S49, S53, S56	18 (26%)
CH3	S1, S2, S3, S4, S7, S8, S10, S11, S13–S16, S19, S22–S25, S27, S28, S30–S37, S39–S43, S45–S47, S49–S52, S54–S57, S59, S60, S64–S66, S69	49 (70%)
CH4	S2, S3, S6, S22, S27, S29, S30, S32–S34, S42, S44, S48, S50, S53, S54, S61–S64, S66–S68	23 (33%)
CH5	S1, S4, S6, S11–S13, S15–S18, S21, S22, S24, S26–S28, S30, S31, S34–S39, S42–S47, S51, S52, S58, S60, S63–S66, S69	39 (56%)
CH6	S35, S37, S48, S51, S53, S56, S61	7 (10%)
CH7	S9, S14, S35, S44, S55, S62, S65	7 (10%)
CH8	S8, S14, S17, S26, S27, S32, S36, S38, S45, S49, S51, S55, S57, S61, S63	15 (21%)

might not be appropriate for detecting relevant source code. Existing evidence [184] also suggests that the same word could have two different meanings when used in natural language texts and in source code, respectively. In other words, the keywords from WordNet can introduce noise to a search query. Thus, the preceding studies on query reformulation might not be able to overcome the vocabulary mismatch problem during their code search.

Several primary studies [47, 79, 121, 135, 154, 157, 159, 180, 213] expand a search query using relevance feedback mechanisms (details in Section 2) to mitigate the vocabulary mismatch problem. However, such an expansion might also hurt the initial query if the good keywords are not selected from the relevance feedback documents [107, 179]. Nie et al. [135] expand a query by collecting important keywords from relevant Q&A threads of Stack Overflow using PRF. Sirres et al. [180] extract important program elements (e.g., method invocations) from such threads. Similarly, Rahman and Roy [157] extract important API classes from the Q&A threads of Stack Overflow to expand a query. All of these studies apply various term weighting methods (e.g., TF-IDF [82]) to select their keywords. Unfortunately, the term weighting method alone might not be able to discard the noisy keywords if the proxy of term importance is not appropriate [151]. Several primary studies [90, 158, 215] use a bug report or a feature request as a query and collect candidate terms from relevant source code to reformulate the query. However, these approaches might also suffer from noisy keywords, as the issue reports and the source code might have different vocabularies and the relevant code could not be retrieved [38, 53]. Martie et al. [121] capture frequent identifier names from the source code retrieved by a given query and use them to expand the query. Huang et al. [79] mine frequently changed identifier names from the code change history to determine the intent of a search query. Eddy et al. [47] also expand a search query by repeating its structured terms (e.g., identifier names) multiple times during feature location. All of the preceding studies greatly rely on the identifier names to expand their queries. However, the identifier names could be poor, generic, and less descriptive [100, 190], which can hurt the original search queries.

A few primary studies [157, 213] learn word embeddings [208] from large corpora (e.g., Stack Overflow, GitHub). They attempt to mitigate the vocabulary mismatch problem by determining semantic relevance between a query and the source code. However, to learn word embeddings, both query keywords and code terms need to be present in the same corpus [23, 127, 128], which might not always hold true. Rahman et al. [159] mine frequent keyword-API co-occurrences from Stack Overflow Q&A threads and use the frequent API classes to expand a search query. However, their sole reliance on co-occurrence might also provide noisy keywords (e.g., false-positive API classes) that can hurt the original query during code search [157].

From Figure 23, we note that the primary studies facing this challenge (i.e., CH1) frequently use three different methodologies: use of term weights and relevance feedback (M1), extraction of semantic relations from co-occurrences and thesauri (M3), and API recommendation using data mining (M6). Although these methodologies are popular, they could be limited by their underlying assumptions, as discussed previously. For example, frequency or co-occurrence might not always be the indication of a keyword's importance [130, 151]. API recommendation can also return false-positive API classes or methods due to the lack of appropriate contexts [157]. Thus, these methodologies might inject noisy keywords into the original search query as a part of query reformulations.

Mitigation strategy. Frequency or co-occurrence might not always be a reliable proxy to keyword importance, which could lead to the selection of noisy keywords. They could be carefully combined with other available proxies (e.g., ambiguity, POS, position of a keyword) to avoid false positives during keyword selection.

CH2: Extra Cognitive Burden on the Developers. Several primary studies [47, 54, 55, 71, 121, 176] impose extra cognitive burden on the developers during query reformulation. Shepherd et al. [176] first introduce an interactive approach, namely *Find-Concept*, to support query reformulation in the context of concern location. Although their approach suggests relevant keywords from multiple sources (e.g., V-DO pairs, synonyms), a developer is ultimately responsible to choose the right keywords and to construct the final query. Hill et al. [71] extend this work with the phrasal concept analysis. Ge et al. [55] further extend Find-Concept by recommending keywords before and after the code search. All of the preceding studies argue in favor of developer involvement rather than a complete automation in query construction. Thus, in essence, they warrant a certain level of expertise or experience from the developers and might not be suitable for novice developers. Gay et al. [54] capture explicit feedback on a query from a developer and expand the query with the feedback to support concept location. Martie et al. [121] implement a similar idea to improve queries for Internet-scale code search. Although the developer's feedback could be useful, capturing it regularly could be infeasible and costly. Several primary studies also warrant significant learning and cognitive efforts from the developers, such as generation of test cases [102, 103], input code examples [18, 137, 197], and controlled search queries [92, 205]. To use the approach of Lemos et al. [102], a developer needs to express the information need as a complex test case. Balachandran [18] expect an input code example that can be transformed into an AST to retrieve the similar code examples. Wursch et al. [205] also expect a semi-structured query from the developers rather than a free-form query during code search. Wang et al. [192] provide multi-faceted information within IDE to support an interactive query reformulation during feature location. However, their technique warrants a certain level of domain expertise so that the developers can recognize the relevant program elements and can make important decisions during feature location [192].

Mitigation strategy. Given the significant learning curves of the preceding techniques, software practitioners might be reluctant to adopt them. Future work can focus on reducing the learning curves by capturing and leveraging the existing knowledge or expertise of the developers. For example, analysis of authored or reviewed code from the version control history might reveal the expertise of a developer, which can be leveraged to customize the preceding techniques.

CH3: Lack of Generalizability. About 70% (49/70) of our primary studies suffer from a lack of generalizability due to their limited datasets, queries, and evaluation. The majority of these studies [54, 87, 156, 158, 192, 212] evaluate their approaches using only Java-based subject systems. According to existing evidence [167], findings from Java-based systems might not always generalize for other systems such as C/C++-based systems. Almost all primary studies also rely on open source systems to conduct their experiments. Only a few studies [44, 140, 146] make use of

proprietary dataset for their experiments. Pérez et al. [146] collect railway software systems from their industry partner (CAF) to evaluate their work. Other studies analyze proprietary search logs from the Bing search engine [148] and Stack Overflow [29] to generate their reformulated queries. According to existing literature [139, 161], open source and closed source systems could be different in various aspects (e.g., development practices, bug resolution, code reviews), and thus findings from these two types of systems might not always align with each other.

Other studies [54, 63, 64, 135, 156, 158] use a single code search engine (e.g., Lucene) to evaluate their reformulated queries. However, the same query might not deliver the same performance across multiple search engines [132]. Several studies [18, 54, 79, 87, 103, 116, 172, 176, 196, 207] also make use of a limited set of queries (e.g., ≤ 50) for their evaluation. A few studies [81, 87] use only one subject system. Thus, their findings might suffer from the lack of generalizability as well. Several primary studies [60, 205, 210, 212] might not perform well for all types of queries due to their unique requirements for a given query (e.g., textual similarity to API classes [210], presence of nouns [212], syntactic correctness [205]), which cannot be always satisfied.

From Figure 23, we note that the primary studies suffering from this limitation (i.e., CH3) frequently adopt three methodologies: M1, M3, and M6. However, unlike the preceding ones, the lack of generalizability is closely associated to the dataset or evaluation mechanism rather than the methodologies adopted in query reformulations.

Mitigation strategy. To improve the generalizability of findings, future work should focus on designing comprehensive experiments. For example, subject systems should come from multiple programming languages (e.g., Java, C/C++, Python, Javascript) [167], development practices (e.g., open source, closed source) [139], and application domains. More subject systems and queries should also be chosen to rigorously evaluate any proposed technique. Furthermore, multiple search engines should be used to evaluate the original and reformulated queries to avoid spurious findings [63, 132].

CH4: Issues Toward Practical Adoption. About 33% (23/70) of our primary studies suffer from major issues that might prevent their adoption by software developers. Several studies [97, 120, 145, 172] make use of LSI and term-document co-occurrence matrix to reformulate their search queries. Although LSI might be useful to mitigate the vocabulary mismatch problem [53], it suffers from the *curse of dimensionality problem*.⁵ In other words, a large, sparse matrix is needed to store the semantic space, which can be both costly and inefficient. In addition, the semantic space needs to be updated frequently for each incoming query before its execution, which is time consuming and undesirable. Several approaches require non-trivial items such as test cases [102], code examples [18, 197], and salient keywords [55, 176] from developers, which could make developers reluctant to use these approaches. Several studies [38, 99, 131] make use of ground truth information to identify appropriate search keywords from a bug report. Although these studies might be useful to provide a proof of concept, they are not suitable for practical use since the ground truth is not generally known during query construction [131]. Several other studies involve costly computations such as GAs [91, 146], frequent AST construction [77], or ontology construction [81], which might outweigh the benefit of query reformulation. A few studies [20, 47, 212] need significant configuration management or parameter tuning, whereas the others use proprietary, publicly unavailable datasets (e.g., Bing search logs [148], Stack Overflow search logs [29]). There also exist other approaches [38, 54, 121] that involve the developers in costly trials and errors. Furthermore, 47% of our studies do not have a replication package and thus might not be replicated properly. All of these constraints and issues might prevent the developers from using $\approx 33\%$ of our primary studies to reformulate their search queries.

⁵http://mlwiki.org/index.php/Vector_Space_Models.

Mitigation strategy. Future work can take several steps to mitigate this threat. First, any cost-benefit analysis through human studies will inform the usability of a technique. Developers frequently used general-purpose search engines (e.g., Google) for code search that supports traditional, natural language queries [149]. Thus, they should not be forced to design non-traditional queries (e.g., test cases, example code) for code search. The designed tools should be easily integrated into the working context of a developer [152]. Furthermore, the replication packages should be available to foster reuse and adoption.

CH5: Human Bias and Lack of Rigor in Evaluation. About 56% (39/70) of our primary studies might suffer from subjective bias and the lack of rigor in their evaluation. Several studies [18, 102, 103, 121, 172, 213] evaluate their approaches involving undergraduate or graduate students in a laboratory setting. These participants neither represent the real users (e.g., professional developers) nor are adequate in number. For example, 80% of our studies that engage humans have fewer than 25 participants each (see Figure 19). These participants were also responsible for both the ground truth preparation and the query evaluation. Thus, there exists a good chance of subjective bias in the evaluation since the appropriate measures (e.g., inter-rater agreement, pooling method [213]) were not always taken. Replication of an existing work might also introduce implementation bias especially if the replication package is not available. A few studies [63, 71, 157–159, 196] might suffer from this bias. Many primary studies [47, 87, 155, 183, 196, 212] report their best performances using their well-tuned configurations against certain subject systems (see Table 5). Unfortunately, the same performance might not be guaranteed for other subject systems with the same configurations. Several studies [103, 145, 146] do not make use of any frequently adopted metrics, whereas the others [29, 81, 87, 112, 204] do not compare with established baselines from the literature. In addition, 49% of the primary studies do not perform any statistical test (e.g., significance tests), which is an important step of rigorous validation.

From Figure 23, we note that the studies suffering from the preceding limitation or challenge (i.e., CH5) frequently adopt four methodologies: M1, M3, M6, and M8. Unlike many of the preceding ones, this challenge is closely associated with the experiment design rather than the methodologies adopted in search query reformulations.

Mitigation strategy. Future work should involve appropriate participants (e.g., software practitioners) and a sufficient number of participants to mitigate the threats from human studies. The ground truth should also be carefully constructed using either a polling method or inter-rater agreement analysis [213]. The source code, experimental data, and other details should be made publicly available to facilitate smooth replication by third parties [84]. Furthermore, the conducted experiments should use appropriate performance metrics and statistical tests based on evaluation type and working contexts (see RQ₂ for details).

CH6: Inappropriate Use of External Tools and Dependencies. Over the past two decades, many tools, methods, and technologies from the IR and Natural Language Processing domains have been used to solve software engineering problems. Although they work with their default configurations, they need to be configured properly for software engineering problems for an optimal performance. They also need to be used with caution [63, 132]. For example, several studies make use of WordNet [60, 114, 176, 212] and the Stanford CoreNLP POS tagger [159, 210] in the context of software engineering problems. Both of these tools were trained on regular English texts (e.g., the Wall Street Journal) rather than the source code. Thus, the semantic inferences and POS tag predictions made by these tools could be either noisy or even inaccurate, as confirmed by the existing evidence [117, 184]. Many primary studies [54, 63, 86, 108, 135, 140, 196, 210] also use TF-IDF [82] as a de facto term weighting method to select important keywords from source code. However, TF-IDF was originally designed for regular texts (e.g., news articles, legal documents) rather than source code. Regular texts and source code differ in their syntax, structures, semantics,

and vocabulary [69]. Thus, the adoption of TF-IDF to select keywords from source code might not be appropriate. Several studies heavily depend on third-party items such as code change history [79, 90], semantic web technologies (e.g., SPARQL) [81, 205], tags from Stack Overflow [135], and input code examples [18, 197], and thus might be negatively affected by their absence.

Mitigation strategy. Future work can take several steps to mitigate this threat. For example, the established tools from other domains (e.g., WordNet [129], Word2Vec [127]) need to be carefully re-trained using software engineering data (e.g., source code, bug reports). Traditional techniques for keyword selection (e.g., TF-IDF [82]) should be adapted and tuned to software engineering data. In addition, the future techniques should depend on open source, well-maintained dependencies to avoid any potential integration issues.

CH7: Lack of a Sound Theory. Several primary studies [35, 79, 104, 140] make counter-intuitive or unsubstantiated theoretical assumptions in their approach. Huang et al. [79] attempt to determine a query's intent using the code change history. First, they mine the code change history of 631 GitHub projects and construct a database that contains thousands of <code segment, changes> pairs. Then they analyze the code segment retrieved by a query, identify its frequently changed identifier names using the database as query intent, and then reformulate the query with the intent. However, the idea of capturing a query's intent with the changed code lacks a strong theoretical justification. A few other studies [77, 204] might also have similar issues. According to Chaparro et al. [35], an ideal bug report contains three parts: OB, EB, and S2R. Using an empirical study, Chaparro et al. [35] demonstrate that OB can be used as a reformulated query for bug localization. However, how OB could be less noisy than the S2R was demonstrated neither theoretically nor empirically. According to a recent work [36], S2R often contains terms that better indicate what the bug is about and includes key terms. Thus, solely relying on OB might lead to the removal of important search keywords from a bug report [36, 131]. According to Lemos et al. [104], WordNet could be effective for query expansion during IDCS (interface driven code search) rather than KBCS (keyword-based code search). However, such a claim was also not theoretically justified.

Mitigation strategy. Future work can take a few steps to mitigate this threat. For example, the intuition or logical reasoning behind any proposed technique should be clearly articulated before presenting any experimental evidence. Comprehensive understanding of the relevant literature can help develop such intuition. Then any proposed theory should also be substantiated by strong empirical evidence to place it in the literature.

CH8: Miscellaneous. About 21% (15/70) of our primary studies suffer from miscellaneous issues and limitations as follows. First, *chronology of bug reports is not respected*. Zhang et al. [215] do not respect the chronology of bug reports through their use of 10-fold cross validation. In other words, they might have identified previous bugs from the history by training their approach on the later bug reports, which is impractical. Second, *underlying semantics are either overlooked or misused*. The primary studies on query quality prediction [64, 130] make use of lexical and syntactic properties of a query but mostly overlook the semantic aspect. They also target binary classification (e.g., good query and bad query), which might not be sufficient since the search queries can have non-binary quality levels (e.g., very good, good, bad, very bad). Huang et al. [77] consider two code segments similar if they have similar AST nodes. However, their approach does not guarantee that both code segments have similar semantics (i.e., implement the same programming task). Huang et al. [79] also assume that a query's intent can be better expressed with the changed identifier names rather than focusing on the semantics of the query, which is counter-intuitive. Third, *corner cases are not considered*. Vinayakarao et al. [191] extend the vocabulary of source code by annotating program entities (e.g., `int arr[]`) with their equivalent concepts from natural language texts (e.g., integer array). However, their approach, as the authors report, might fall short for the nested

program entities with subsumption problems. Kevic and Fritz [86] reformulate natural language query (i.e., change request) into source code elements by mining version history. However, their approach might be restricted to existing code elements and thus might be insufficient for change requests where new software features are requested.

Mitigation strategy. To mitigate the preceding threats, future work can take several steps including performing time-aware training with bug reports, leveraging query semantics, and considering corner cases in their techniques.

Summary for RQ₃: Almost half of our primary studies might add unexpected noise to a query during its reformulation, whereas 26% of them impose extra cognitive burden on the developers. The majority of these studies also suffer from a lack of generalizability due to their limited datasets, queries, and evaluation. About 33% of our studies suffer from the issues that might prevent their adoption by software developers, whereas more than half of them could have subjective bias in their evaluation. Furthermore, several studies heavily depend on third-party items for their query reformulations and the others make counter-intuitive assumptions in their approaches.

4.4 Answering RQ₄: Evidence of Existing Research on Automated Query Reformulation

We collect publication year, venue, and authors of each primary study to construct the evidence of existing researches on automated query reformulations. Such evidence not only demonstrates the interest and enthusiasm around the research topic but also shows the quality of the work being produced and published. Tables 11 and 12 and Figure 24 present various statistics on the conducted studies over the past 15+ years.

Table 11 shows the frequency of publications on automated query reformulations per calendar year. We see that the first work on automated query reformulation was published back in 2004. Marcus et al. [120] first introduced query reformulation in the context of concept location where they used word similarity derived from *latent semantic space* to automatically reformulate a given search query (e.g., *font style*). From 2005 to 2012, there were a few studies targeting query reformulations, and on average two to three papers were published every year. However, query reformulation has received much attention as a research topic since 2013. We believe that such attention or enthusiasm could be partially attributed to a few contemporary doctoral theses on query reformulations and code searches [62, 70]. Since 2013, there has been a significant increase in the research activities and publication counts. For example, from 2015 to 2021, 47 studies on query reformulations were published, in which 2016 and 2017 were the most productive years (e.g., 10 studies per year). It also should be noted that $\approx 80\%$ of our selected primary studies were conducted from 2013 to 2021.

Table 12 shows how many primary studies were published at each venue in different calendar years. We see that 50% (35/70) of our selected studies were published in the top venues of the Software Engineering domain, such as ICSE, ASE, ICSME, SANER, and MSR. About 28% of them (20/70) also made it into flagship conferences such as ICSE (A*) and ASE (A*). Many of these 70 studies were extended later and published in reputed journals such as EMSE and JSEP. There were also several Journal-First studies on query reformulations published in high-quality journals such as TSE, TOSEM, EMSE, JSEP, SPE, and TSC. Although ASE and ICSE communities pioneered this research topic, other communities (e.g., ICSME, MSR, SANER) have been catching up to speed especially in the past few years.

We also investigate the comparative research interest in query reformulation and in code search. It should be noted each primary study is related to code search, but each study on code search might

Table 11. Existing Publications on Query Reformulation Targeting Source Code Search

Year	Published Studies	Total
2004	S50 (WCRE)	1
2006	S18 (LNCS)	1
2007	S43 (ASOD)	1
2009	S12 (ICSE), S30 (ICSM), S55 (LNCS)	3
2010	S48 (ICSE), S51 (ASE)	2
2011	S3 (IST), S31 (ICPC), S47 (ASE)	3
2012	S8 (ASE)	1
2013	S6 (MSR), S10 (ICSE), S24 (ICSE), S28 (MSR), S52 (ASE)	5
2014	S1 (IST), S4 (ASE), S11 (ICSE), S27 (EMSE), S42 (MSR), S45 (MSR)	6
2015	S13 (SCAM), S14 (ASE), S15 (ASE), S33 (ICSME), S36 (SANER), S46 (ICSE), S53 (SANER)	7
2016	S5 (Internetware), S17 (VLC), S23 (SPLC), S29 (ICSE), S34 (TSC), S37 (Internetware), S38 (ASE), S39 (SANER), S41 (ICSE), S56 (ASE)	10
2017	S2 (ICIT), S20 (TSE), S21 (JSEP), S22 (ASE), S26 (ASE), S32 (TOSEM), S40 (SANER), S44 (ICSME), S49 (WSDM), S54 (JSEP)	10
2018	S7 (EMSE), S9 (DKE), S16 (FCS), S19 (ICSME), S25 (FSE), S35 (SPE), S60 (Access), S67 (ICSME), S68 (ICSME)	9
2019	S57 (IST), S58 (MAPL), S59 (AUSE), S61 (SAC), S63 (SPE), S65 (Access), S69 (TSE), S70 (ICPC)	8
2020	S62 (IST), S64 (Access)	2
2021	S66 (ICSE)	1
		70

not be related to query reformulation. Thus, we repeat the relevant part of our study selection process and identify the studies on code search as well. Figure 24 summarizes our analysis using a bar plot. According to our investigation, 34% (70/207) of the code search studies focus on query reformulations. As shown in Figure 24, a significant fraction of code search studies target query reformulation every year. For example, a maximum of 23 code search studies were conducted in 2015, and 7 of them focused on search query reformulations.

We also identify the authors from each of our 70 primary studies and calculate their contribution frequency. We found a total of 172 software engineering researchers who contributed to the research of automated query reformulations to improve code search. Appendix E shows the authors who contributed to at least two primary studies.

The demand for low-cost, accurate, and easy-to-use solutions for code search is likely to grow in the coming years due to the constant growth of open source repositories (e.g., GitHub, BitBucket) and the increasing complexity or size of modern software systems. Reformulation of a query is an inevitable step of code search regardless of the search context. Thus, automated query reformulation is likely to be an active, interesting research topic for years to come.

Table 12. Existing Publications on Query Reformulation Targeting Source Code Search

Venue	Published Studies	Total
ASE	S4 (2014), S8 (2012), S14 (2015), S15 (2015), S22 (2017), S26 (2017), S38 (2016), S47 (2011), S51 (2010), S52 (2013), S56 (2016)	11
ICSE	S10 (2013), S11 (2014), S12 (2009), S24 (2013), S29 (2016), S41 (2016), S46 (2015), S48 (2010), S66 (2021)	9
ICSM/ICSME	S19 (2018), S30 (2009), S33 (2015), S44 (2017), S67 (2018), S68 (2018)	6
WCRE/SANER	S50 (2004), S36 (2015), S39 (2016), S40 (2017), S53 (2015)	5
IST	S1 (2014), S3 (2011), S57 (2019), S62 (2020)	4
MSR	S6 (2013), S28 (2013), S42 (2014), S45 (2014)	4
Access	S60 (2018), S64 (2020), S65 (2019)	3
Internetwork	S5 (2016), S37 (2016)	2
SPE	S35 (2018), S63 (2019)	2
LNCS	S18 (2006), S55 (2009)	2
JSEP	S21 (2017), S54 (2017)	2
EMSE	S7 (2018), S27 (2014)	2
TSE	S20 (2017), S69 (2019)	2
ICPC	S31 (2011), S70 (2019)	2
VLC	S17 (2016)	1
WSDM	S49 (2017)	1
AUSE	S59 (2019)	1
FSE	S25 (2018)	1
ASOD	S43 (2007)	1
SCAM	S13 (2015)	1
DKE	S9 (2018)	1
TOSEM	S32 (2017)	1
TSC	S34 (2016)	1
ICCIT	S2 (2017)	1
SPLC	S23 (2016)	1
MAPL	S58 (2019)	1
SAC	S61 (2019)	1
FCS	S16 (2018)	1
		70

Summary of RQ₄: To date, according to our investigation, a total of 70 primary studies on automated query reformulations supporting code search have been conducted. About 80% of them were conducted in the past 9 years. This trend not only indicates the timeliness or relevance but also the attention or enthusiasm around this research topic. A large group of 172 researchers contributed to this topic. About 50% of the primary studies were published at top venues in the Software Engineering domain such as ICSE, ASE, ICSME, MSR, and SANER. Given the increasing size and complexity of modern software systems, the research activities on automated query reformulations and code searches are likely to increase in the coming years.

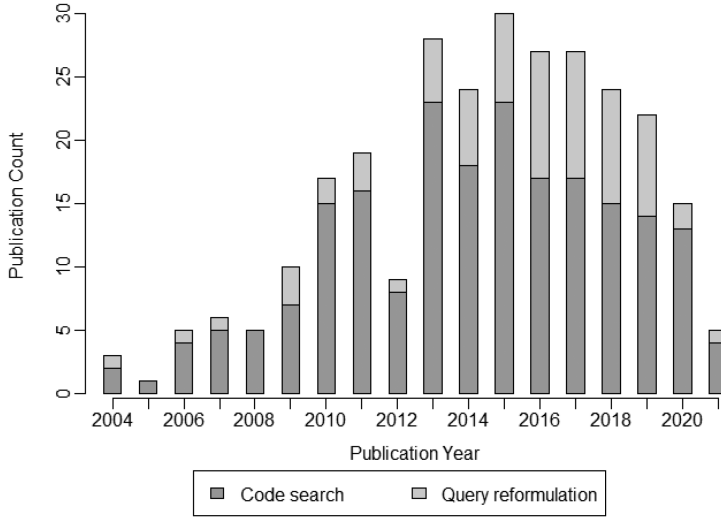


Fig. 24. Publications on code search and query reformulation.

4.5 Answering RQ₅: A Closer Look at Local Code Search and Internet-Scale Code Search

Local code search such as bug localization, concept location, or feature location attempts to find the location of a software bug, a concept, or a feature, respectively, within the code of a software system. However, Internet-scale code search attempts to find out relevant, reusable code segments from thousands of software projects stored at Internet-scale code repositories (e.g., GitHub). More specifically, whereas the former detects code locations that need to be modified, the latter finds code segments that can be reused. Although these two searches differ in their working context and end goal, they could overlap in several other aspects. In this systematic literature review, we analyze 70 primary studies in which 58% of them reformulate queries for local code searches and the remaining 42% of studies attempt to improve Internet-scale code search through query reformulations (e.g., see Figure 10). We refer to these two groups of studies as QR_{LCS} and QR_{ICS} , respectively. A closer look at the query reformulations supporting these two searches can provide valuable insights. In this section, we compare and contrast between these two groups of primary studies based on their underlying methodologies, algorithms, evaluation methods, challenges, and limitations.

Underlying Methodologies and Algorithms in Query Reformulation. Among 70 primary studies, 40 studies (QR_{LCS}) reformulate queries to support local code searches (e.g., bug localization, concept location), whereas the remaining 30 studies (QR_{ICS}) target Internet-scale code search. Figure 25 shows the methodologies adopted by these two groups of primary studies. We see that term weighting and the relevance feedback mechanism are frequently used to select keywords and to reformulate queries by the primary studies associated with local code searches. About 23% of them adopt this methodology, whereas the ratio is only 11% for the studies from QR_{ICS} . In local code search, there are several software artifacts such as bug reports and feature requests from which the keywords can be chosen based on their weight or appropriateness, which explains the popularity of term weighting methods. However, in Internet-scale code search, query keywords are often chosen by the developers on the fly according to their information needs. Semantic relations, word co-occurrences, and thesauri have also been used by 15% and 25% of the primary studies from these two groups, respectively. However, the most popular methodologies for query

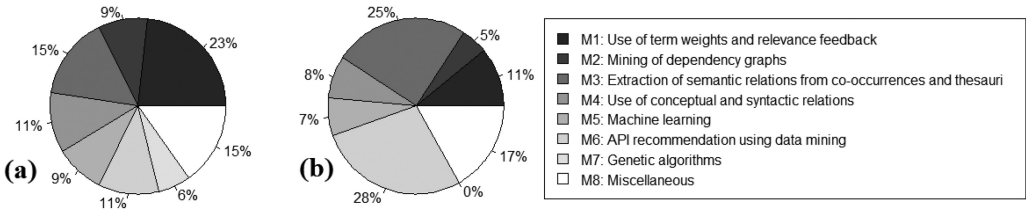


Fig. 25. Use of methodologies by the primary studies supporting local code search (a) and Internet-scale code search (b).

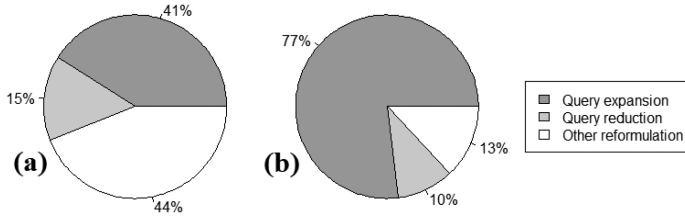


Fig. 26. Types of reformulation provided by the primary studies supporting local code search (a) and Internet-scale code search (b).

reformulations that support Internet-scale code search are data mining and API recommendation. About 28% of the studies from QR_{ICS} adopt this methodology, whereas the ratio is only 11% for the counterpart. In Internet-scale code search, search queries are often short (e.g., two to three keywords), and thus they require an expansion to retrieve the desired code [17, 166]. Many primary studies [42, 76, 157, 159, 213] expand these queries with relevant API classes or methods mined from large software repositories (e.g., GitHub, Stack Overflow), which explains the popularity of API recommendation methodology (i.e., M6). Several studies [103, 104, 106, 114, 191] also expand these short queries with synonyms or similar words from well-known thesauri such as WordNet [129]. This explains a high usage ratio (25%) of thesaurus-based methodology (i.e., M3) by the studies from QR_{ICS} (see Figure 25(b)).

Figure 26 shows the types of query reformulations performed by primary studies supporting both local and Internet-scale code searches. We see that query expansion is quite common for both groups (e.g., 41%–77%). However, queries are more frequently expanded in Internet-scale code search due to their short length [15]. Query reduction is less common for both groups of studies. Interestingly, in local code search, a significant number of queries (e.g., 44%) receive other types of reformulations, such as the combination of query expansion, reduction, or replacement.

Figure 27 shows how relevance feedback is used to reformulate queries by the primary studies. We see that query reformulations in local code search use comparatively more feedback. About 54% of them use either direct feedback from developers or PRF from a search engine. However, only 33% of studies from Internet-scale code search (QR_{ICS}) use any form of feedback to reformulate their queries. It should be noted that these QR_{ICS} studies capture less feedback, although they involve human developers more frequently than their counterpart (see Section 4.2(d)). Any local code search takes place on a single software project, whereas Internet-scale code search deals with thousands of cross-domain software projects. Thus, the results from Internet-scale code search might be noisier, and capturing relevance feedback against them could be costly and challenging. Such high cost might have discouraged any form of relevance feedback. According to Figure 27, 46% to 67% of studies from both groups did not use any form of relevance feedback to reformulate their search queries.

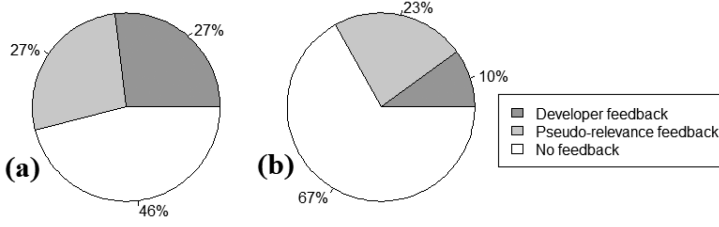


Fig. 27. Relevance feedback by the primary studies supporting local code search (a) and Internet-scale code search (b).

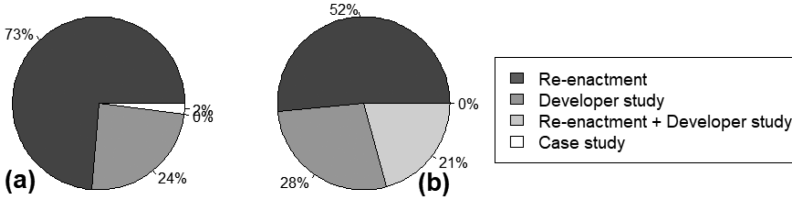


Fig. 28. Evaluation methods used by the primary studies supporting local code search (a) and Internet-scale code search (b).

Evaluation and Validation in Query Reformulation. Among 70 primary studies, 45 (64%) studies use re-enactment-based evaluation, 18 (26%) studies involve human participants, and 6 (9%) studies use a combination of both (see Figure 11). Figure 28 shows a breakdown of these statistics for two groups of primary studies. We see that 73% of the studies supporting local code searches evaluate their queries through *re-enactment* using historical artifacts (e.g., bug reports, version control history). On the contrary, 52% of the studies supporting Internet-scale code search adopt the same where they make use of existing benchmarks and programming Q&A sites (e.g., Stack Overflow). However, human participants were more frequently involved in Internet-scale code searches. For example, $\approx 50\%$ ($28\% + 21\%$) of the studies from the Internet-scale code search involve human developers in their evaluation. Such a ratio is only 24% for the counterpart. Existing primary studies from local code search often construct their dataset and ground truth from the version control history (e.g., bug-fixing commits) of open source projects that are widely available. On the contrary, historical data from Internet-scale code search such as search engine logs are proprietary artifacts that might not be publicly available. Thus, these studies often involve human developers to evaluate their queries.

Figure 29 further contrasts between two groups of our primary studies based on their use of popular performance metrics. We see that Hit@K, recall, MAP, and MRR are generally popular among both groups. However, the studies from local code search are more interested in QE. In local code searches such as concept location, bug localization, or feature location, the quick identification of the first correct result is important. The underlying idea is that once the first correct document is located, the other relevant documents can be easily located through their structural dependencies (e.g., call graphs) within a software project [62]. The same might not be true for Internet-scale code search, and 7% of their studies use QE. Their search results are retrieved from thousands of cross-domain software projects, and static dependencies might not be applicable beyond a single project. We also see that unlike QR_{LCS} , QR_{ICS} studies are more interested in developer-oriented metrics such as spent time, NDCG, and Likert that capture a developer's impression on the results retrieved by a query.

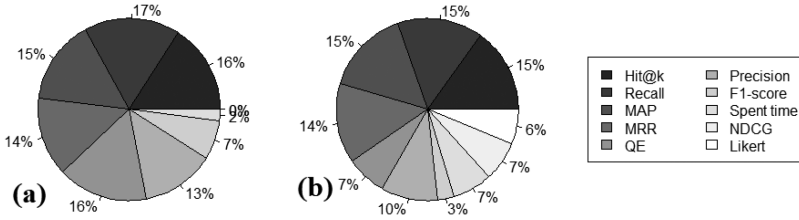


Fig. 29. Popular evaluation metrics used by the primary studies supporting local code search (a) and Internet-scale code search (b).

Challenges and Limitations in Query Reformulation. Since the studies supporting local code search (QR_{LCS}) and the studies supporting Internet-scale code search (QR_{ICS}) overlap in their methodologies (see Figure 25), they might suffer from similar types of challenges or limitations. Figure 30 summarizes our analysis in this aspect. We see that the studies from both groups are similarly affected by several challenges and limitations (see Table 9). About 17% of QR_{LCS} studies could suffer from noise in their queries, whereas the corresponding statistic is 21% for the QR_{ICS} studies. In local code search, developers can select keywords from existing artifacts (e.g., bug reports), whereas they need to select keywords on the fly during Internet-scale code search. Thus, the later queries need more extension (see Figure 27) and might have a higher chance of getting noisy keywords. The existing query reformulation approaches in the local code search impose more cognitive burden on the developers than their counterpart. Changes to wrong code often have major negative consequences (e.g., subsequent bugs) [24, 25]. Thus, precisely locating the source code of interest is a major step in local code searches. On the contrary, developers might accept the partially relevant code examples during Internet-scale code search, which they can customize based on their needs. Both groups of studies also suffer from a lack of generalizability, which is the most prevalent issue. The findings from 23% to 28% of the studies from both groups might not be generalizable, as acknowledged or implied by their authors. Although existing software artifacts (e.g., software, bug reports, version control history) are extensively used to evaluate query reformulations in local code search, the majority of them target Java-based systems. According to existing evidence [167], the findings from Java-based systems might not generalize for other systems, such as C/C++. However, the datasets used to evaluate query reformulations in Internet-scale code search are either small or proprietary in nature. For example, 60% of the QR_{ICS} studies use 75 or fewer queries each in their evaluation. A few studies make use of proprietary items such as search logs from Bing [148] and the Stack Overflow search engine [29], which might not be available for a public reuse and replication. From Figure 30, we also see that both groups suffer similarly with the remaining issues including human bias, weak evaluation (see CH5 in Table 9), extreme external dependencies (see CH6 in Table 9), and a lack of sound theoretical justification (see CH7 in Table 9).

Table 13 further compares and contrasts the query reformulation approaches supporting local code search and Internet-scale code search. We see that they are different from each other in terms of the majority of dimensions such as query length, keyword selection methodology, operational context, search intent, and even ranking priority. For example, in a local code search, developers attempt to precisely locate the source code entity that needs a change. Changing the wrong code entity can have serious consequences [24, 25]. On the contrary, in Internet-scale code search, developers attempt to locate the most relevant code example that can be reused to implement a programming task. Despite these differences, the two groups of studies are also surprisingly similar across several dimensions, such as their performance metrics (e.g., Hit@K, recall, MAP) and experienced challenges or limitations (e.g., lack of generalizability).

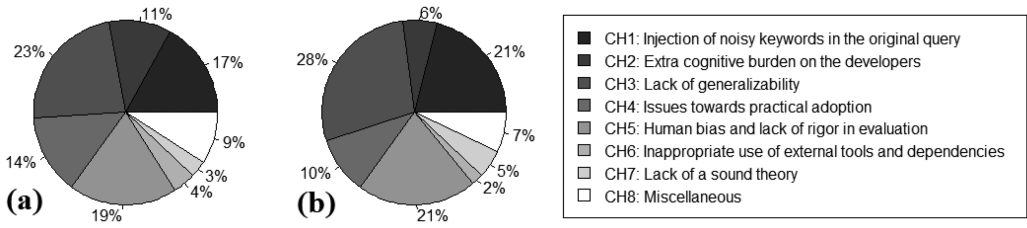


Fig. 30. Challenges and limitations of query reformulations targeting local code search (a) and Internet-scale code search (b).

Summary of RQ₅: About 58% of our primary studies support local code searches (e.g., concept location, bug localization), whereas the remaining 42% focus on Internet-scale code search. The query reformulation approaches supporting these searches share their methodologies, algorithms, and performance metrics, and even suffer from similar challenges and limitations. Appropriate keyword selection is an essential step in any query reformulation. In local code search, keywords are often selected using various term weighting methods (e.g., TF-IDF), whereas data mining and API recommendations are adopted in the context of Internet-scale code search. The primary studies from these two searches also differ in their reformulation types, evaluation methods, and human participation. However, both groups of studies suffer from a common set of issues, such as a lack of generalizability, weak evaluation, and noisy queries.

4.6 Answering RQ₆: Recommended Practices in Query Reformulations for Code Search

Query reformulation is an integral part of source code search regardless of the working context. Our systematic review using 70 primary studies informs the frequently adopted methodologies (RQ₁) and evaluation methods (RQ₂), and identifies their common challenges and limitations (RQ₃). Although query reformulation practices could be significantly different for local and Internet-scale code searches (RQ₅), there are several recommended, well-established practices that should be followed. In this section, we discuss several recommended practices in query reformulations and code search based on experimental evidence and the general understanding of community.

(a) Capture Search Queries from Human Users. In local code search, the *title* and *description* of a change request have been frequently used as baseline queries assuming that they need a reformulation [155, 156, 168, 195, 217]. However, in Internet-scale code search, several primary studies [105, 108, 116, 180, 210] use the title of a question from Stack Overflow as the baseline query. Yet, these queries might not always represent an actual user's queries [63]. Thus, although capturing search queries from secondary sources might be necessary for an exhaustive evaluation, queries should be also collected from actual users (e.g., professional developers).

(b) Choose Appropriate Granularity for Corpus Documents. Unlike regular texts (e.g., news article), source code has different granularity levels: package, file, class/module, method, and line. To search for code using IR methods, the source code of a project needs to be indexed at an appropriate granularity level. Many primary studies [158, 168, 195, 217] choose files and classes to index their corpus as a part of bug localization. Since software bugs might be scattered through multiple methods [138], a granularity level such as *file* or *class* might be appropriate. However, most studies from a concept or feature location [63, 71, 156] use methods to index their corpus. They identify the relevant entity in the code that needs a change [63]. Choosing a small granularity such as *method* is likely to help the post-search analysis by developers who can leverage their cognitive abilities

Table 13. Comparison Between Query Reformulations for Local Code Search and Internet-Scale Code Search

Dimension	Local Code Search (LCS)	Internet-Scale Code Search (ICS)
Query and search intent		
Initial query (ad hoc)	Preprocessed version of a bug report, a change request, or a feature request	Developer-chosen keywords
Initial query length	Comparatively long	Short (e.g., 1–3 keywords [17])
Keyword selection	Performed with term weighting and relevance feedback (M1)	Performed with data mining and API recommendation (M6)
Query reduction	Used by 15% of the studies	Used by 10% of the studies
Operational context	Software change tasks (e.g., bug fixing)	General-purpose programming tasks
Search intent	Precisely locating an entity of interest	Locating the most relevant code example to a programming task
End goal	Making changes to the existing code	Implementing a programming task
Relevance feedback	Used by 54% of the studies	Used by only 33% of the studies
Corpus, ground truth, and evaluation		
Corpus documents	Source documents (e.g., classes, methods) from a single software project (e.g., JEdit [54])	Source documents from thousands of software projects (e.g., Sourcerer [16])
Ground truth construction	Commits from the version control history [63, 158]	Relevance judgment from developers using a pooling method [213]
External resources	Past bug reports, bug-fixing history, author history, knowledge graph [108]	WordNet, Stack Overflow, GitHub
Ranking priority	First correct result should rank high	All correct results should rank high
Popular metrics	Hit@K, recall, MAP	Hit@K, recall, MAP
Miscellaneous		
Major challenges	Lack of generalizability, developer bias, weak evaluation, and noisy queries	Lack of generalizability, developer bias, weak evaluation, and noisy queries
Experience required	Domain knowledge (e.g., design, architectures) of a system under study	Relevant API libraries
Research evidence	58% of the primary studies	42% of the primary studies

[71, 192]. In Internet-scale code search, developers often look for short, working code examples [134], which also makes *method* a feasible choice for corpus indexing [124, 152].

(c) Apply Standard Preprocessing to Source Code. Corpus documents (e.g., classes, methods) need to be preprocessed as a part of indexing. Traditionally, three preprocessing steps have been applied to source code: *stop word removal*, *token splitting*, and *stemming*. First, stop words are frequently used words, but they convey very little semantics. Our primary studies often use a standard list of stop words [1] and programming keywords [2] to preprocess their corpora. Several studies [63, 156] remove such words that have a length of less than three characters or that occur in more than 25% of the corpus documents. Second, source code is full of structured entities such as camel-case tokens, snake-case tokens, method invocations, and field invocations. Our primary studies [35, 63, 155, 156] split these entities using appropriate regular expressions and advanced algorithms (e.g., Samurai [49]). Third, stemming is a widely used preprocessing step that extracts the root of any given word. Two stemming algorithms—*Porter* [143] and *Snowball* [144]—have been frequently used by the primary studies. However, the impacts of stemming have been mixed according to a few studies [73, 87, 168]. For example, stemming might have a strong impact on short queries but negligible impact on large queries [73]. Thus, it should be used selectively based on the type or size of queries and the search task at hand (e.g., bug localization).

(d) Choose Appropriate Reformulations to Improve a Query. Queries can be reformulated in three different ways: expansion, reduction, and replacement (Section 2). Given an initial query, appropriate reformulation techniques should be chosen to reformulate the query. Developers frequently use short queries to search for reusable code on the Internet [17, 149, 166]. Similarly, they choose a few keywords from a change request as a query and execute the query to detect the code of interest (e.g., faulty code, software feature). If a query is short, it is expanded with relevant, complementary keywords [71, 160]. However, long queries (e.g., preprocessed bug report) are reduced by discarding noisy, irrelevant keywords from them [38, 63]. Similarly, hard, difficult queries are replaced with better alternatives [31, 57]. Sometimes, a combination of multiple reformulation techniques might be required where machine learning can be employed to suggest the best reformulated query [63, 155].

(e) Use Multiple Search Engines to Evaluate the Same Query. To retrieve relevant source code, a query needs to be executed with a search engine. In a local code search (e.g., bug localization, concept location), primary studies frequently use two search engines—*Lucene* [6] or *Indri* [168]—to execute their queries. Lucene is a VSM-based search engine that combines Boolean search and the BM25 algorithm [178]. However, Indri combines probabilistic language modeling and the BM25 algorithm [168]. These engines can be installed against a local code repository and are easy to use. Several studies [42, 116, 157, 160] also use Lucene to execute their queries in Internet-scale code search. A few studies make use of well-established search engines such as Google [160], GitHub search [180], and Stack Overflow search [105, 180]. According to existing literature [63, 132], the same query can perform differently with different search configurations, which could pose a threat to the generalizability of findings. Thus, the same reformulated query needs to be evaluated with multiple search engines to mitigate the threat.

(f) Choose Appropriate Performance Metrics for Evaluation. Queries need to be evaluated using appropriate performance metrics [201]. Our investigation (see Figure 13) suggests that several metrics, such as Hit@K, MAP, MRR, QE, precision, and recall, have been frequently used by the primary studies for the past 15+ years. Although precision and recall are traditional metrics, Hit@K, MAP, MRR, and QE have received significant attention since 2013. We found that these metrics have been widely used in both local and Internet-scale code searches (see Figure 15). Yet, they are more prevalent in re-enactment-based evaluation than in developer studies (see Figure 14). Thus, future studies can choose them for their re-enactment-based evaluation. However, Likert and spent time can be used for developer study based evaluation.

(g) Choose the Appropriate Dataset for Experiments. Our analysis suggests that the use of an experimental dataset (e.g., subject systems) could vary based on the context of code search (see Table 5). For example, primary studies frequently use individual systems in local code searches (e.g., bug localization, concept location), whereas large-scale software repositories (e.g., GitHub, SourceForge, Stack Overflow) are frequently used in Internet-scale code search. The widely used subject systems can be found in Figure 17, which could be a good starting point for future work. We also note that only a few studies evaluate their query reformulation techniques using proprietary systems (e.g., CAAF [140]), whereas the majority use open source software systems, which could lead to threats to generalizability (e.g., see CH3 in Table 9) [88, 139]. Thus, future work should use appropriate systems from both development practices to achieve generalizability [201].

(h) Compare with State-of-the-Art Techniques. Comparison with state-of-the-art techniques is essential to place one's work in the literature [66]. Table 7 records all of the techniques used by our primary studies for their validation. We see that validation targets could be different based on the working context. However, a few techniques (e.g., BM25 [178], Rocchio [163], RSV [162]) were used for validation across multiple code search contexts. Figure 20 shows the frequently used validation targets, which could be a good starting point for future work.

(i) Use Appropriate Statistical Tests to Answer Research Questions. To test any null hypothesis, statistical tests are essential [52, 201]. According to our investigation, 51% (36/70) of our primary studies conduct one or more statistical tests. Table 8 outlines the use cases of various tests, whereas Figures 21 and 22 focus on their uses by the primary studies. From Figure 21, we see that three non-parametric tests (Wilcoxon Signed Rank, Mann-Whitney Wilcoxon, and Cliff's δ) and one parametric test (Student's t) have been frequently used by the primary studies. Non-parametric tests are used when the probability distributions of test samples are not assumed, whereas parametric tests should be chosen when the samples have a normal distribution [177]. All of these items could be a reference point for future work for any statistical analysis.

(j) Discuss Limitations and Threats to Validity. Threats or limitations in research could emerge due to various choices made by the researchers [51, 201]. According to our investigation, primary studies in query reformulations suffer from eight different challenges or limitations (see Table 9). Interestingly, several challenges (e.g., lack of generalizability (CH3), injection of noise into the original query (CH1), human bias (CH5)) are common in both local and Internet-scale code searches (see Figure 30). As done in earlier work, future work should also mitigate and discuss their challenges and threats, which is key to advancing the current state of knowledge.

(k) Provide a Replication Package for Reuse. Among 70 primary studies, we found that 24 studies provide a complete replication package including dataset and executable code and 13 studies offer only partial information (e.g., dataset only), whereas 33 studies do not provide any replication package (Appendix B). Replication of any research work could be the key to its validity and long-term impacts [83, 84]. Thus, making a replication package publicly available is one of the recommended practices to foster extended work and reuse.

Summary of RQ₆: Although query reformulation practices could be significantly different between local and Internet-scale code searches, there is a set of practices that have been well accepted and recommended by the community. For example, collecting search queries from professional developers and selecting both open and closed source systems are recommended for designing an experiment. Choosing appropriate granularity levels and preprocessing steps is essential to indexing the corpus. The same reformulated query should be executed against multiple search engines to ensure a fair evaluation. However, comparison with state-of-the-art techniques and conducting statistical tests are required to validate a proposed technique. Furthermore, threats and limitations should be discussed and the replication package should be shared publicly to advance the current state of knowledge.

5 ANSWERING RQ₇: FUTURE RESEARCH DIRECTIONS

Our selection criteria and personal knowledge led to a set of 70 primary studies being identified since 1998 supporting code search through query reformulations. Among them, 40 studies focus on local code searches that involve software change tasks (e.g., bug localization, feature location, concept location). However, 30 studies reformulate queries to support Internet-scale code search. Despite these significant numbers of studies, we believe that there is still room for future work and many novel dimensions are yet to be explored. Based on our empirical and qualitative analyses, in the following we present a list of future research directions that can improve query reformulations and support code search.

5.1 Adapt Traditional Keyword Selection Algorithms to Bug Reports for Bug Localization

IR has been adopted by at least 20 software engineering tasks including bug localization [63]. Bug localization is a type of local code search where bug reports serve as a major source of queries. A

few recent studies [89, 96, 193] point out potential biases and limitations in this local code search. According to them, IR-based localization is only good when the bug reports contain localization hints (e.g., program entity names). However, these empirical studies use the whole texts of a bug report as a query, which are often noisy, and overlook the potential of optimal search queries. Mills et al. [131] recently conducted a large-scale empirical study using GAs, and they present positive evidence for both bug reports and IR-based localization. They suggest that the bug reports contain sufficient keywords to return the faulty source code at high ranks using IR. In particular, IR-based localization can succeed 67% to 88% of the time even if their bug reports do not contain any localization hints [131]. Similar finding has been reported by a recent work of Rahman et al. [151]. Thus, the real challenge becomes extraction of the right search keywords from a bug report regardless of its localization hints.

Future Work. Appropriate term weighting algorithms can play a major role in selecting the search keywords. To date, existing term weighting studies exploit each term's frequency [63, 82, 87, 163], statistical relations and syntactic dependencies [156, 158], or occurrence probability [30, 181, 183] as a proxy to its relative importance (a.k.a. term weight). However, many of these algorithms were designed for regular texts in the IR domain and might not be well configured for bug reports containing a mix of regular texts and structured entities (e.g., stack traces, test cases, program elements, commit diffs). Future studies can focus on better adapting the IR algorithms to these structured entities. For example, rather than treating these entities as regular texts, they can be represented as graphs and thus can be better leveraged with graph-based algorithms (e.g., the PageRank algorithm [27]) for keyword selection.

5.2 Design Practical GA-Based Query Reformulations for IR-Based Bug Localization

Mills et al. [131] demonstrate that GAs are capable of generating optimal or near-optimal queries from a bug report (that can localize faulty code) when the ground truth information is provided. Unfortunately, in practice, the ground truth might not be known when a search query is constructed to find a bug. Thus, the ground truth should not be a part of fitness calculation when designing a GA-based query reformulation technique. In other words, we need a fitness function that can identify the better one from two different search queries without needing their ground truth, which is a major challenge.

Future Work. Several studies [63, 64, 130, 132, 155] make use of query difficulty metrics (e.g., specificity, coherency [33]) and machine learning algorithms to predict a query's success against the search engines. However, these metrics have non-linear relationships with a query's real performance according to recent studies [130, 151]. One possible way to address this is to formulate the query construction as a multi-objective optimization problem, as demonstrated by Kim and Lee [91]. They design a complex fitness function that attempts to optimize 15 objectives while reformulating a bug report into an optimal query for IR-based bug localization. Future work can design one or more fitness functions that can be used in practice for GA-based keyword selection from bug reports. For example, the difficulty of a query could be a part of its fitness calculation where the easy queries will be prioritized over the difficult ones during evolutionary search.

5.3 Leverage Word Embeddings in Query Reformulation and Code Search

Several primary studies [55, 71, 104, 114] use English language thesauri such as WordNet [129] to expand their query with synonyms or semantically similar words. However, Sridhara et al. [184] suggest that the same word can have two different meanings when used in source code and when used in regular texts. Since WordNet is based on regular English texts, its suggested keywords might not be appropriate for source code search. Thus, several studies [75, 206, 207, 211] construct software-specific thesauri from the mining of software repositories (e.g., open source

projects, programming Q&A sites). However, construction and update of these thesauri are costly and their effectiveness in the query reformulation is not well tested. Recently, several studies [40, 157, 208, 213] have made use of word embedding technology to determine semantic similarity or relevance between any two software-specific words. A few of them [157, 213] use embeddings to reformulate queries for code search and report positive findings. However, the true potential of word embeddings in query reformulations might not have been explored yet.

Future Work. Word embedding technology approximates the meaning of a word using a high-dimensional numeric vector [127]. The vector places each word as a single coordinate within a high-dimensional semantic space (a.k.a. semantic hyperspace). Such a geometric approximation of word semantics can be leveraged for various tasks in both *local* and *Internet-scale code searches*. For example, by placing all keywords from a document (e.g., bug report, feature request, source code) in the semantic hyperspace, important keywords could be detected using various geometric theories (e.g., Hopkins statistics [19]). By visualizing the semantic hyperspace using appropriate tools (e.g., t-SNE [189]), we could make a better sense of relevance between a query and its target code, or the difference between OB and EB of a reported bug.

5.4 Complement Term Weighting Algorithms with Contextual Information

Determining the importance of a term within a body of texts (e.g., bug report, source document) has been as a major challenge [20, 87]. TF-IDF [82] is a term weighting algorithm that has been extensively used in the IR and Software Engineering domains. The algorithm determines a term's importance without considering its contextual information (e.g., surrounding terms). However, a term's semantics are often determined by its contexts [129, 211]. In addition, several primary studies demonstrate the benefits of incorporating context in their term weighting algorithms. To date, several contextual items, such as spatial code proximity [183], positional relevance [158, 181], term co-occurrences [55, 71, 155, 156, 176], syntactic dependencies [156], time-awareness [212], and structural awareness [20, 47, 158], have been incorporated in term weighting. However, their use was ad hoc and their roles in query reformulations were not investigated exhaustively.

Future Work. Although context can deliver useful information in term weighting, it should be used carefully to avoid noise. Future work can investigate how multiple contexts can be incorporated in the term weight calculation. GAs can be used to optimize the relative weights of multiple contexts during their combination [211]. Machine learning can also be used to determine complex, non-linear relationships between these contexts and a term's importance [87].

5.5 Leverage Stack Overflow in Query Reformulation for Local Code Search

Software developers frequently use natural language queries to find relevant code on the web [16, 17]. However, their queries are often short and thus fail to capture their information needs [17]. Existing findings [17, 103, 157, 213] suggest that the inclusion of relevant API classes or methods in a query can improve its chance of retrieving the relevant code. A few studies [135, 159, 180] analyze the Q&A threads from Stack Overflow to expand a natural language query and to support Internet-scale code search. For example, Rahman et al. [159] analyze co-occurrences of query keywords and API classes in the question title and accepted answer, respectively, and expand a query with frequently co-occurring API classes. However, the true potential of Stack Overflow in query reformulation might not have been explored yet.

Future Work. Several studies [112, 148, 172] perform machine translation, in which they translate a natural language query into relevant API classes or methods. However, they make use of proprietary search logs (e.g., Bing logs) that might not be publicly available for reuse. Interestingly, Stack Overflow can be leveraged to generate such logs where keywords and API classes can be extracted from question titles and accepted answers, respectively. Given the size

of Stack Overflow, millions of logs can be captured at low costs. Then the machine translation techniques based on these logs can be used to suggest either relevant API classes for a new feature request [187] or potentially buggy API classes for a bug report.

5.6 Minimize Query Worsening During Reformulation

Query reformulations have both benefits and costs. Existing studies suggest that automated query reformulation might improve the performance of a query up to 20% during code search [160, 212]. However, several studies [71, 140, 179] also question the idea of a complete automation in reformulation. Without human intervention, automated reformulations might add noisy keywords that could drift a query away from its original topic [179]. Thus, we need a mechanism to maximize the benefits and minimize the costs of automated query reformulations.

Future Work. Several studies [63, 64, 151, 155] use query difficulty metrics and machine learning to automatically identify the difficult queries and then reformulate them. However, unless the models are 100% accurate, there will be always false-positive results. Thus, the risk of query worsening due to reformulations is still prevalent. According to Dietrich et al. [44], human developers might perform well in removing irrelevant terms from a search query due to their cognitive abilities. On the contrary, automated techniques perform well in adding the relevant terms to a query [44]. Thus, the query reformulation task can be considered as multi-step changes to a query (e.g., query expansion, query reduction) in which human and machine capabilities are combined together.

5.7 Apply PRF to Internet-Scale Code Search

Relevance feedback is an important mechanism to reformulate a search query [67, 98]. However, capturing developers' feedback on queries by analyzing their results could be costly and sometimes impractical. Many primary studies [63, 154, 155, 183] thus use PRF (Section 2) as a feasible choice to reformulate their queries. PRF has been reported to improve the queries during code search [63, 155]. However, the majority of studies, except a few [79, 121], focus on local code searches such as concept location, bug localization, and feature location.

Future Work. In Internet-scale code search, the results are retrieved from thousands of open source projects, which could be noisier and less homogeneous than that from local code search. PRF naively assumes the top few results retrieved by a query as *relevant* [169], which might not apply to Internet-scale search. Thus, future work can adapt the PRF mechanism to mitigate the noise and to support the query reformulations in Internet-scale code search.

5.8 Leverage the PageRank Algorithm in Term Weighting and Source Code Retrieval

Rahman and Roy adapt PageRank [27], a popular algorithm from IR, to reformulate queries in several software engineering tasks such as bug localization [158], concept location [155, 156], and Internet-scale code search [157]. Other studies [109, 123, 174] make use of the PageRank algorithm to rank code snippets during their code search. However, the true potential of the PageRank algorithm in query reformulation and code search might not have been explored exhaustively.

Future Work. PageRank operates on a graph with nodes and edges, implements a notion of voting, and then identifies the most important nodes from the graph using a recursive computation [21, 126]. Source code has an unrestricted vocabulary [69] and is full of structures, entities, and dependencies among them (e.g., control dependency, data dependency), which can be represented as a graph [155]. Thus, unlike TF-IDF, PageRank is a natural choice for analyzing the source code. Although there have been a few attempts using static relationships [155, 158], future work can use the PageRank algorithm to exploit the code structures to support query reformulations and code search. For example, PageRank can be used to detect meaningful terms from PRF documents for query expansion and shortlist terms from source code for corpus indexing.

5.9 Standardize Query Reformulation in IR-Based Bug Localization

Antoniol et al. [12] first use the VSM to recover traceability links. Zhou et al. [217] later refine VSM as rVSM and incorporate past bug reports in IR-based bug localization. Saha et al. [168] make use of structures both from bug reports and from source code documents to localize the bugs. Wong et al. [202] boost up the bug-proneness score of a source document based on the stack traces found in a bug report. Sisman and Kak [182] and Wen et al. [199] incorporate version control history in IR-based bug localization. Finally, Wang and Lo [195] incorporate five major items—past bug reports, structures, stack traces, version history, and author history—from the literature and outperform the earlier approaches on IR-based localization. Thus, the existing literature often adopts an incremental approach and includes more and more external artifacts in their approaches. Although these artifacts have positive impact upon localization performance, their inclusion makes the proposed approaches *less scalable* and *less usable*. In addition, these approaches become subject to external dependencies, which is not an ideal choice from the design point of view. Such limitations might partially explain the reluctance of software practitioners in adopting IR-based bug localization techniques in their workflow [96, 138, 193, 218].

Future Work. Several approaches equipped with query reformulation [35, 63, 155, 158] make effective use of primary resources available to practitioners (e.g., *bug report* and *source code*), and localize the bugs with a competitive accuracy and at low cost. Given that IR has been adopted by at least 20 software engineering tasks, query reformulation has the potential to significantly improve them. For example, several tasks, such as bug localization [35], duplicate bug report detection [36], bug triaging, and bug report summarization, can benefit from query reformulations.

Summary of RQ₇: Despite significant research, there is still room for further work in automated query reformulations and code searches (e.g., bug localization, concept location, feature location, Internet-scale code search). Existing findings suggest that up to 88% of bug reports contain appropriate keywords that make good queries and can find the bugs using IR methods. Unfortunately, many existing techniques are not effective enough to identify these keywords. The selection of keywords could be further improved by adding more contexts (e.g., time awareness), designing an appropriate fitness function (for GA-based solutions), leveraging the structures from source code, or using neural language modeling (e.g., word embeddings). The adverse effects of automated query reformulations could be mitigated by combining both human cognitive power and tools' strengths. Large software repositories (e.g., GitHub) and programming Q&A sites (e.g., Stack Overflow) have a lot to offer to the research of query reformulations and code search. They store invaluable technical knowledge curated by a large technical crowd, which can be leveraged to support not only code searches but also other software development and maintenance activities.

6 THREATS TO VALIDITY

We identify several threats to the validity of our findings in this work. We did our best to mitigate the majority of these threats; however, a few of them can be addressed in the future surveys on automated query reformulations. We classify our identified threats into several standard categories and discuss them next.

6.1 Threats to Internal Validity

Threats to internal validity are associated with experimental or internal errors and human biases [211]. Selection of primary studies and their classification into certain categories could be a source of such threats. However, we take careful steps to mitigate them.

First, we choose 70 primary studies using a well-established, systematic approach (Section 3) that involves generating keywords from research questions using the PIO criterion, extensive searching against 11 widely used publication databases (e.g., IEEE Xplore, ACM Digital Library), and multi-level filtration based on specific criteria. We first collect a large set of 2,970 studies that match our search keywords, then carefully select 70 studies from them using six levels of noise filtration. Despite these careful, time-consuming steps, relevant studies might be omitted due to sub-optimal queries or the limitations of search engines provided by the publication databases (Section 3.3). To mitigate this threat, we added a few studies that were relevant (according to our prior work experience) but were accidentally missed by the study selection process. However, we still cannot rule out the possibility of missing a few relevant studies on the topic of interest.

Second, we use the Grounded Theory approach (Section 3.5) to classify the primary studies based on their adopted methodologies, algorithms (Section 4.1), and experienced challenges (Section 4.3). Grounded Theory has been a popular choice for qualitative analysis in Software Engineering research for decades [35, 58, 185]. Thus, the threats identified previously might be mitigated.

Third, local code search and Internet-scale code search differ in their working context and end goals. The former targets software change tasks (e.g., bug localization, feature location), and the latter focuses on code reuse or general-purpose programming tasks. One might argue about the design of our survey since the query reformulation studies from both searches were captured. However, according to our investigation, query reformulation approaches from these two searches have a significant overlap in their reformulation techniques (see Table 4 and Figure 25), evaluation methods (see Figures 28 and 29), and even strengths or limitations (see Figure 30). Thus, analysis restricted to only either local code searches or Internet-scale code search misses a significant number of strongly related studies from the literature. In addition, collecting both sets of studies allows us to compare and contrast between them, which has led us to meaningful insights (Section 4.5) and novel ideas for future work (Section 5). There also exists a large body of work on query reformulations in the IR domain [30, 33]. Many of our primary studies adapt their ideas from them. However, since we deal with software engineering problems such as code search, those studies from the IR literature were not included in our survey.

Fourth, our systematic review is restricted to only such studies that deliver either automated or semi-automated tool supports to reformulate queries for code search. In other words, it does not include empirical studies, developer surveys, and or sometimes even the simple incremental works. Such a choice was made to construct a homogeneous set of primary studies that solve a single research problem—that is, automated query reformulation to support code search.

6.2 Threats to External Validity

Threats to external validity relate to the generalizability of any reported findings [211]. Our primary studies are carefully chosen from a large population of 2,970 results retrieved from 11 popular publication databases (see Figure 4). Thus, these primary studies are likely to represent the core studies on query reformulation supporting code search in the literature. Consequently, our findings and insights derived from these studies could also generalize for the remaining, accidentally missed studies from the literature. Furthermore, we leverage the Grounded Theory approach for our qualitative analysis, which has been a standard practice for years [35, 37, 58, 185]. Thus, the threats to external validity might be mitigated.

6.3 Threats to Construct Validity

Threats to construct validity relate to the appropriateness of the evaluation methodology used in a study [211]. Since our selection of primary studies involves certain quality control mechanisms, such threats may arise. We select 70 studies through a well-established approach of extensive

database search and well-documented filtration criteria (see Figure 4). Furthermore, we control the quality of these studies by asking 10 common questions about their quality (Section 3.4). Although the majority of these studies pass the quality check, the remaining few do not meet certain quality standards (e.g., lack of rigorous evaluation and validation). However, we retain them in our analysis due to their strong relevance to our survey topic—query reformulation in source code search. It also should be noted that three studies [44, 46, 57] on traceability link recovery were also included in our survey due to their solid contributions in query reformulation algorithms.

6.4 Threats to Conclusion Validity

Threats to conclusion validity arise when the relationships between two variables are not backed up by solid evidence or reliable data [35]. In our qualitative analysis (Section 4.5), we compare and contrast between local code searches and Internet-scale code search, and make several statements about their characteristics, methodologies, strengths, and limitations. We also outline several future research directions based on our analyses. Our claims and suggestions were backed up not only by substantial analytical data (RQ₁, RQ₃) but also by empirical evidence (RQ₄). Thus, threats to the conclusion validity of our systematic survey might also be mitigated.

7 RELATED WORK

Source code search has two major steps: query construction and retrieval of relevant code [63, 186]. Several surveys [45, 165, 203] examine the evidence of existing research on local code searches (e.g., feature location, bug localization). They mostly focus on the studies that deal with the second step of code search: retrieval of relevant code. Unfortunately, to the best of our knowledge, there exists no systematic survey that deals with query reformulation approaches adopted in source code search. The closely related work is a similar survey [33] from the IR domain. Although IR-based methods have been widely adopted by at least 20 software engineering tasks [63, 131], they experience unique challenges (Section 4.3), which warrants further investigation [132, 193]. Our survey identifies the existing work on query reformulations supporting software engineering tasks, categorizes them based on their underlying methodologies or limitations, and then discusses their open issues.

Dit et al. [45] first provide a comprehensive survey of 89 studies on **Feature Location Techniques (FLT)**s. They analyze each of these studies using seven different dimensions: type of analysis, user input, data sources, output, programming language support, evaluation method, and software systems used. Based on an analysis, they also provide a detailed taxonomy on FLT studies. Rubin and Chechik [165] perform a similar survey using 24 studies on FLT. They provide necessary guidelines for software practitioners to choose the right FLT for their tasks. Wong et al. [203] analyze 331 studies on software fault localization and provide a detailed classification of their studies. Unlike the preceding work, our survey deals with not only feature location or bug localization but also other types of code searches such as concept location and Internet-scale code search. More specifically, we deal with a *cross-cutting concept*, namely query reformulation that concerns each of these code search activities. Our survey also analyzes 70 primary studies on query reformulations and categorize them based on several aspects such as query reformulation methodology (RQ₁), evaluation method (RQ₂), and their experienced challenges or limitations (RQ₃).

Zhang et al. [214] present a survey on bug report analysis where they classify the existing work on bug report optimization (e.g., bug severity prediction), bug report triage (e.g., bug report assignment), and bug fixing (e.g., fixing time prediction). Similarly, we also deal with the primary studies that perform bug report analysis. However, our focus was to capture the studies that accept a bug report as an input, construct appropriate queries from the input, and then localize the bug using these queries.

Table 14. Comparison Between Existing Surveys/Reviews and Our Systematic Reviews

Survey	Topic	FQR	Period	#Studies	SLR	Dimension
Carpineto and Romano [33]	Information retrieval	Yes	1993–2009	27	No	Performance limitations
Dit et al. [45]	Feature location	No	1992–2011	89	Yes	Type of analysis, user input, data sources, output, programming language support, evaluation methods, and software systems
Rubin and Chechik [165]	Feature location	No	1995–2010	24	No	Underlying technology, program representation, input, and user interactions
Wong et al. [203]	Fault localization	No	1977–2014	331	No	Underlying technology, subject programs, evaluation metrics, and critical aspects
Liu et al. [110]	Internet-scale code search	No	2002–2020	81	Yes	Code search technique, evaluation, and publication trends
Our systematic review	Concept location, feature location, bug localization, and Internet-scale code search	Yes	2004–2021	70	Yes	Underlying methodology, evaluation design, publication trends, limitations, and challenges

FQR, focus on query reformulation; SLR, use of systematic literature review.

A few other studies [35, 36, 110, 131] also investigate the existing work on query reformulations supporting code searches and other software engineering tasks. However, their analysis is limited and restricted to either local [35, 131] or Internet-scale code search [110]. Many of them also perform traditional literature review rather than a systematic review [93], which might not guarantee that all eligible primary studies are captured from literature. However, we conduct a systematic literature review guided by a set of research questions and well-crafted search queries (e.g., the PIO criterion). The quality of our primary studies is also controlled by clear exclusion/inclusion criteria and collected from 11 popular publication databases. Thus, our goal was to establish a comprehensive understanding of the existing research on query reformulations targeting code search. We also perform qualitative analysis using the Grounded Theory approach [58], a widely used qualitative technique, to answer several of our research questions. No earlier survey addresses the same topic in this scale, which makes our work novel. Table 14 also demonstrates how our survey fills in the gap of the literature by focusing on query reformulations in the context of source code searches.

8 CONCLUSION

Software maintenance can cost up to 80% of the total budget in modern software development. Searching for the code of interest (e.g., buggy code, reusable code example) is an integral part of software maintenance. Developers often choose a few important keywords capturing their information needs and attempt to find the code of interest either from a local software codebase or an Internet-scale code repository (e.g., GitHub). Unfortunately, existing evidence suggests that the developers often perform poorly in choosing their keywords. As a result, they spend a significant amount of their time manually reformulating queries and analyzing the corresponding search results. Toward this goal, several studies attempt to support the developers through auto-

mated query reformulations. In this article, we conducted a systematic literature review using 70 primary studies on automated query reformulations intended for source code search. We summarized our quantitative and qualitative findings next and answered our research questions briefly as follows:

- *RQ₁*: Existing approaches for query reformulation adopt eight major methodologies including term weighting, relevance feedback, semantic relations, thesaurus lookup, and data mining to reformulate their queries and to support code searches in various working contexts.
- *RQ₂*: Existing studies evaluate their approaches through re-enactment, developer surveys, or using both. They use a total of 30 performance metrics, including the popular ones (e.g., Hit@K, MAP). However, developer involvement is generally low, and the number of queries used to evaluate queries in Internet-scale code search is often small. About 30% of the previous approaches were selected for comparison with the later approaches.
- *RQ₃*: Existing query reformulation approaches suffer from eight major challenges and limitations including noisy keywords in their queries, vocabulary mismatch problem, lack of generalizability, human-induced biases, weak evaluation, and other prevailing issues that might prevent them from adoption by the software practitioners.
- *RQ₄*: Our selection criteria and personal knowledge led to a set of 70 primary studies from the past 15+ years, in which 80% of them were done in the past 9 years. These studies appeared in top Software Engineering venues such as ICSE, ASE, ICSME, TSE, TOSEM, and EMSE. Given the increasing size and complexity of modern software systems, research topics such as automated query reformulation and code search are likely to gain increased attention in the coming years.
- *RQ₅*: About 58% of our primary studies perform query reformulations to support local code searches (e.g., concept location, bug localization, feature location), whereas the remaining 42% focus on Internet-scale code search. The primary studies from these two searches differ in their adopted methodologies, reformulation types, evaluation methods, and human participation. However, both groups of studies suffer from a common set of issues, such as a lack of generalizability, weak evaluation, and noisy queries.
- *RQ₆*: Despite noticeable differences between local and Internet-scale code searches, there is a set of common, well-established practices in query reformulations targeting code search. These practices include selection of search queries from professional developers, selection of subject systems from both open and closed source domains, execution of a query against multiple search engines, and comparison of a proposed technique with state-of-the-art techniques through replication.
- *RQ₇*: Constructing the right queries for code search has been a challenge for decades. Despite significant research, there is still room for further investigation. Future work can focus on adding more contexts (e.g., time awareness) to their keyword selection algorithms, designing an appropriate fitness function for GA-based solutions, leveraging the structures from source code, or using neural language modeling (e.g., word embeddings) to deliver better queries for the code search.

APPENDICES

A SELECTED PRIMARY STUDIES

Table 15 shows the 70 primary studies selected for our systematic literature review.

Table 15. Selected Primary Studies for Systematic Literature Review

ID	Primary Study	Description
S1	Zamani et al. [212]	Query reduction using noun term selection and time-aware term weighting for concept location
S2	Satter and Sakib [172]	Query expansion using keyword co-occurrences across the past queries from code search logs
S3	Lemos et al. [102]	Ad hoc keyword selection from given test cases for semantic code search
S4	Wang et al. [196]	Query expansion using modified Rocchio's method and the re-ranking of results for improved code search
S5	Yu et al. [210]	Query reduction by extracting nouns, verbs, adjectives, and type information from a free-form query for code search
S6	Sisman and Kak [183]	Query reformulation using spatial code proximity for IR-based bug localization
S7	Sirres et al. [180]	Query expansion using structured code entities from the relevant Q&A threads of Stack Overflow
S8	Haiduc et al. [64]	Automatic performance prediction of a given search query using query difficulty analysis and machine learning without executing the query
S9	Pérez et al. [140]	Query reformulation for feature location using Rocchio, RSV, and Dice methods with software model families
S10	Haiduc et al. [63]	Best query reformulation suggestion using Rocchio, RSV, Dice methods, query difficulty analysis, and machine learning
S11	Kevic and Fritz [87]	Query reduction using advanced heuristics and logistic regression
S12	Hill et al. [71]	Semi-automated reformulation of NL queries with query keyword contexts and NL phrase extraction from method and field signatures
S13	Lemos et al. [103]	Query reformulation with an NL thesaurus (e.g., WordNet) in the context of interface-driven and keyword-based code searches
S14	Martie et al. [121]	Query reformulation using frequent identifier names from result files and other contextual information
S15	Lv et al. [116]	Query reformulation with relevant APIs by matching query texts and corresponding API documentation
S16	Li et al. [106]	Query expansion by using tag co-occurrences in the same and duplicate questions of Stack Overflow
S17	Ge et al. [55]	Query reformulation using pre-search (e.g., identifier and V-DO completion, term co-occurrences) and post-search recommendations (e.g., synonym suggestion, spelling corrections)

(Continued)

Table 15. Continued

ID	Primary Study	Description
S18	Dourdas et al. [46]	Query reformulation using ontology models for web service discovery
S19	Rahman and Roy [157]	Query reformulation using crowd-generated knowledge and large-scale data analytics derived from the Stack Overflow corpus
S20	Zhang et al. [213]	Query reformulation using semantically related API classes for the improved Internet-scale code search
S21	Sisman et al. [181]	Query reformulation using spatial code proximity, term ordering, Markov Random Field, and query conditioning
S22	Rahman and Roy [155]	Query reformulation using the term weighting method, CodeRank, query difficulty analysis, and machine learning
S23	Lapeña et al. [97]	Query reformulation of an NL query using textual similarity with product requirements for feature location
S24	Wang et al. [192]	Semi-automated query reformulation using structure, intent, and dependency facets of search results
S25	Rahman and Roy [158]	Query reformulation using bug report quality dynamics and graph-based term weighting algorithms
S26	Lin et al. [108]	Query reformulation using relevant API entities from RecoDec, conceptual knowledge, TF-IDF, and advanced heuristics
S27	Yang and Tan [207]	Query reformulation using semantically similar word pairs mined from comment-code contexts of source documents
S28	Howard et al. [75]	Query reformulation using semantically similar word pairs mined from leading comments and method signatures across projects
S29	Chaparro and Marcus [38]	Query reduction using gradual term removal with the help of QE
S30	Gay et al. [54]	Query reformulation using explicit relevance feedback from the developers and Rocchio's expansion method
S31	Panchenko et al. [137]	Query reformulation with AST generation and XPath representation of an example code snippet
S32	Mills et al. [130]	Best query prediction using query difficulty analysis and machine learning for concept location and traceability recovery
S33	Balachandran [18]	Query reduction by subtree generation from the AST of input code
S34	Nie et al. [135]	Query expansion using Rocchio's method and PRF from Stack Overflow
S35	Huang et al. [79]	Query expansion based on intent mining from code change history and machine learning
S36	Lu et al. [114]	Query expansion using POS tagging and synonym addition from WordNet for improved concept location
S37	Li et al. [105]	Query reformulation by term replacement with synonymous tags and by term expansion with related tags from Stack Overflow

(Continued)

Table 15. Continued

ID	Primary Study	Description
S38	Rahman and Roy [154]	Query reformulation using frequently co-occurred keywords from source code and Stack Overflow threads
S39	Rahman et al. [159]	Query reformulation using keyword-API co-occurrences in the Q&A threads of Stack Overflow
S40	Rahman and Roy [156]	Query reduction by selecting important keywords using graph-based term weighting, namely PageRank
S41	Raghothaman et al. [148]	Query reformulation using co-occurrence probability of keyword and API classes in the search engine logs
S42	Lemos et al. [104]	Query expansion using synonyms/antonyms from three thesauri: WordNet, Yang and Tan [207], and type thesaurus
S43	Shepherd et al. [176]	Query reformulation using V-DO extraction and their expansion using stemmed words, synonyms, and similar usages
S44	Chaparro et al. [35]	Query reduction using OB identification, Grounded Theory, and natural language discourse analysis
S45	Kevic and Fritz [86]	Query reformulation using NL-SCL mapping between changeset and IDE interaction history, and term weighting
S46	Gvero and Kuncak [60]	Query reformulation with API method signatures using sentence parse tree extraction, uni-gram, and probabilistic context-free grammar
S47	Kimmig et al. [92]	Query reduction using POS tagging, selection of nouns and verbs, and source code ontology
S48	Wursch et al. [205]	Query reformulation using semantic web based ontology models, Ginseng, and SPARQL
S49	Vinayakarao et al. [191]	Query/corpus reformulation using the mapping between programming concepts and their syntactic forms from Stack Overflow Q&A threads
S50	Marcus et al. [120]	Query reformulation using similar terms from the source code with LSI
S51	Gibiec et al. [57]	Query reformulation using web data mining and term weighting methods: domain term frequency, domain specificity, and concept generality
S52	Dietrich et al. [44]	Query reformulation using trace query transformation rules generated by the FP-growth algorithm
S53	Jiang et al. [81]	Query reformulation using ontology models across multiple repositories and candidate navigation path ranking
S54	Bassett and Kraft [20]	Query/corpus reformulation by weight boosting (e.g., repeating) of structured entities from various contexts in the source code
S55	Chatterjee et al. [39]	Query/corpus expansion by appending corresponding API documentations against method invocations
S56	Wang et al. [197]	Query suggestion by mining program dependency graphs from a given code segment
S57	Zhang et al. [215]	Query reformulation by expanding methods with neighboring methods based on semantic similarity, temporal proximity, and call dependency

(Continued)

Table 15. Continued

ID	Primary Study	Description
S58	Liu et al. [112]	Query expansion using co-occurring API methods from the underlying corpus using encoder-decoder transformation and conditional probability analysis
S59	Pérez et al. [141]	Query expansion using experts' descriptions of a feature, their confidence level, and Rocchio's method
S60	Lu et al. [113]	Query expansion using first- and second-order word co-occurrences, and object-oriented relations among the words
S61	Kim and Lee [90]	Query reformulation by adding attachment contents and by reducing noisy keywords from a bug report
S62	Kim and Lee [91]	Query reformulation using a GA and multi-objective optimization (e.g., query difficulty, OB, S2E, EB)
S63	Huang et al. [77]	Query expansion by deep learning change sequences and predicting the potentially changed terms for improved code search
S64	Pérez et al. [145]	Query reformulation for feature location using Rocchio's method, relevance feedback, and expert's descriptions of a feature as the feedback documents
S65	Wu and Yang [204]	Query reformulation using old code examples as relevance feedback and then extracting expansion keywords (i.e., intent) from the change history of those code examples
S66	Cao et al. [29]	Query reformulation by learning to predict reformulated query from search logs of Stack Overflow using sequence-to-sequence learning and the attention mechanism
S67	Lawrie and Binkley [99]	Query selection for bug localization using a GA and information need analysis as a fitness function
S68	Mills et al. [131]	Query reformulation for bug localization using a GA and QE as a fitness function
S69	Pérez et al. [146]	Query reformulation for feature location using a GA with expansion, reduction, and selection as a mutation operation, and textual similarity as a fitness function
S70	Da Silva et al. [42]	Query expansion for code example search using semantic similarity analysis, API suggestion, and term weighting (e.g., TF-IDF)

B QUALITY ASSESSMENT OF PRIMARY STUDIES

Table 16 shows answers for the questions outlined in Section 3.4. We represent our responses as follows: ● as “Yes,” ○ as “No,” and ◐ or ◑ as “Somewhat Yes.”

Table 16. Quality Assessment of Primary Studies

ID	Primary Study	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
S1	Zamani et al. [212]	●	●	●	●	●	●	●	●	●	◐
S2	Satter and Sakib [172]	●	●	●	●	●	●	●	○	◐	●
S3	Lemos et al. [102]	●	●	●	◐	●	●	●	●	●	●
S4	Wang et al. [196]	●	●	●	●	●	●	●	●	●	○
S5	Yu et al. [210]	●	●	●	●	◐	●	◐	○	◐	○
S6	Sisman and Kak [183]	●	●	●	●	●	●	●	●	●	○
S7	Sirres et al. [180]	●	●	●	◐	●	●	●	●	○	●
S8	Haiduc et al. [64]	●	●	●	○	●	●	●	●	●	●
S9	Pérez et al. [140]	●	●	●	●	●	●	●	●	○	○
S10	Haiduc et al. [63]	●	●	●	●	●	●	●	●	◐	◐
S11	Kevic and Fritz [87]	●	●	●	●	◐	●	●	○	●	●
S12	Hill et al. [71]	●	●	●	●	●	●	●	●	●	◐
S13	Lemos et al. [103]	●	●	●	●	●	●	●	●	●	●
S14	Martie et al. [121]	●	●	●	●	●	●	●	●	●	●
S15	Lv et al. [116]	●	●	●	●	●	●	●	●	●	○
S16	Li et al. [106]	●	●	●	●	●	●	●	●	●	○
S17	Ge et al. [55]	●	●	●	●	●	●	●	●	●	●
S18	Dourdas et al. [46]	●	●	●	◐	◐	○	◐	○	●	○
S19	Rahman and Roy [157]	●	●	●	●	●	●	●	●	●	●
S20	Zhang et al. [213]	●	●	●	●	●	●	●	●	●	◐
S21	Sisman et al. [181]	●	●	●	●	●	●	●	●	●	◐
S22	Rahman and Roy [155]	●	●	●	●	●	●	●	●	●	●
S23	Lapeña et al. [97]	●	●	●	●	◐	●	●	●	●	○
S24	Wang et al. [192]	●	●	●	◐	●	●	●	●	●	◐
S25	Rahman and Roy [158]	●	●	●	●	●	●	●	●	●	●
S26	Lin et al. [108]	●	●	●	●	●	●	●	●	●	○
S27	Yang and Tan [207]	●	●	●	◐	●	●	●	●	●	●
S28	Howard et al. [75]	●	●	●	◐	●	●	●	●	●	○
S29	Chaparro and Marcus [38]	●	●	○	◐	◐	●	●	○	◐	●
S30	Gay et al. [54]	●	●	●	●	●	●	●	●	●	◐
S31	Panchenko et al. [137]	●	●	●	●	●	●	●	●	●	○
S32	Mills et al. [130]	●	●	●	○	●	●	●	●	●	○
S33	Balachandran [18]	●	●	●	●	●	●	●	○	●	○
S34	Nie et al. [135]	●	●	●	●	●	●	●	●	●	●
S35	Huang et al. [79]	●	●	●	●	●	●	●	●	●	○
S36	Lu et al. [114]	●	●	●	●	◐	●	●	○	●	○
S37	Li et al. [105]	●	●	●	●	●	●	●	●	●	○

(Continued)

Table 16. Continued

ID	Primary Study	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
S38	Rahman and Roy [154]	●	●	●	●	◐	●	●	⊖	●	◐
S39	Rahman et al. [159]	●	●	●	●	●	●	●	●	●	●
S40	Rahman and Roy [156]	●	●	●	●	●	●	●	●	●	●
S41	Raghothaman et al. [148]	●	●	●	●	●	●	●	⊖	●	⊖
S42	Lemos et al. [104]	●	●	●	●	●	●	●	●	●	◐
S43	Shepherd et al. [176]	●	●	●	●	●	●	●	●	●	◐
S44	Chaparro et al. [35]	●	●	●	●	●	●	●	●	●	●
S45	Kevic and Fritz [86]	●	●	●	●	◐	●	●	⊖	●	●
S46	Gvero and Kuncak [60]	●	●	●	◐	◐	●	●	⊖	●	⊖
S47	Kimmig et al. [92]	●	●	●	●	◐	●	●	⊖	●	⊖
S48	Wursch et al. [205]	●	●	●	●	●	●	●	⊖	●	◐
S49	Vinayakarao et al. [191]	●	●	●	●	●	●	●	●	●	●
S50	Marcus et al. [120]	●	●	●	◐	●	●	●	⊖	●	⊖
S51	Gibiec et al. [57]	●	●	●	●	●	●	●	⊖	●	●
S52	Dietrich et al. [44]	●	●	●	●	◐	●	●	●	●	⊖
S53	Jiang et al. [81]	●	●	●	●	●	●	●	●	●	⊖
S54	Eddy et al. [47]	●	●	●	⊖	●	●	●	●	●	◐
S55	Chatterjee et al. [39]	●	●	●	◐	●	●	●	⊖	●	⊖
S56	Wang et al. [197]	●	●	●	●	●	●	●	●	●	⊖
S57	Zhang et al. [215]	●	●	●	●	●	●	●	●	●	⊖
S58	Liu et al. [112]	●	●	●	●	●	●	●	⊖	●	⊖
S59	Pérez et al. [141]	●	●	●	●	●	●	●	●	◐	⊖
S60	Lu et al. [113]	●	●	●	●	●	●	●	●	●	⊖
S61	Kim and Lee [90]	●	●	●	●	●	●	●	⊖	◐	⊖
S62	Kim and Lee [91]	●	●	◐	◐	◐	●	◐	⊖	◐	⊖
S63	Huang et al. [77]	●	●	●	●	●	●	●	●	⊖	⊖
S64	Pérez et al. [145]	●	●	●	●	◐	●	●	●	●	⊖
S65	Wu and Yang [204]	●	●	●	●	◐	◐	●	◐	●	⊖
S66	Cao et al. [29]	●	●	●	●	●	●	●	⊖	●	●
S67	Lawrie and Binkley [99]	●	●	●	◐	◐	●	●	⊖	●	◐
S68	Mills et al. [131]	●	●	●	◐	◐	●	●	●	●	●
S69	Pérez et al. [146]	●	●	●	●	●	●	●	●	●	●
S70	Da Silva et al. [42]	●	●	●	●	●	●	●	●	●	●

C EVALUATION METRICS

Table 17 defines the performance metrics frequently used by the primary studies.

Table 17. Metrics for Performance Evaluation

Metric	Overview	Formula
Hit@K	Percentage of all search queries (Q) for each of which at least one relevant result is returned within the top-K results.	$\text{Hit@K} = \frac{\sum_{q \in Q} \text{rel}(q, K)}{ Q }$
QE	QE refers to the rank of the first relevant result returned by a search query.	
MRR	Reciprocal rank is the multiplicative inverse of the rank of the first relevant result returned by a search query ($q \in Q$). MRR averages the reciprocal ranks for all search queries (Q).	$\text{MRR}(Q) = \frac{1}{ Q } \sum_{q \in Q} \frac{1}{\text{rank}(q)}$
Precision	Precision refers to the fraction of retrieved results that are relevant. Here, TP means true positive and FP means false positive.	$P = \frac{TP}{TP+FP}$
Recall	Recall refers to the fraction of ground truth results that are retrieved. Here, FN means false negative.	$R = \frac{TP}{TP+FN}$
F1-score	F1-score calculates the harmonized mean of precision and recall measures.	$\text{F1-score} = \frac{2 \times P \times R}{P+R}$
MAP	Precision@k calculates precision at every occurrence of a relevant result within a ranked list. Average precision (AP) averages over all Precision@k measures for a given query q . MAP calculates the mean of AP for all queries (Q). Here, P_k refers to precision@k and RR means the relevant results retrieved by a query.	$\text{AP}(q) = \frac{\sum_{k=1}^K P_k}{ RR }$ $\text{MAP}(Q) = \frac{\sum_{q \in Q} \text{AP}(q)}{ Q }$
NDCG	Discounted Cumulative Gain (DCG) determines the quality of a ranking technique by rewarding when the highly relevant results are found at the top positions and by penalizing when the highly relevant results are found at low positions of ranked list. NDCG calculates to the ratio between DCG and Ideal Discounted Cumulative Gain (IDCG), which is based on ground truth information. Here, gr_{el_k} refers to graded relevance of the k_{th} result based on ground truth information.	$\text{DCG}(q) = \sum_{k=1}^K \frac{gr_{el_k}}{\log_2(k+1)}$ $\text{NDCG}(q) = \frac{\text{DCG}(q)}{\text{IDCG}(q)}$ $\text{NDCG}(Q) = \frac{1}{ Q } \sum_{q \in Q} \text{NDCG}(q)$
Spent time	Time spent to accomplish a certain programming task.	
Likert	Assessment made by human participants on a numerical scale.	

D EXPERIMENTAL DATASET

Table 18 shows the datasets used by the primary studies.

Table 18. Dataset Used by Primary Studies

ID	Queries	Systems	Origin of Queries
S1	200	04	Change requests, version control history
S2	22	–	Code search logs
S3	34	–	Test cases
S4	70	01	Existing benchmark
S5	70	–	Stack Overflow questions
S6	4,393	02	Bug reports
S7	174	01	Stack Overflow questions, study participants
S8	164	05	Change requests
S9	217	02	Feature requests, feature description
S10	282	05	Change requests, study participants
S11	06	01	Change requests
S12	28	–	Study participants
S13	128	–	Study participants
S14	06	–	Code search logs
S15	129	–	Existing benchmark, code search logs, Stack Overflow questions, study participants
S16	82	–	Study participants
S17	4,917	–	Code search logs
S18	07	–	Study participants
S19	310	04	Existing benchmark, programming tutorial sites
S20	74	24,666	Existing benchmark, Stack Overflow questions
S21	4,684	03	Bug reports
S22	1,675	08	Change requests
S23	21	07	Bug reports, product descriptions
S24	04	–	Existing benchmark
S25	5,139	06	Bug reports
S26	300	03	Stack Overflow questions
S27	08	04	Existing benchmark
S28	97	–	Code comment
S29	424	08	Bug reports
S30	30	03	Change requests
S31	08	–	Code search logs
S32	1,043	15	Bug reports, use cases
S33	05	–	Study participants
S34	20	01	Stack Overflow questions
S35	54	–	Existing benchmark
S36	19	01	Existing benchmark
S37	14	–	Study participants
S38	510	02	Change requests
S39	175	03	Programming tutorial sites

(Continued)

Table 18. Continued

ID	Queries	Systems	Origin of Queries
S40	1,939	08	Change requests
S41	30	1	Code search logs
S42	13	100	Existing benchmark
S43	09	04	Bug reports
S44	451	21	Bug reports
S45	1,787	04	Change requests
S46	45	–	Hand-picked
S47	276	–	Study participants
S48	–	–	Case study
S49	169	–	Study participants
S50	88	01	Study participants, source code
S51	59	10	Regulatory documents
S52	383	01	Requirement documents
S53	5	01	Task description
S54	372	04	Feature description
S55	12	–	Study participants
S56	47	04	Task description
S57	531	05	Bug reports
S58	209	737	Existing benchmark
S59	43	23	Study participants
S60	20	1	Bug reports
S61	1,546	6	Existing benchmark
S62	1,546	6	Existing benchmark
S63	54	625	Existing benchmark
S64	43	1	Industry partner
S65	34	2,151	Existing benchmark
S66	65,103	1	Stack Overflow search logs
S67	613	13	Existing benchmark
S68	620	13	Bug reports
S69	121	23	Feature descriptions
S70	121	1	Stack Overflow questions

E CONTRIBUTING AUTHORS

Table 19 shows the authors who contributed to at least two primary studies.

Table 19. Contributing Authors

Author	#Studies
A. Marcus	7
M. M. Rahman	6
C. Roy	6
D. Lo	5
S. Haiduc	4
S. Wang	4
L. Pollock	3
G. Bavota	3
C. Lopes	3
R. Oliveto	3
Z. Li	2
O. Chaparro	2
Y. Zou	2
A. Kak	2
A. Lucia	2
K. Vijay-Shanker	2
Y. Zhang	2
E. Hill	2
F. Pérez	2
O. Lemos	2
D. Shepherd	2
J. Zhao	2
L. Jiang	2
J. Font	2
X. Peng	2
W. Zhao	2
S. Jain	2
T. Fritz	2
T. Menzies	2
T. Wang	2
G. Yin	2
B. Sisman	2
K. Kevic	2
C. Cetina	2

REFERENCES

[1] Google. n.d. Stop-Words. Retrieved July 10, 2023 from <https://code.google.com/archive/p/stop-words>.
[2] Oracle. 2015. Java Language Keywords. Retrieved July 10, 2023 from https://docs.oracle.com/javase/tutorial/java/nutsandbolts/_keywords.html.
[3] TechRepublic. 2017. Report: Software Failure Caused \$1.7 Trillion in Financial Losses in 2017. Retrieved July 10, 2023 from <https://tek.io/2FBNl2i>.

- [4] Reuters. 2018. Boeing Eyes Lion Air Crash Software Upgrade in 6 to 8 Weeks. Retrieved July 10, 2023 from <https://tinyurl.com/c3f2rsw3>.
- [5] Medium. 2019. The 737Max and Why Software Engineers Might Want to Pay Attention. Retrieved July 10, 2023 from <https://bit.ly/2CmeTqB>.
- [6] Apache Lucene. 2019. Apache Lucene Core. Retrieved July 10, 2023 from <https://lucene.apache.org/core>.
- [7] National Post. 2019. Boeing 737 Jets Grounded Globally as Officials Investigate Technical Issues Behind Fatal Crash. Retrieved July 10, 2023 from <https://goo.gl/ieBgYN>.
- [8] Tabnine. 2019. Codota Code Search. Retrieved July 10, 2023 from <https://www.codota.com/code>.
- [9] GitHub. 2019. GitHub Code Search. Retrieved July 10, 2023 from <https://github.com/search>.
- [10] National Post. 2019. Here's the Terrifying Reason Boeing's 737 MAX 8 Is Grounded Across the Globe. Retrieved July 10, 2023 from <https://goo.gl/GwXv6H>.
- [11] GitHub. 2023. Replication Package: A Systematic Review of Automated Query Reformulations in Source Code Search. Retrieved July 10, 2023 from <https://bit.ly/3eccmLZ>.
- [12] G. Antoniol, G. Canfora, G. Casazza, A. De Lucia, and E. Merlo. 2002. Recovering traceability links between code and documentation. *TSE* 28, 10 (2002), 970–983.
- [13] J. Anvik, L. Hiew, and G. C. Murphy. 2005. Coping with an open bug repository. In *Proceedings of OOPSLA/Eclipse*. 35–39.
- [14] M. Asaduzzaman, C. K. Roy, K. A. Schneider, and D. Hou. 2016. A simple, efficient, context-sensitive approach for code completion. *JSEP* 28, 7 (2016), 512–541.
- [15] S. Bajracharya and C. Lopes. 2009. Mining search topics from a code search engine usage log. In *Proceedings of MSR*. 111–120.
- [16] S. Bajracharya, T. Ngo, E. Linstead, Y. Dou, P. Rigor, P. Baldi, and C. Lopes. 2006. Sourcerer: A search engine for open source code supporting structure-based search. In *Proceedings of OOPSLA-C*. 681–682.
- [17] S. K. Bajracharya and C. V. Lopes. 2012. Analyzing and mining a code search engine usage log. *EMSE* 17, 4–5 (2012), 424–466.
- [18] V. Balachandran. 2015. Query by example in large-scale code repositories. In *Proceedings of SANER*. 467–476.
- [19] A. Banerjee and R. N. Dave. 2004. Validating clusters using the Hopkins statistic. In *Proceedings of FUZZY*, Vol. 1. 149–153.
- [20] B. Bassett and N. A. Kraft. 2013. Structural information based term weighting in text retrieval for feature location. In *Proceedings of ICPC*. 133–141.
- [21] R. Blanco and C. Lioma. 2012. Graph-based term weighting for information retrieval. *Inf. Retr.* 15, 1 (2012), 54–92.
- [22] D. M. Blei, A. Y. Ng, and M. I. Jordan. 2003. Latent Dirichlet allocation. *J. Mach. Learn. Res.* 3 (2003), 993–1022.
- [23] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. 2016. Enriching word vectors with subword information. *arXiv preprint arXiv:1607.04606* (2016).
- [24] A. Bosu. 2014. Characteristics of the vulnerable code changes identified through peer code review. In *Proceedings of ICSE-C (ICSE Companion '14)*. 736–738.
- [25] A. Bosu, J. C. Carver, M. Hafiz, P. Hilley, and D. Janni. 2014. Identifying the characteristics of vulnerable code changes: An empirical study. In *Proceedings of FSE*. 257–268.
- [26] J. Brandt, P. J. Guo, J. Lewenstein, M. Dontcheva, and S. R. Klemmer. 2009. Two studies of opportunistic programming: Interleaving web foraging, learning, and writing code. In *Proceedings of SIGCHI*. 1589–1598.
- [27] S. Brin and L. Page. 1998. The anatomy of a large-scale hypertextual web search engine. *Comput. Netw. ISDN Syst.* 30, 1–7 (1998), 107–117.
- [28] D. Cai, C. J. van Rijsbergen, and J. M. Jose. 2001. Automatic query expansion based on divergence. In *Proceedings of CIKM*. 419–426.
- [29] K. Cao, C. Chen, S. Baltes, C. Treude, and X. Chen. 2021. Automated query reformulation for efficient search based on query logs from Stack Overflow. In *Proceedings of ICSE*. 13.
- [30] D. Carmel and E. Yom-Tov. 2010. *Estimating the Query Difficulty for Information Retrieval*. Morgan & Claypool.
- [31] D. Carmel, E. Yom-Tov, A. Darlow, and D. Pelleg. 2006. What makes a query difficult? In *Proceedings of SIGIR*. 390–397.
- [32] C. Carpineto, R. de Mori, G. Romano, and B. Bigi. 2001. An information-theoretic approach to automatic query expansion. *ACM Trans. Inf. Syst.* 19, 1 (2001), 1–27.
- [33] C. Carpineto and G. Romano. 2012. A survey of automatic query expansion in information retrieval. *ACM Comput. Surv.* 44, 1 (2012), Article 1, 50 pages.
- [34] W. Chan, H. Cheng, and D. Lo. 2012. Searching connected API subgraph via text phrases. In *Proceedings of FSE*. Article 10, 11 pages.
- [35] O. Chaparro, J. M. Florez, and A. Marcus. 2017. Using observed behavior to reformulate queries during text retrieval-based bug localization. In *Proceedings of ICSME*. 376–387.

- [36] O. Chaparro, J. M. Florez, U. Singh, and A. Marcus. 2019. Reformulating queries for duplicate bug report detection. In *Proceedings of SANER*. 12.
- [37] O. Chaparro, J. Lu, F. Zampetti, L. Moreno, M Di Penta, A. Marcus, G. Bavota, and V. Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of ESEC/FSE*. 396–407.
- [38] O. Chaparro and A. Marcus. 2016. On the reduction of verbose queries in text retrieval based software maintenance. In *Proceedings of ICSE-C*. 716–718.
- [39] S. Chatterjee, S. Juvekar, and K. Sen. 2009. SNIFF: A search engine for Java using free-form queries. In *Proceedings of FASE*. 385–400.
- [40] C. Chen, Z. Xing, and Y. Liu. 2019. What's Spain's Paris? Mining analogical libraries from Q&A discussions. *EMSE* 24, 3 (2019), 1155–1194.
- [41] J. Cordeiro, B. Antunes, and P. Gomes. 2012. Context-based recommendation to support problem solving in software development. In *Proceedings of RSSE*. 85–89.
- [42] R. F. G. Da Silva, C. K. Roy, M. M. Rahman, K. Schneider, K. Paixo, and M. Maia. 2019. Recommending comprehensive solutions for programming tasks by mining crowd knowledge. In *Proceedings of ICPC*. 358–368.
- [43] B. Dagenais and M. P. Robillard. 2012. Recovering traceability links between an API and its learning resources. In *Proceedings of ICSE*. 47–57.
- [44] T. Dietrich, J. Cleland-Huang, and Y. Shin. 2013. Learning effective query transformations for enhanced requirements trace retrieval. In *Proceedings of ASE*. 586–591.
- [45] B. Dit, M. Revelle, M. Gethers, and D. Poshyvanyk. 2013. Feature location in source code: A taxonomy and survey. *JSEP* 25, 1 (2013), 53–95.
- [46] N. Dourdas, X. Zhu, N. Maiden, S. Jones, and K. Zachos. 2006. Discovering remote software services that satisfy requirements: Patterns for query reformulation. In *Advanced Information Systems Engineering*. Lecture Notes in Computer Science, Vol. 4001. Springer, 239–254.
- [47] Brian P. Eddy, Nicholas A. Kraft, and Jeff Gray. 2018. Impact of structural weighting on a latent Dirichlet allocation based feature location technique. *JSEP* 30, 1 (2018), e1892.
- [48] F. Ensan, E. Bagheri, and M. Kahani. 2007. The application of users' collective experience for crafting suitable search engine query recommendations. In *Proceedings of CNSR*. 148–156.
- [49] E. Enslen, E. Hill, L. Pollock, and K. Vijay-Shanker. 2009. Mining source code to automatically split identifiers for software analysis. In *Proceedings of MSR*. 71–80.
- [50] L. Favre. 2008. Modernizing software & system engineering processes. In *Proceedings of ICSENG*. 442–447.
- [51] R. Feldt and A. Magazinius. 2010. Validity threats in empirical software engineering Research—An initial survey. In *Proceedings of SEKE*. 374–379.
- [52] R. Fisher. 1955. Statistical methods and scientific induction. *J. R. Stat. Soc. Series B Methodol.* 17, 1 (1955), 69–78.
- [53] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais. 1987. The vocabulary problem in human-system communication. *Commun. ACM* 30, 11 (1987), 964–971.
- [54] G. Gay, S. Haiduc, A. Marcus, and T. Menzies. 2009. On the use of relevance feedback in IR-based concept location. In *Proceedings of ICSM*. 351–360.
- [55] X. Ge, D. C. Shepherd, K. Damevski, and E. Murphy-Hill. 2017. Design and evaluation of a multi-recommendation system for local code search. *J. Vis. Lang. Comput.* 39 (2017), 1–9.
- [56] M. Ghafari and H. Moradi. 2017. A framework for classifying and comparing source code recommendation systems. In *Proceedings of SANER*. 555–556.
- [57] M. Gibiec, A. Czauderna, and J. Cleland-Huang. 2010. Towards mining replacement queries for hard-to-retrieve traces. In *Proceedings of ASE*. 245–254.
- [58] B. G. Glaser and A. L. Strauss. 1967. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Aldine Publishing, Chicago, IL.
- [59] R. L. Glass. 2001. Frequently forgotten fundamental facts about software engineering. *IEEE Softw.* 18, 3 (2001), 112–111.
- [60] T. Gvero and V. Kuncak. 2015. Interactive synthesis using free-form queries. In *Proceedings of ICSE*. 689–692.
- [61] S. Haiduc. 2011. Automatically detecting the quality of the query and its implications in IR-based concept location. In *Proceedings of ASE*. 637–640.
- [62] Sonia Haiduc. 2013. *Supporting Text Retrieval Query Formulation in Software Engineering*. Ph.D. dissertation. Wayne State University.
- [63] S. Haiduc, G. Bavota, A. Marcus, R. Oliveto, A De Lucia, and T. Menzies. 2013. Automatic query reformulations for text retrieval in software engineering. In *Proceedings of ICSE*. 842–851.
- [64] S. Haiduc, G. Bavota, R. Oliveto, A De Lucia, and A. Marcus. 2012. Automatic query performance assessment during the retrieval of software artifacts. In *Proceedings of ASE*. 90–99.

- [65] S. Haiduc and A. Marcus. 2011. On the effect of the query in IR-based concept location. In *Proceedings of ICPC*. 234–237.
- [66] S. Hanneman. 2008. Design, analysis, and interpretation of method-comparison studies. *AACN Adv. Crit. Care* 19 (2008), 223–234.
- [67] D. Harman. 1992. Relevance feedback revisited. In *Proceedings of SIGIR*. 1–10.
- [68] J. H. Hayes, A. Dekhtyar, and S. K. Sundaram. 2006. Advancing candidate link generation for requirements tracing: The study of methods. *TSE* 32, 1 (2006), 4–19.
- [69] V. J. Hellendoorn and P. Devanbu. 2017. Are deep neural networks the best choice for modeling source code? In *Proceedings of ESEC/FSE*. 763–773.
- [70] Emily Hill. 2010. *Integrating Natural Language and Program Structure Information to Improve Software Search and Exploration*. Ph.D. dissertation. University of Delaware.
- [71] E. Hill, L. Pollock, and K. Vijay-Shanker. 2009. Automatically capturing source code context of NL-queries for software maintenance and reuse. In *Proceedings of ICSE*. 232–242.
- [72] E. Hill, L. Pollock, and K. Vijay-Shanker. 2011. Improving source code search with natural language phrasal representations of method signatures. In *Proceedings of ASE*. 524–527.
- [73] E. Hill, S. Rao, and A. Kak. 2012. On the use of stemming for concern location and bug localization in Java. In *Proceedings of SCAM*. 184–193.
- [74] R. Holmes and G. C. Murphy. 2005. Using structural context to recommend source code examples. In *Proceedings of ICSE*. 117–125.
- [75] M. J. Howard, S. Gupta, L. Pollock, and K. Vijay-Shanker. 2013. Automatically mining software-based, semantically-similar words from comment-code mappings. In *Proceedings of MSR*. 377–386.
- [76] Q. Huang, X. Xia, Z. Xing, D. Lo, and X. Wang. 2018. API method recommendation without worrying about the task-API knowledge gap. In *Proceedings of ASE (ASE’18)*. 293–304.
- [77] Q. Huang, Y. Yang, and M. Cheng. 2019. Deep learning the semantics of change sequences for query expansion. *SPE* 49, 11 (2019), 1600–1617.
- [78] Q. Huang, Y. Yang, X. Wang, H. Wan, R. Wang, and G. Wu. 2017. Query expansion via intent predicting. *IJSEKE* 27, 09n10 (2017), 1591–1601.
- [79] Q. Huang, Y. Yang, X. Zhan, H. Wan, and G. Wu. 2018. Query expansion based on statistical learning from code changes. *SPE* 48, 7 (2018), 1333–1351.
- [80] S. F. Hussain and G. Bisson. 2010. Text categorization using word similarities based on higher order co-occurrences. In *Proceedings of SDM*. 1–12.
- [81] S. Jiang, L. Shen, X. Peng, Z. Lv, and W. Zhao. 2015. Understanding developers’ natural language queries with interactive clarification. In *Proceedings of SANER*. 13–22.
- [82] K. S. Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *J. Doc.* 28, 1 (1972), 11–21.
- [83] N. Juristo. 2013. Towards understanding replication of software engineering experiments. In *Proceedings of ESEM*. 4–4.
- [84] N. Juristo and O. S. Gómez. 2012. Replication of software engineering experiments. In *Empirical Software Engineering and Verification: LASER Summer School 2008–2010*. Springer, 60–88.
- [85] D. Kelly and J. Teevan. 2003. Implicit feedback for inferring user preference: A bibliography. *SIGIR Forum* 37, 2 (2003), 18–28.
- [86] K. Kevic and T. Fritz. 2014. A dictionary to translate change tasks to source code. In *Proceedings of MSR*. 320–323.
- [87] K. Kevic and T. Fritz. 2014. Automatic search term identification for change tasks. In *Proceedings of ICSE*. 468–471.
- [88] A. Khanjani and R. Sulaiman. 2011. The aspects of choosing open source versus closed source. In *Proceedings of ISCI*. 646–649.
- [89] M. Kim and E. Lee. 2018. Are information retrieval-based bug localization techniques trustworthy? In *Proceedings of ICSE*. 248–249.
- [90] M. Kim and E. Lee. 2019. A novel approach to automatic query reformulation for IR-based bug localization. In *Proceedings of SAC*. 1752–1759.
- [91] M. Kim and E. Lee. 2020. ManQ: Many-objective optimization-based automatic query reduction for IR-based bug localization. *IST* 125 (2020), 106334.
- [92] M. Kimmig, M. Monperrus, and M. Mezini. 2011. Querying source code with natural language. In *Proceedings of ASE*. 376–379.
- [93] B. Kitchenham and P. Brereton. 2013. A systematic review of systematic review process research in software engineering. *IST* 55, 12 (2013), 2049–2075.
- [94] B. Kitchenham and S. Charters. 2007. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Technical Report. University of Durham.

- [95] Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *TSE* 32, 12 (2006), 971–987.
- [96] P. S. Kochhar, Y. Tian, and D. Lo. 2014. Potential biases in bug localization: Do they matter? In *Proceedings of ASE*. 803–814.
- [97] R. Lapeña, J. Font, F. Pérez, and C. Cetina. 2016. Improving feature location by transforming the query from natural language into requirements. In *Proceedings of SPLC*. 362–369.
- [98] V. Lavrenko and W. B. Croft. 2001. Relevance based language models. In *Proceedings of SIGIR*. 120–127.
- [99] D. Lawrie and D. Binkley. 2018. On the value of bug reports for retrieval-based bug localization. In *Proceedings of ICSME*. 524–528.
- [100] D. Lawrie, C. Morrell, H. Feild, and D. Binkley. 2006. What’s in a name? A study of identifiers. In *Proceedings of ICPC*. 3–12.
- [101] B. Lemaire and G. Denhière. 2008. Effects of high-order co-occurrences on word semantic similarities. *arXiv:0804.0143* (2008).
- [102] O. A. L. Lemos, S. Bajracharya, J. Ossher, P. C. Masiero, and C. Lopes. 2011. A test-driven approach to code search and its application to the reuse of auxiliary functionality. *IST* 53, 4 (2011), 294–306.
- [103] O. A. L. Lemos, A. C. de Paula, H. Sajnani, and C. V. Lopes. 2015. Can the use of types and query expansion help improve large-scale code search? In *Proceedings of SCAM*. 41–50.
- [104] O. A. L. Lemos, A. C. de Paula, F. C. Zanichelli, and C. V. Lopes. 2014. Thesaurus-based automatic query expansion for interface-driven code search. In *Proceedings of MSR*. 212–221.
- [105] Z. Li, T. Wang, Y. Zhang, Y. Zhan, and G. Yin. 2016. Query reformulation by leveraging crowd wisdom for scenario-based software search. In *Proceedings of Internetware*. 36–44.
- [106] Z. Li, G. Yin, T. Wang, Y. Zhang, Y. Yu, and H. Wang. 2018. Correlation-based software search by leveraging software term database. *Front. Comput. Sci.* 12, 5 (2018), 923–938.
- [107] J. Lin and G. C. Murray. 2005. Assessing the term independence assumption in blind relevance feedback. In *Proceedings of SIGIR*. 635–636.
- [108] Z. Lin, Y. Zou, J. Zhao, and B. Xie. 2017. Improving software text retrieval using conceptual knowledge in source code. In *Proceedings of ASE*. 123–134.
- [109] E. Linstead, S. Bajracharya, T. Ngo, P. Rigor, C. Lopes, and P. Baldi. 2009. Sourcerer: Mining and searching Internet-scale software repositories. *Data Min. Knowl. Discov.* 18, 2 (2009), 300–336.
- [110] C. Liu, X. Xia, D. Lo, C. Gao, X. Yang, and J. C. Grundy. 2020. Opportunities and challenges in code search tools. *CoRR abs/2011.02297* (2020).
- [111] D. Liu, A. Marcus, D. Poshyvanyk, and V. Rajlich. 2007. Feature location via information retrieval based filtering of a single scenario execution trace. In *Proceedings of ASE*. 234–243.
- [112] J. Liu, S. Kim, V. Murali, S. Chaudhuri, and S. Chandra. 2019. Neural query expansion for code search. In *Proceedings of MAPL*. 29–37.
- [113] J. Lu, Y. Wei, X. Sun, B. Li, W. Wen, and C. Zhou. 2018. Interactive query reformulation for source-code search with word relations. *IEEE Access* 6 (2018).
- [114] Meili Lu, X. Sun, S. Wang, D. Lo, and Yucong Duan. 2015. Query expansion via WordNet for effective code search. In *Proceedings of SANER*. 545–549.
- [115] A. D. Lucia, R. Oliveto, and P. Sgueglia. 2006. Incremental approach and user feedbacks: A silver bullet for traceability recovery. In *Proceedings of ICSM*. 299–309.
- [116] F. Lv, H. Zhang, J. Lou, S. Wang, D. Zhang, and J. Zhao. 2015. CodeHow: Effective code search based on API understanding and extended Boolean model. In *Proceedings of ASE*. 260–270.
- [117] A. Mahmoud and G. Bradshaw. 2015. Estimating semantic relatedness in source code. *TOSEM* 25, 1 (Dec. 2015), Article 10, 35 pages.
- [118] D. Mandelin, L. Xu, R. Bodík, and D. Kimelman. 2005. Jungloid mining: Helping to navigate the API jungle. In *Proceedings of PLDI*. 48–61.
- [119] C. D. Manning, P. Raghavan, and H. Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press, New York, NY.
- [120] A. Marcus, A. Sergeyev, V. Rajlich, and J. I. Maletic. 2004. An information retrieval approach to concept location in source code. In *Proceedings of WCRE*. 214–223.
- [121] L. Martie, T. D. LaToza, and A. V. D. Hoek. 2015. CodeExchange: Supporting reformulation of Internet-scale code queries in context. In *Proceedings of ASE*. 24–35.
- [122] P. McCullagh and J. Nelder. 1989. *Generalized Linear Models*. Chapman & Hall/CRC, Boca Raton, FL.
- [123] C. McMillan, M. Grechanik, D. Poshyvanyk, Q. Xie, and C. Fu. 2011. Portfolio: Finding relevant functions and their usage. In *Proceedings of ICSE*. 111–120.

- [124] C. Mcmillan, D. Poshyvanyk, M. Grechanik, Q. Xie, and C. Fu. 2013. Portfolio: Searching for relevant functions and their usages in millions of lines of code. *TOSEM* 22, 4 (2013), Article 37, 30 pages.
- [125] J. Miao, J. X. Huang, and Z. Ye. 2012. Proximity-based Rocchio's model for pseudo relevance. In *Proceedings of SIGIR*. 535–544.
- [126] R. Mihalcea and P. Tarau. 2004. TextRank: Bringing order into texts. In *Proceedings of EMNLP*. 404–411.
- [127] T. Mikolov, K. Chen, G. Corrado, and J. Dean. 2013. Efficient estimation of word representations in vector space. *CoRR abs/1301.3781* (2013).
- [128] T. Mikolov, K. Chen, G. Corrado, and J. Dean. 2013. Efficient estimation of word representations in vector space. *arXiv:1301.3781* (2018).
- [129] George A. Miller. 1995. WordNet: A lexical database for English. *Commun. ACM* 38, 11 (1995), 39–41.
- [130] C. Mills, G. Bavota, S. Haiduc, R. Oliveto, A. Marcus, and A. D. Lucia. 2017. Predicting query quality for applications of text retrieval to software engineering tasks. *TOSEM* 26, 1 (2017), Article 3, 45 pages.
- [131] C. Mills, J. Pantiuchina, E. Parra, G. Bavota, and S. Haiduc. 2018. Are bug reports enough for text retrieval-based bug localization? In *Proceedings of ICSME*. 381–392.
- [132] L. Moreno, G. Bavota, S. Haiduc, M. Di Penta, R. Oliveto, B. Russo, and A. Marcus. 2015. Query-based configuration of text retrieval solutions for software engineering tasks. In *Proceedings of ESEC/FSE*. 567–578.
- [133] L. Moreno, J. J. Treadway, A. Marcus, and W. Shen. 2014. On the use of stack traces to improve text retrieval-based bug localization. In *Proceedings of ICSME*. 151–160.
- [134] S. M. Nasehi, J. Sillito, F. Maurer, and C. Burns. 2012. What makes a good code example?: A study of programming Q & A in StackOverflow. In *Proceedings of ICSM*. 25–34.
- [135] L. Nie, H. Jiang, Z. Ren, Z. Sun, and X. Li. 2016. Query expansion based on crowd knowledge for code search. *TSC* 9, 5 (2016), 771–783.
- [136] D. Pal, M. Mitra, and S. Bhattacharya. 2015. Exploring query categorisation for query expansion: A study. *CoRR abs/1509.05567* (2015).
- [137] O. Panchenko, J. Karstens, H. Plattner, and A. Zeier. 2011. Precise and scalable querying of syntactical source code patterns using sample code snippets and a database. In *Proceedings of ICPC*. 41–50.
- [138] C. Parnin and A. Orso. 2011. Are automated debugging techniques actually helping programmers? In *Proceedings of ISSA*. 199–209.
- [139] J. W. Paulson, G. Succi, and A. Eberlein. 2004. An empirical study of open-source and closed-source software products. *TSE* 30, 4 (2004), 246–256.
- [140] F. Pérez, J. Font, L. Arcega, and C. Cetina. 2018. Automatic query reformulations for feature location in a model-based family of software products. *DKE* 116 (2018), 159–176.
- [141] F. Pérez, J. Font, L. Arcega, and C. Cetina. 2019. Collaborative feature location in models through automatic query expansion. *AUSE* 26, 1 (2019), 161–202.
- [142] M. Petticrew and H. Roberts. 2005. *Systematic Reviews in the Social Sciences: A Practical Guide*. Wiley.
- [143] M. F. Porter. 1997. An algorithm for suffix stripping. *Program* 40, 3 (1997), 313–316.
- [144] M. F. Porter. 2001. Snowball: A Language for Stemming Algorithms. Retrieved July 10, 2023 from <http://snowball.tartarus.org/texts/introduction.html>.
- [145] F. Pérez, A. C. Marcén, R. Lapeña, and C. Cetina. 2020. Evaluating low-cost in internal crowdsourcing for software engineering: The case of feature location in an industrial environment. *IEEE Access* 8 (2020), 65745–65757.
- [146] F. Pérez, T. Ziadi, and C. Cetina. 2022. Utilizing automatic query reformulations as genetic operations to improve feature location in software models. *TSE* 48, 2 (2022), 713–731.
- [147] D. Qiu, B. Li, S. Ji, and H. Leung. 2014. Regression testing of web service: A systematic mapping study. *ACM Comput. Surv.* 47, 2 (2014), Article 21, 46 pages.
- [148] M. Raghothaman, Y. Wei, and Y. Hamadi. 2016. SWIM: Synthesizing what I mean: Code search and idiomatic snippet synthesis. In *Proceedings of ICSE*. 357–367.
- [149] M. M. Rahman, J. Barson, S. Paul, J. Kayani, F. A. Lois, S. F. Quezada, C. Parnin, K T. Stolee, and Baishakhi Ray. 2018. Evaluating how developers use general-purpose web-search for code retrieval. In *Proceedings of MSR*. 465–475.
- [150] M. M. Rahman, F. Khomh, and M. Castelluccio. 2020. Why are some bugs non-reproducible? An empirical investigation using data fusion. In *Proceedings of ICSME*. 12.
- [151] M. M. Rahman, F. Khomh, S. Yeasmin, and C. K. Roy. 2021. The forgotten role of search queries in IR-based bug localization: An empirical study. *EMSE* 26 (2021), 116.
- [152] M. M. Rahman and C. K. Roy. 2014. On the use of context in recommending exception handling code examples. In *Proceedings of SCAM*. 285–294.
- [153] M. M. Rahman and C. K. Roy. 2015. TextRank based search term identification for software change tasks. In *Proceedings of SANER*. 540–544.

- [154] M. M. Rahman and C. K. Roy. 2016. QUICKAR: Automatic query reformulation for concept location using crowd-sourced knowledge. In *Proceedings of ASE*. 220–225.
- [155] M. M. Rahman and C. K. Roy. 2017. Improved query reformulation for concept location using CodeRank and document structures. In *Proceedings of ASE*. 428–439.
- [156] M. M. Rahman and C. K. Roy. 2017. STRICT: Information retrieval based search term identification for concept location. In *Proceedings of SANER*. 79–90.
- [157] M. M. Rahman, C. K. Roy, and D. Lo. 2018. Effective reformulation of query for code search using crowdsourced knowledge and extra-large data analytics. In *Proceedings of ICSME*. 516–527.
- [158] M. M. Rahman and C. K. Roy. 2018. Improving IR-based bug localization with context-aware query reformulation. In *Proceedings of ESEC/FSE*. 621–632.
- [159] M. M. Rahman, C. K. Roy, and D. Lo. 2016. RACK: Automatic API recommendation using crowdsourced knowledge. In *Proceedings of SANER*. 349–359.
- [160] M. M. Rahman, C. K. Roy, and D. Lo. 2018. Automatic query reformulation for code search using crowdsourced knowledge. *EMSE* 24 (2018), 1869–1924.
- [161] Peter C. Rigby, Daniel M. German, Laura Cowen, and Margaret-Anne Storey. 2014. Peer review on open-source software projects: Parameters, statistical models, and theory. *TOSEM* 23, 4 (2014), 1–33.
- [162] S. E. Robertson. 1991. On term selection for query expansion. *J. Doc.* 46, 4 (1991), 359–364.
- [163] J. J. Rocchio. 1971. Relevance feedback in information retrieval. In *The SMART Retrieval System—Experiments in Automatic Document Processing*, G. Salton (Ed.). Prentice Hall, Upper Saddle River, NJ, 313–323.
- [164] M. Roldan-Vega, G. Mallet, E. Hill, and J. A. Fails. 2013. CONQUER: A tool for NL-based query refinement and contextualizing code search results. In *Proceedings of ICSM*. 512–515.
- [165] Julia Rubin and Marsha Chechik. 2013. A survey of feature location techniques. In *Domain Engineering, Product Lines, Languages, and Conceptual Models*, Iris Reinhartz-Berger, Arnon Sturm, Tony Clark, Sholom Cohen, and Jorn Bettin (Eds.). Springer, 29–58.
- [166] C. Sadowski, K. T. Stolee, and S. Elbaum. 2015. How developers search for code: A case study. In *Proceedings of ESEC/FSE*. 191–201.
- [167] R. K. Saha, J. Lawall, S. Khurshid, and D. E. Perry. 2014. On the effectiveness of information retrieval based bug localization for C programs. In *Proceedings of ICSME*. 161–170.
- [168] R. K. Saha, M. Lease, S. Khurshid, and D. E. Perry. 2013. Improving bug localization using structured information retrieval. In *Proceedings of ASE*. 345–355.
- [169] G. Salton and C. Buckley. 1997. Improving retrieval performance by relevance feedback. In *Readings in Information Retrieval*, Karen Sparck Jones and Peter Willet (Eds.). Morgan Kaufmann, San Francisco, CA, 355–364.
- [170] G. Salton and M. J. McGill. 1986. *Introduction to Modern Information Retrieval*. McGraw-Hill Computer Science Series. McGraw-Hill, New York, NY.
- [171] G. Salton, A. Wong, and C. S. Yang. 1975. A vector space model for automatic indexing. *Commun. ACM* 18, 11 (1975), 613–620.
- [172] A. Satter and K. Sakib. 2016. A search log mining based query expansion technique to improve effectiveness in code search. In *Proceedings of ICCIT*. 586–591.
- [173] T. Savage, M. Reville, and D. Poshyvanyk. 2010. FLAT3: Feature location and textual tracing tool. In *Proceedings of ICSE*. 255–258.
- [174] G. Scanniello, A. Marcus, and D. Pascale. 2015. Link analysis algorithms for static concept location: An empirical assessment. *EMSE* 20, 6 (2015), 1666–1720.
- [175] H. A. Shafiq and Z. Arshad. 2014. *Automated Debugging and Bug Fixing Solutions: A Systematic Literature Review and Classification*. Master's thesis. Blekinge Institute of Technology.
- [176] D. Shepherd, Z. P. Fry, E. Hill, L. Pollock, and K. Vijay-Shanker. 2007. Using natural language program analysis to locate and understand action-oriented concerns. In *Proceedings of ASOD*. 212–224.
- [177] David J. Sheskin. 2007. *Handbook of Parametric and Nonparametric Statistical Procedures* (4th ed.). Chapman & Hall/CRC, Boca Raton, FL.
- [178] Z. Shi, J. Keung, and Q. Song. 2014. An empirical study of BM25 and BM25F based feature location techniques. In *Proceedings of InnoSWDev*. 106–114.
- [179] A. Shtok, O. Kurland, D. Carmel, F. Raiber, and G. Markovits. 2012. Predicting query performance by query-drift estimation. *TOIS* 30, 2 (2012), Article 11, 35 pages.
- [180] R. Sirres, T. F. Bissyandé, D. Kim, D. Lo, J. Klein, K. Kim, and Y. L. Traon. 2018. Augmenting and structuring user queries to support efficient free-form code search. *EMSE* 23 (2018), 2622–2654.
- [181] Bunyamin Sisman, Shayan A. Akbar, and Avinash C. Kak. 2017. Exploiting spatial code proximity and order for improved source code retrieval for bug localization. *JSEP* 29, 1 (2017), e1805.

- [182] B. Sisman and A. C. Kak. 2012. Incorporating version histories in information retrieval based bug localization. In *Proceedings of MSR*. 50–59.
- [183] B. Sisman and A. C. Kak. 2013. Assisting code search with automatic query reformulation for bug localization. In *Proceedings of MSR*. 309–318.
- [184] G. Sridhara, E. Hill, L. Pollock, and K. Vijay-Shanker. 2008. Identifying word relations in software: A comparative study of semantic similarity tools. In *Proceedings of ICPC*. 123–132.
- [185] K. Stol, P. Ralph, and B. Fitzgerald. 2016. Grounded theory in software engineering research: A critical review and guidelines. In *Proceedings of ICSE*. 120–131.
- [186] K. T. Stolee, S. Elbaum, and D. Dobos. 2014. Solving the search for source code. *TOSEM* 23, 3 (2014), Article 26, 45 pages.
- [187] F. Thung, D. Lo, and J. Lawall. 2013. Automated library recommendation. In *Proceedings of WCRE*. 182–191.
- [188] F. Thung, S. Wang, D. Lo, and J. Lawall. 2013. Automatic recommendation of API methods from feature requests. In *Proceedings of ASE*. 290–300.
- [189] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *JMLR* 9, 11 (2008), 1–27.
- [190] C. Vassallo, S. Panichella, M. Di Penta, and G. Canfora. 2014. CODES: Mining source code descriptions from developers discussions. In *Proceedings of ICPC*. 106–109.
- [191] V. Vinayakara, A. Sarma, R. Purandare, S. Jain, and S. Jain. 2017. ANNE: Improving source code search using entity retrieval approach. In *Proceedings of WSDM*. 211–220.
- [192] J. Wang, X. Peng, Z. Xing, and W. Zhao. 2013. Improving feature location practice with multi-faceted interactive exploration. In *Proceedings of ICSE*. 762–771.
- [193] Q. Wang, C. Parnin, and A. Orso. 2015. Evaluating the usefulness of IR-based fault localization techniques. In *Proceedings of ISSTA*. 1–11.
- [194] S. Wang and D. Lo. 2014. Version history, similar report, and structure: Putting them together for improved bug localization. In *Proceedings of ICPC*. 53–63.
- [195] S. Wang and D. Lo. 2016. AmaLgam+: Composing rich information sources for accurate bug localization. *JSEP* 28, 10 (2016), 921–942.
- [196] S. Wang, D. Lo, and L. Jiang. 2014. Active code search: Incorporating user feedback to improve code search relevance. In *Proceedings of ASE*. 677–682.
- [197] S. Wang, D. Lo, and L. Jiang. 2016. AutoQuery: Automatic construction of dependency queries for code search. *ASE* 23, 3 (Sept. 2016), 393–425.
- [198] T. Wei, Y. Lu, H. Chang, Q. Zhou, and X. Bao. 2015. A semantic approach for text clustering using WordNet and lexical chains. *Expert Syst. Appl.* 42, 4 (2015), 2264–2275.
- [199] M. Wen, R. Wu, and S. C. Cheung. 2016. Locus: Locating bugs from software changes. In *Proceedings of ASE*. 262–273.
- [200] L. A. Wilson. 2010. Using ontology fragments in concept location. In *Proceedings of ICSM*. 1–2.
- [201] C. Wohlin, P. Runeson, M. Hst, M. C. Ohlsson, B. Regnell, and A. Wessln. 2012. *Experimentation in Software Engineering*. Springer.
- [202] C. P. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei. 2014. Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis. In *Proceedings of ICSME*. 181–190.
- [203] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa. 2016. A survey on software fault localization. *TSE* 42, 8 (2016), 707–740.
- [204] H. Wu and Y. Yang. 2019. Code search based on alteration intent. *IEEE Access* 7 (2019), 56796–56802.
- [205] M. Wursch, G. Ghezzi, G. Reif, and H. C. Gall. 2010. Supporting developers with natural language queries. In *Proceedings of ICSE*. 165–174.
- [206] J. Yang and L. Tan. 2012. Inferring semantically related words from software context. In *Proceedings of MSR*. 161–170.
- [207] J. Yang and L. Tan. 2014. SWordNet: Inferring semantically related words from software context. *EMSE* 19, 6 (2014), 1856–1886.
- [208] X. Ye, H. Shen, X. Ma, R. Bunescu, and C. Liu. 2016. From word embeddings to document similarities for improved information retrieval in software engineering. In *Proceedings of ICSE*. 404–415.
- [209] K. C. Youm, J. Ahn, J. Kim, and E. Lee. 2015. Bug localization based on code change histories and bug reports. In *Proceedings of APSEC*. 190–197.
- [210] H. Yu, W. Song, and T. Mine. 2016. APIBook: An effective approach for finding APIs. In *Proceedings of Internetware*. 45–53.
- [211] T. Yuan, D. Lo, and J. Lawall. 2014. Automated construction of a software-specific word similarity database. In *Proceedings of CSMR-WCRE*. 44–53.
- [212] S. Zamani, S. Peck Lee, R. Shokripour, and J. Anvik. 2014. A noun-based approach to feature location using time-aware term-weighting. *Inf. Softw. Technol.* 56, 8 (2014), 991–1011.

- [213] F. Zhang, H. Niu, I. Keivanloo, and Y. Zou. 2018. Expanding queries for code search using semantically related API class-names. *TSE* 44, 11 (2018), 1070–1082.
- [214] Jie Zhang, XiaoYin Wang, Dan Hao, Bing Xie, Lu Zhang, and Hong Mei. 2015. A survey on bug-report analysis. *SCIS* 58, 2 (2015), 1–24.
- [215] W. Zhang, Z. Li, Q. Wang, and J. Li. 2019. FineLocator: A novel approach to method-level fine-grained bug localization by query expansion. *IST* 110 (2019), 121–135.
- [216] Y. Zhang, D. Lo, X. Xia, G. Scanniello, T. B. Le, and J. Sun. 2018. Fusing multi-abstraction vector space models for concern localization. *EMSE* 23, 4 (2018), 2279–2322.
- [217] J. Zhou, H. Zhang, and D. Lo. 2012. Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports. In *Proceedings of ICSE*. 14–24.
- [218] W. Zou, D. Lo, Z. Chen, X. Xia, Y. Feng, and B. Xu. 2020. How practitioners perceive automated bug report management techniques. *TSE* 46, 8 (2020), 836–862.

Received 14 September 2021; revised 9 October 2022; accepted 23 May 2023